TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Teoría de la Programación
Docente:	José Oswaldo Guamán Quinche
Estudiante(s):	Daniela Loreley Alejandro Lozano
Curso / Paralelo:	1A
Fecha de entrega:	Miércoles 4 de noviembre del 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

Enunciado del algoritmo

EJ-056 (Ejercicio 1)

En una cooperativa, construya un algoritmo que lea una fecha: día (entero), mes (entero) y año (entero). Use constantes simbólicas DIA_MAX = 31, MES_MAX = 12, AÑO_MIN = 1900, AÑO_MAX = 2025. Evalúe la expresión lógica compuesta (día >= 1 AND día <= DIA_MAX AND mes >= 1 AND mes <= MES_MAX AND año >= AÑO_MIN AND año <= AÑO_MAX). Asigne el resultado a fecha_valida (booleano). Calcule la edad aproximada con (2025 - año). Muestre por pantalla la validación y la edad. Además, calcule el valor mínimo de todas las variables utilizadas. Identifique explícitamente los datos de entrada y la información de salida.

Análisis del problema

Datos de entrada:

- **Constantes**
  dia_max = 31
  mes_max = 12
  anio_min = 1900
  anio_max = 2025
  anio_actual = 2025
- **Variables**: dia, mes, anio

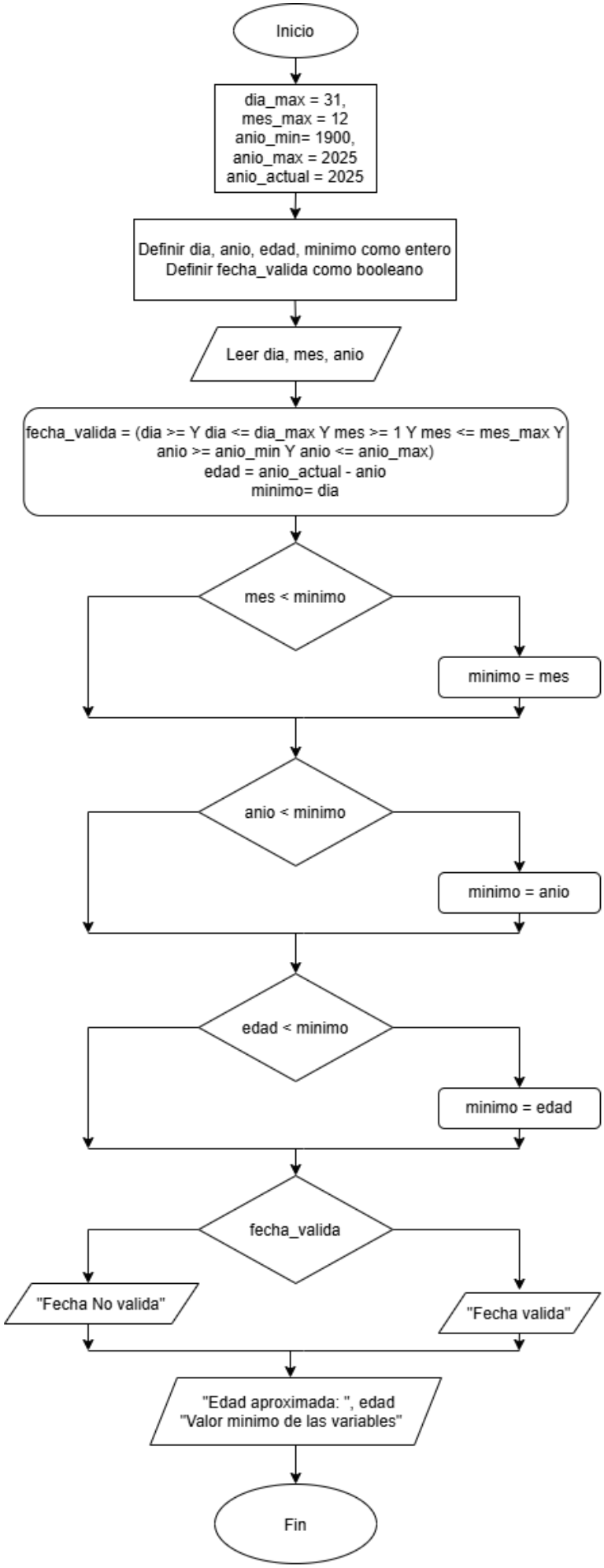
Proceso

- **fecha_valida** = (dia >= 1 AND dia <= dia_max AND mes >= 1 AND mes <= mes_max AND anio >= anio_min AND anio <= anio_max)
- **edad** = 2025 – anio
- **minimo** = min (dia, mes, año, dia_max, mes_max, anio_min, anio_max, fecha_valida, edad)

Datos de salida:

- fecha_valida
- edad
- mínimo

Diagrama de flujo



Pseudocódigo

```
Inicio
//variables y constantes
Constante dia_max = 31
Constante mes_max = 12
Constante anio_min = 1900
Constante anio_max = 2025
Constante anio_actual = 2025
Entero dia, mes, año, edad, minimo
Booleano fecha_valida
//pedir información
Leer dia
Leer mes
Leer año
//proceso
fecha_valida = (dia >= Y dia <= dia_max Y mes >= 1 Y mes <= mes_max Y anio >=
anio_min Y anio <= anio_max)
edad = anio_actual – anio
minimo = dia
Si (mes < minimo) Entonces {
        minimo = mes
} FinSi
Si (anio < minimo) Entonces {
        minimo = anio
} FinSi
Si (edad < minimo) Entonces {
        minimo = edad
} FinSi
Escribir ("¿Fecha válida? " + fecha_valida)
Escribir (“Edad aproximada: " + edad)
Escribir ("Valor mínimo de todas las variables: " + minimo)
Fin
```

Prueba de escritorio

Prueba 1

Paso	dia	mes	anio	Evaluación Expresión Lógica
1	12	5	2020	---
2	12	5	2020	Verdadero (true)

Paso	edad	Comparación valor_minimo	valor_minimo	Salida de pantalla
3	2025 -2020	Inicializa con dia (12)	12	

	= 5			
4	5	¿mes (5) < valor_minimo (12)? → **Verdadero**	5	
5	5	¿anio (2020) < valor_minimo (5)? → **Falso**	5	
6	5	¿edad (5) < valor_minimo (5)? → **Falso**	5	
7	5	---	5	**Fecha valida:** Verdadero **Edad aproximada**: 5 **Valor minimo de las variables utilizadas:** 1

Prueba 2

Paso	dia	mes	anio	Evaluación Expresión Lógica
1	20	5	2027	---
2	20	5	2027	Falso (false)

Paso	edad	Comparación valor_minimo	valor_minimo	Salida de pantalla
3	2025 -2027 = -2	Inicializa con dia (20)	12	
4	5	¿mes (5) < valor_minimo (20)? → **Verdadero**	5	
5	5	¿anio (2027) < valor_minimo (5)? → **Falso**	5	
6	5	¿edad (−2) < valor_minimo (5)? → **Verdadero**	5	
7	5	---	5	**Fecha valida:** Falso **Edad aproximada**: -2 **Valor minimo de las variables utilizadas:** -2

Codificación en lenguaje C

```c
int main() {
    //Constantes y variables
    const int dia_max = 31;
    const int mes_max = 12;
    const int anio_min = 1900;
    const int anio_max = 2025;
    const int anio_actual = 2025;
    int dia, mes, anio, edad, minimo;
    bool fecha_valida;
    //Pedir datos
    printf("Ingrese el dia: ");
    scanf("%d", &dia);
    printf("Ingrese el mes: ");
    scanf("%d", &mes);
    printf("Ingrese el anio: ");
    scanf("%d", &anio);

    //Proceso
    fecha_valida = (dia >= 1 && dia <= dia_max &&
                    mes >= 1 && mes <= mes_max &&
                    anio >= anio_min && anio <= anio_max);

    edad = anio_actual - anio;
    minimo = dia;
    if (mes < minimo) minimo = mes;
    if (anio < minimo) minimo = anio;
    if (edad < minimo) minimo = edad;
    // Salida
    printf("Fecha valida? %s\n", fecha_valida ? "true" : "false");
    printf("Edad aproximada: %d\n", edad);
    printf("Valor minimo de todas las variables: %d\n", minimo);

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ1_056.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> gcc EJ_056.c -o ejer056.exe
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer056.exe
Ingrese el dia: 12
Ingrese el mes: 5
Ingrese el anio: 2020
Fecha valida? true
Edad aproximada: 5
Valor minimo de todas las variables: 5
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer056.exe
Ingrese el dia: 20
Ingrese el mes: 5
Ingrese el anio: 2027
Fecha valida? false
Edad aproximada: -2
Valor minimo de todas las variables: -2
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- Cumple lo que pide el enunciado. Lee tres datos enteros, valida la fecha con una sola expresión lógica compuesta, calcula la edad aproximada y muestra los resultados. Todo es secuencial y usa solo operadores relacionales y lógicos, que es lo que se practica en este tipo de ejercicio.

- Las constantes simbólicas lo hacen fácil de mantener. Si el rango de años cambia, se modifica una sola línea y la expresión sigue funcionando. Con números escritos directamente habría que buscarlos por todo el código.

Enunciado del algoritmo

EJ-057 (Ejercicio 2)

En un restaurante, diseñe un algoritmo que lea un monto en dólares (real) y lo convierta a euros, pesos y soles. Use contantes simbólicas USD_EUR = 0.92, USD_MXN = 17.50, USD_PEN = 3.75. Calcule cada conversión con expresiones aritméticas. Evalué la expresión lógica (monto > 0 AND monto <= 10000). Asigne los resultados a variables reales. Muestre por pantalla todas las conversiones. Justifique el uso de constantes. Además, calcule el valor mínimo de todas las variables utilizadas. Asegúrese de inicializar todas las variables antes de usarlas.

Análisis del problema

Datos de entrada:

- **Constantes**:
- Usd_eur = 0.92
- Usd_mxn = 17.50
- Usd_pen = 3.75
- **Variables**: monto

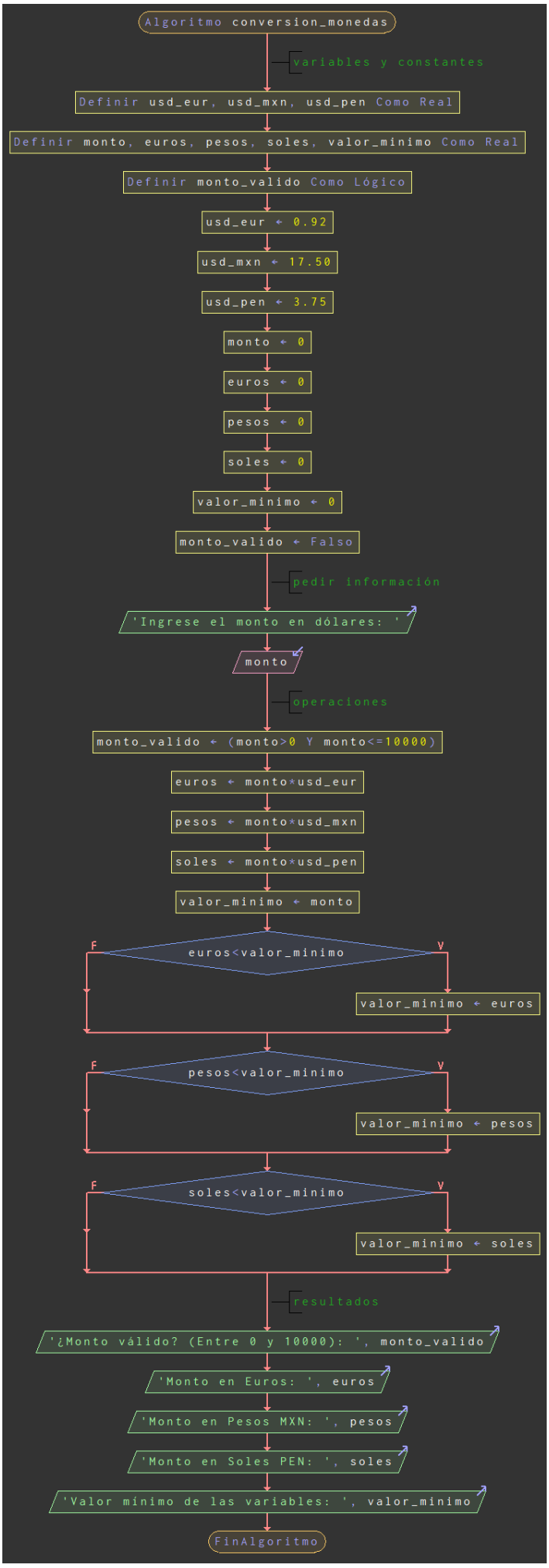
Proceso

- euros = monto * usd_eur
- pesos = monto * usd_mxn
- soles = monto * usd_pen

Datos de salida:

- euros
- Pesos
- Soles
- monto_valido
- valor_minimo

Diagrama de flujo



Pseudocódigo

```
Inicio
//variables y constantes
usd_eur = 0.92
usd_mxn = 17.50
usd_pen = 3.75
Real monto = 0.0
Real euros = 0.0
Real pesos = 0.0
Real soles = 0.0
Real valor_minimo = 0.0
Booleano monto_valido = Falso
//pedir información
Escribir "Ingrese el monto en dólares: "
Leer monto
//operaciones
monto_valido = (monto > 0 Y monto <= 10000)
euros <- monto * usd_eur
pesos <- monto * usd_mxn
soles <- monto * usd_pen
valor_minimo = monto
Si (euros < valor_minimo) Entonces {
        valor_minimo = euros
} FinSi
Si (pesos < valor_minimo) Entonces {
        valor_minimo = pesos
} FinSi
Si (soles < valor_minimo) Entonces {
        valor_minimo = soles
} FinSi
Escribir ("¿Monto valido? (Entre 0 y 10000): " + monto_valido)
Escribir ("Monto en Euros: " + euros)
Escribir ("Monto en Pesos MXN: " + pesos)
Escribir ("Monto en Soles PEN: " + soles)
Escribir ("Valor minimo de las variables: " + valor_minimo)
Fin
```

Prueba de escritorio

Prueba 1

Paso	monto	monto_valido	euros	pesos	soles	comparacion
1	20	False	0.0	0.0	0.0	---
2	20	true	0.0	0.0	0.0	$20 > 0 \land 20 \leq$

						10000 → **Verdadero**
3	20	True	18.4	350	75	Asigna conversiones
4	20	True	18.4	350	75	Inicializa con monto (20.0)
5	20	True	18.4	350	75	¿euros (18.4) < minimo (20.0)? → **Sí**
6	20	True	18.4	350	75	¿pesos (350.0) < minimo (18.4)? → **No**
7	20	True	18.4	350	75	¿soles (75.0) < minimo (18.4)? → **No**

Pasos	minimo	Salida en Pantalla
8	18.4	**¿Monto valido?** Verdadero (1) **Euros (EUR):** 18.40 **Pesos (MXN):** 350.00 **Soles (PEN):** 75.00 **Valor minimo de las variables reales**: 18.40

Prueba 2

Paso	monto	monto_valido	euros	pesos	soles	comparacion
1	35000	False	0.0	0.0	0.0	---
2	35000	False	0.0	0.0	0.0	$35000 > 0 \wedge 35000 \leq 10000 \rightarrow$ ***Falso***
3	35000	False	32200	612500	131250	Asigna conversiones
4	35000	False	32200	612500	131250	Inicializa con monto (35000.0)
5	35000	False	32200	612500	131250	¿euros (32200.0) <

						minimo (35000.0)? → **Sí**
6	35000	False	32200	612500	131250	¿pesos (612500.0) < minimo (32200.0)? → **No**
7	35000	False	32200	612500	131250	¿soles (131250.0) < minimo (32200.0)? → **No**

Pasos	minimo	Salida en Pantalla
8	32200	**Monto valido?** Falso (0) **Euros (EUR):** 32200.00 **Pesos (MXN):** 612500.00 **Soles (PEN):** 131250.00 **Valor minimo de las variables reales:** 32200.00

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
int main() {
    //constantes y variables
    const float usd_eur = 0.92;
    const float usd_mxn = 17.50;
    const float usd_pen = 3.75;
    float monto = 0.0;
    float euros = 0.0;
    float pesos = 0.0;
    float soles = 0.0;
    float valor_minimo = 0.0;
    bool monto_valido = false;
    //Pedir datos
    printf("Ingrese el monto en dolares: ");
    scanf("%f", &monto);
    //operaciones
    monto_valido = (monto > 0.0 && monto <= 10000.0);
    euros = monto * usd_eur;
    pesos = monto * usd_mxn;
    soles = monto * usd_pen;
    valor_minimo = monto;
    if (euros < valor_minimo) {
        valor_minimo = euros;
    }
    if (pesos < valor_minimo) {
        valor_minimo = pesos;
    }
    if (soles < valor_minimo) {
        valor_minimo = soles;
    }
    //resultados
    printf("Monto en dolares: %.2f\n", monto);
    printf("Euros:            %.2f\n", euros);
    printf("Pesos mexicanos:  %.2f\n", pesos);
    printf("Soles peruanos:   %.2f\n", soles);
    printf("Monto valido?     %s\n", monto_valido ? "true" : "false");
    printf("Valor minimo:     %.2f\n", valor_minimo);
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ2_057.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer057.exe
Ingrese el monto en dolares: 20
Monto en dolares: 20.00
Euros:            18.40
Pesos mexicanos:  350.00
Soles peruanos:   75.00
Monto valido?     true
Valor minimo:     18.40
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer057.exe
Ingrese el monto en dolares: 35000
Monto en dolares: 35000.00
Euros:            32200.00
Pesos mexicanos:  612500.00
Soles peruanos:   131250.00
Monto valido?     false
Valor minimo:     32200.00
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El programa permite realizar la conversión directa de una suma en dólares a tres divisas distintas (euros, pesos mexicanos y soles peruanos) utilizando operaciones matemáticas simples y multiplicando por tasas fijas. Además, comprueba mediante una condición lógica si la cifra ingresada es válida, es decir, si se encuentra en un rango mayor a cero y no supera los 10,000 dólares.

- Por otro lado, el algoritmo garantiza un cálculo seguro del valor mínimo al comparar únicamente las magnitudes numéricas reales (el monto original y sus conversiones) y excluir las constantes y la variable lógica. De esta manera se evita el error común donde los valores booleanos distorsionan el resultado final de la comparación.

Enunciado del algoritmo

EJ-058 (Ejercicio 3)

En una panadería, cree un algoritmo que simule un sistema de evaluación de crédito. Lea el ingreso mensual (real), los gastos (real), el monto solicitado (real) y el plazo en meses (entero). Use constantes simbólicas INGRESO_MIN = 800, PORCENTAJE_MAX = 0.30, TASA_INTERES = 0.18. Calcule: ingreso_neto = ingreso – gastos; capacidad_pago = ingreso_neto * PORCENTAJE_MAX; pago_mensual = (monto*(1+ TASA_INTERES)) /plazo. Evalué la expresión lógica (ingreso >= INGRESO_MIN AND pago_mensual <= capacidad_pago). Asigne el resultado a crédito_aprobado. Muestre por pantalla el análisis completo. Además, calcule el valor mínimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones

Análisis del problema

Datos de entrada:

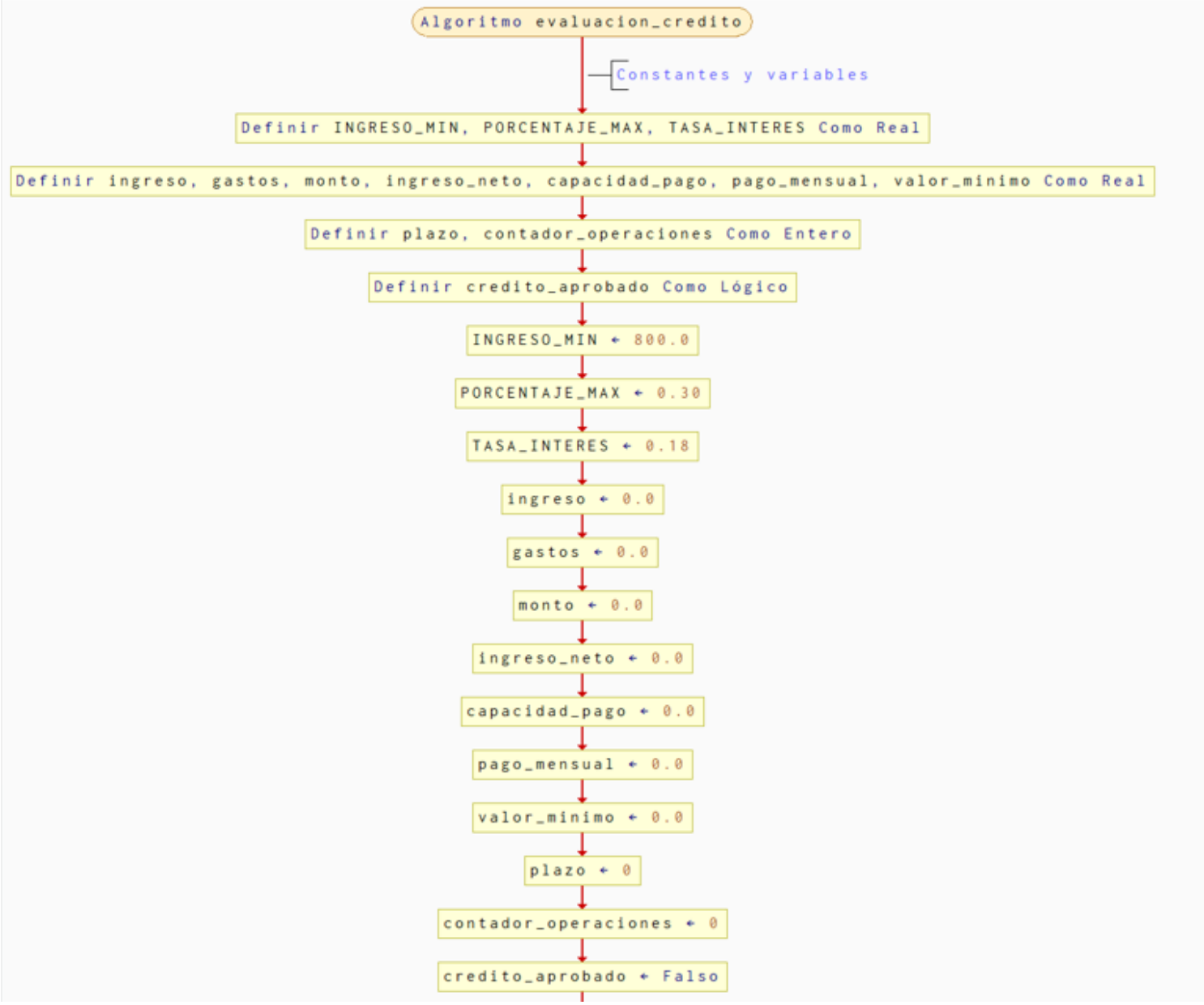
- **Constantes**:
  - INGRESO_MIN = 800
  - PORCENTAJE_MAX = 0.30
  - TASA_INTERES = 0.18.
- **Variables**: ingreso, gastos, monto, plazo

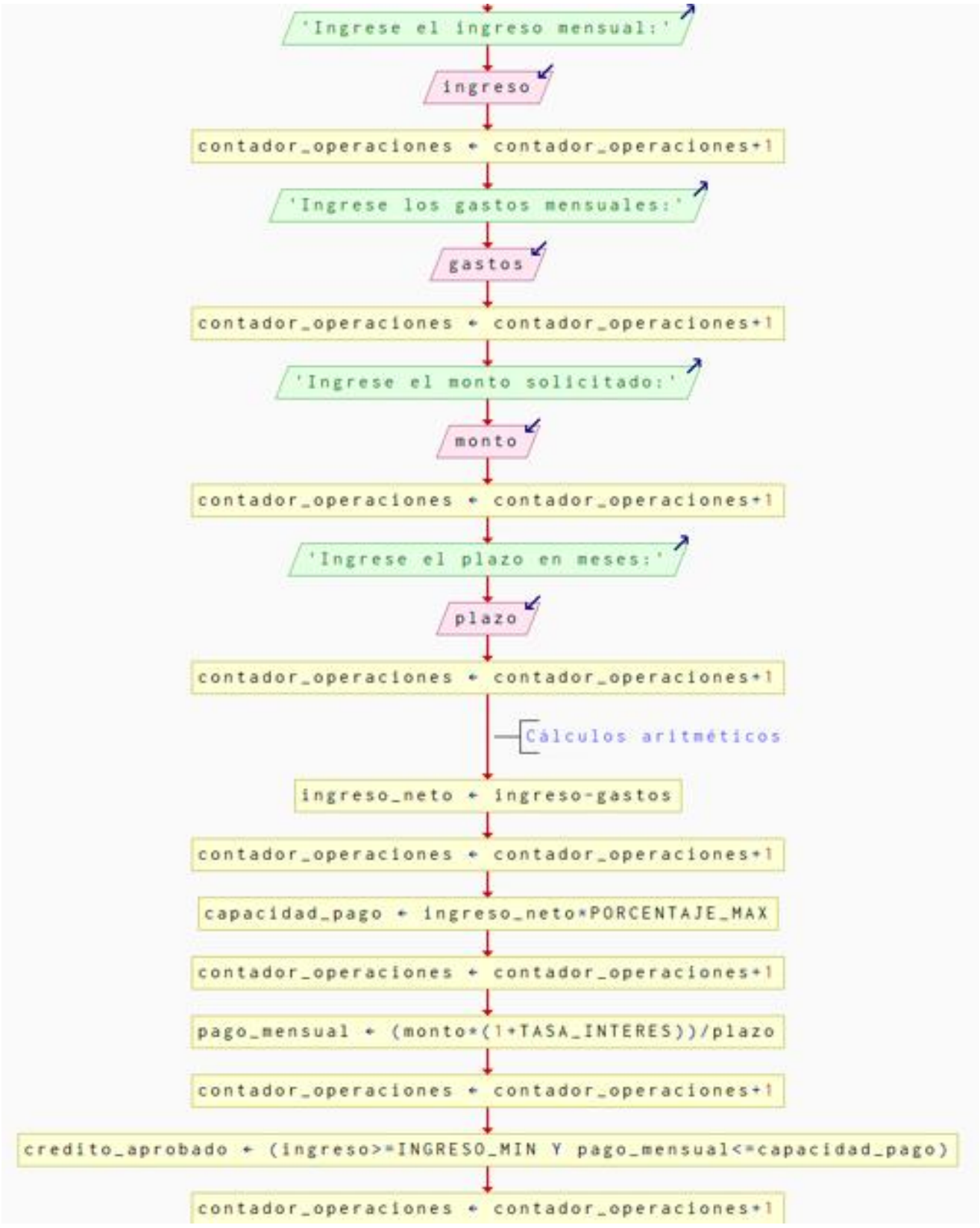
Proceso

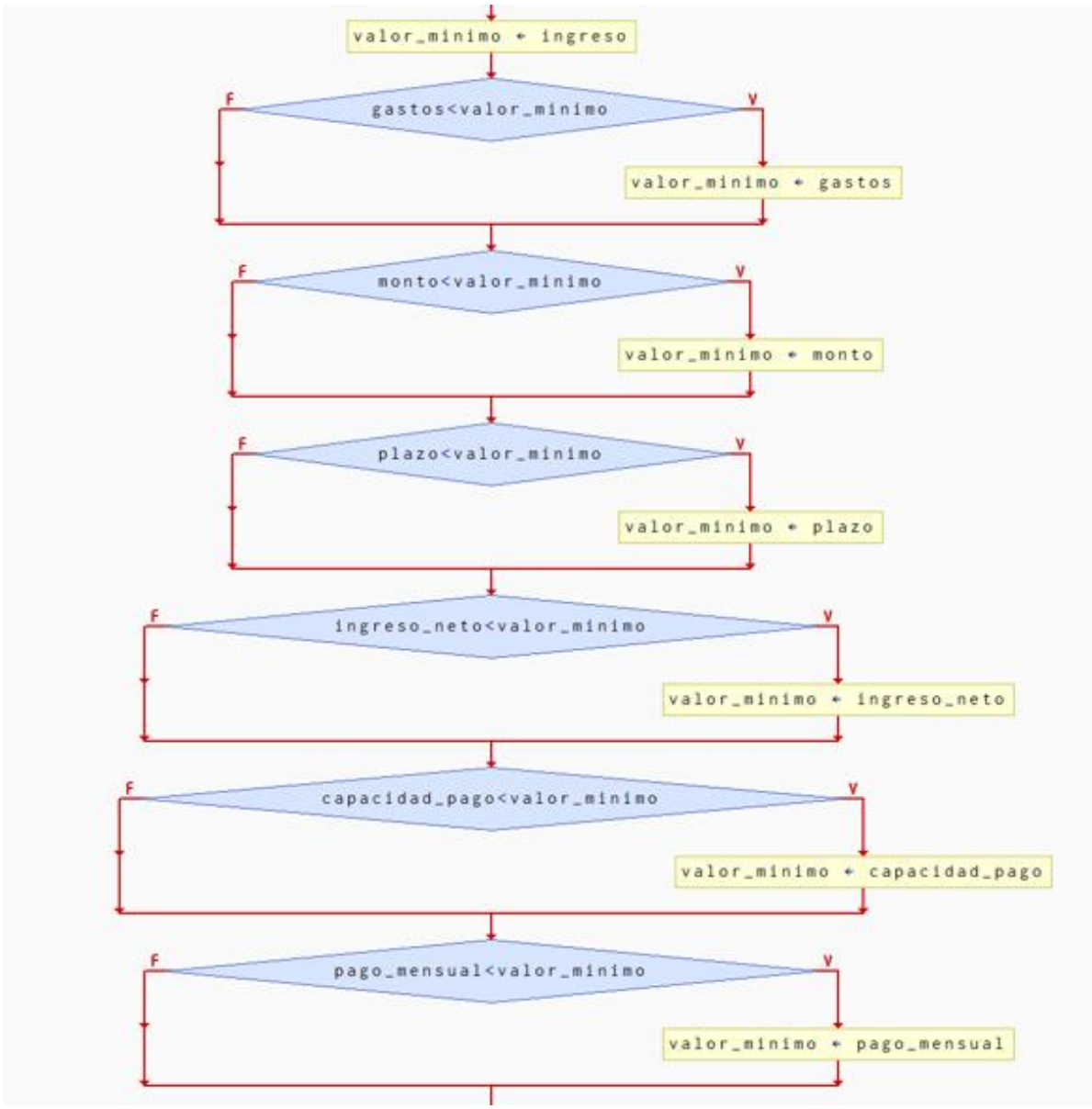
Datos de salida

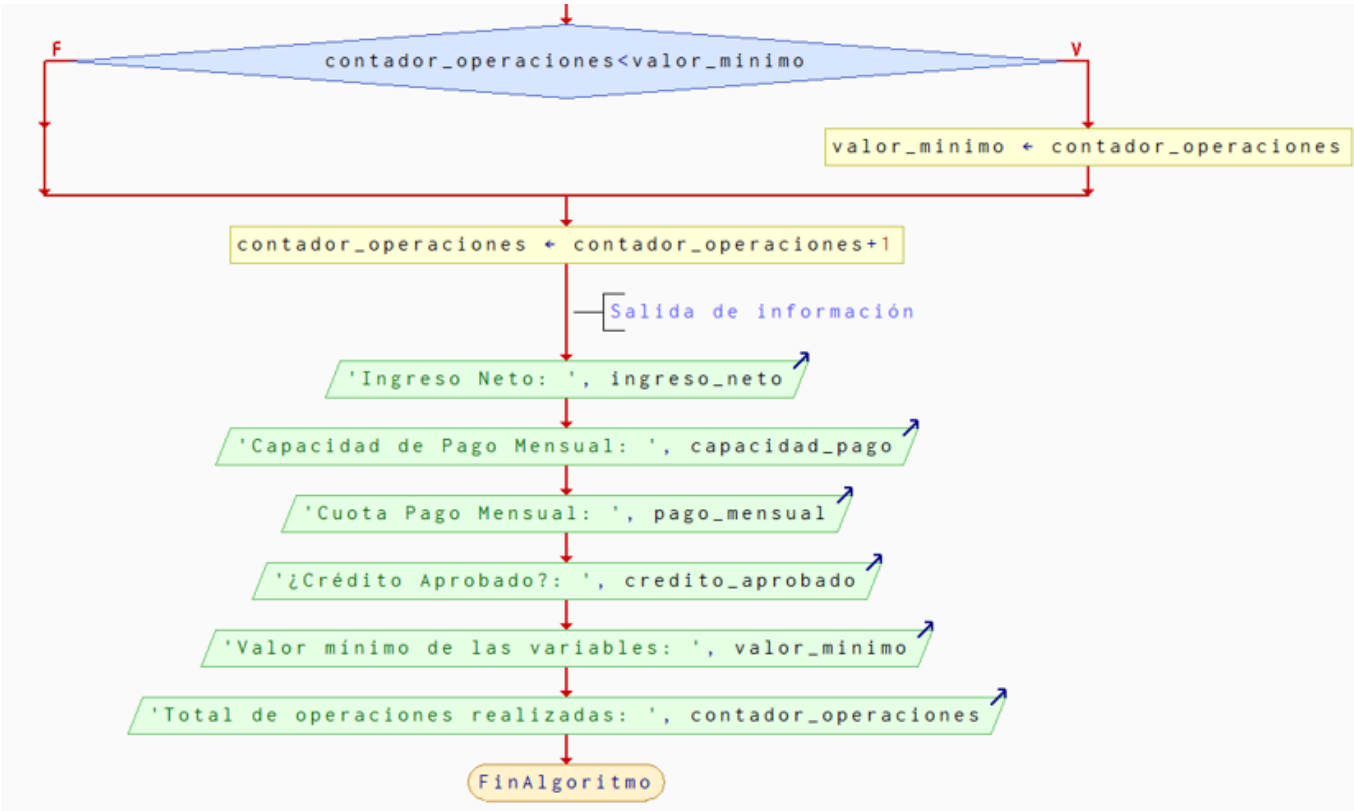
- Ingreso_neto
- Capacidad_pago
- Pago_mensual
- Crédito_aprobado
- Mínimo
- contador

Diagrama de flujo




'Ingrese el ingreso mensual:'
ingreso
contador_operaciones ← contador_operaciones+1
'Ingrese los gastos mensuales:'
gastos
contador_operaciones ← contador_operaciones+1
'Ingrese el monto solicitado:'
monto
contador_operaciones ← contador_operaciones+1
'Ingrese el plazo en meses:'
plazo
contador_operaciones ← contador_operaciones+1
Cálculos aritméticos
ingreso_neto ← ingreso-gastos
contador_operaciones ← contador_operaciones+1
capacidad_pago ← ingreso_neto*PORCENTAJE_MAX
contador_operaciones ← contador_operaciones+1
pago_mensual ← (monto*(1+TASA_INTERES))/plazo
contador_operaciones ← contador_operaciones+1
credito_aprobado ← (ingreso>=INGRESO_MIN Y pago_mensual<=capacidad_pago)
contador_operaciones ← contador_operaciones+1


valor_minimo ← ingreso
gastos<valor_minimo
F
V
valor_minimo ← gastos
monto<valor_minimo
F
V
valor_minimo ← monto
plazo<valor_minimo
F
V
valor_minimo ← plazo
ingreso_neto<valor_minimo
F
V
valor_minimo ← ingreso_neto
capacidad_pago<valor_minimo
F
V
valor_minimo ← capacidad_pago
pago_mensual<valor_minimo
F
V
valor_minimo ← pago_mensual


contador_operaciones<valor_minimo
F
V
valor_minimo ← contador_operaciones
contador_operaciones ← contador_operaciones+1
Salida de información
'Ingreso Neto: ', ingreso_neto
'Capacidad de Pago Mensual: ', capacidad_pago
'Cuota Pago Mensual: ', pago_mensual
'¿Crédito Aprobado?: ', credito_aprobado
'Valor mínimo de las variables: ', valor_minimo
'Total de operaciones realizadas: ', contador_operaciones
FinAlgoritmo

Pseudocódigo

```
Inicio
// Constantes y variables
INGRESO_MIN = 800.0
PORCENTAJE_MAX = 0.30
TASA_INTERES = 0.18
Definir ingreso, gastos, monto, ingreso_neto, capacidad_pago, pago_mensual,
valor_minimo Como Real
Definir plazo, contador_operaciones Como Entero
Definir credito_aprobado Como Logico
ingreso = 0.0
gastos = 0.0
monto = 0.0
ingreso_neto = 0.0
capacidad_pago = 0.0
pago_mensual = 0.0
valor_minimo = 0.0
plazo = 0
contador_operaciones = 0
credito_aprobado = Falso
// Lectura de datos
Escribir "Ingrese el ingreso mensual:"
Leer ingreso
contador_operaciones = contador_operaciones + 1
Escribir "Ingrese los gastos mensuales:"
Leer gastos
contador_operaciones = contador_operaciones + 1
Escribir "Ingrese el monto solicitado:"
Leer monto
contador_operaciones = contador_operaciones + 1
Escribir "Ingrese el plazo en meses:"
Leer plazo
contador_operaciones = contador_operaciones + 1
// Cálculos aritméticos
ingreso_neto = ingreso - gastos
contador_operaciones = contador_operaciones + 1
capacidad_pago = ingreso_neto * PORCENTAJE_MAX
contador_operaciones = contador_operaciones + 1
pago_mensual = (monto * (1 + TASA_INTERES)) / plazo
contador_operaciones = contador_operaciones + 1
credito_aprobado = (ingreso >= INGRESO_MIN Y pago_mensual <= capacidad_pago)
contador_operaciones = contador_operaciones + 1
```

```
valor_minimo = ingreso
Si gastos < valor_minimo Entonces
    valor_minimo = gastos
FinSi
Si monto < valor_minimo Entonces
    valor_minimo = monto
FinSi
Si plazo < valor_minimo Entonces
    valor_minimo = plazo
FinSi
Si ingreso_neto < valor_minimo Entonces
    valor_minimo = ingreso_neto
FinSi
Si capacidad_pago < valor_minimo Entonces
    valor_minimo = capacidad_pago
FinSi
Si pago_mensual < valor_minimo Entonces
    valor_minimo = pago_mensual
FinSi
Si contador_operaciones < valor_minimo Entonces
    valor_minimo = contador_operaciones
FinSi
contador_operaciones = contador_operaciones + 1
// Salida de información
Escribir "Ingreso Neto: ", ingreso_neto
Escribir "Capacidad de Pago Mensual: ", capacidad_pago
Escribir "Cuota Pago Mensual: ", pago_mensual
Escribir "¿Credito Aprobado?: ", credito_aprobado
Escribir "Valor minimo de las variables: ", valor_minimo
Escribir "Total de operaciones realizadas: ", contador_operaciones
Fin
```

Prueba de escritorio

Prueba 1

Paso	ingreso	gastos	monto	plazo	Ingreso_neto	Capacidad_pago
1	1500	500	1200	12	1000	300

Paso	Pago_mesual	Ingreso >= 800	Pago <= capacidad	Crédito_aprobado	Contador	Mínimo
1	118	V	V	Verdadero	4	4

Paso	Salida en pantalla
2	**Ingreso Neto:** $1000

	Capacidad de Pago: $300 **Pago Mensual Requerido:** $118 **Credito Aprobado?** Verdadero (Si) **Valor minimo de las variables utilizadas:** 4 **Contador de operaciones realizadas:** 4

Prueba 2

Paso	ingreso	gastos	monto	plazo	Ingreso_neto	Capacidad_pago
1	500	200	100	30	300	90

Paso	Pago_mesual	Ingreso >= 800	Pago <= capacidad	Crédito_aprobado	Contador	Mínimo
1	3.93	F	V	falso	4	3.92

Paso	Salida en pantalla
2	**Ingreso Neto:** $300.00 **Capacidad de Pago:** $90.00 Pago Mensual **Requerido:** $3.93 **Credito Aprobado?** Falso (NO) **Valor minimo de las variables utilizadas:** 3.93 **Contador de operaciones realizadas:** 9

Codificación en lenguaje C

EJ3_058.c > main()

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    // Declaración de variables
    const float INGRESO_MIN = 800.0;
    const float PORCENTAJE_MAX = 0.30;
    const float TASA_INTERES = 0.18;
    float ingreso = 0.0, gastos = 0.0, monto = 0.0;
    float ingreso_neto = 0.0, capacidad_pago = 0.0, pago_mensual = 0.0;
    float valor_minimo = 0.0;
    int plazo = 0, contador_operaciones = 0;
    bool credito_aprobado = false;

    // Lectura de datos
    printf("Ingrese el ingreso mensual: ");
    scanf("%f", &ingreso);
    contador_operaciones++;

    printf("Ingrese los gastos mensuales: ");
    scanf("%f", &gastos);
    contador_operaciones++;

    printf("Ingrese el monto solicitado: ");
    scanf("%f", &monto);
    contador_operaciones++;

    printf("Ingrese el plazo en meses: ");
    scanf("%d", &plazo);
    contador_operaciones++;

```

```c
    // Proceso de cálculo
    ingreso_neto = ingreso - gastos;
    contador_operaciones++;
    capacidad_pago = ingreso_neto * PORCENTAJE_MAX;
    contador_operaciones++;
    pago_mensual = (monto * (1.0f + TASA_INTERES)) / plazo;
    contador_operaciones++;
    // Evaluación lógica
    credito_aprobado = (ingreso >= INGRESO_MIN && pago_mensual <= capacidad_pago);
    contador_operaciones++;
    // Cálculo del mínimo (excluyendo la variable booleana)
    valor_minimo = ingreso;
    if (gastos < valor_minimo) valor_minimo = gastos;
    if (monto < valor_minimo) valor_minimo = monto;
    if (plazo < valor_minimo) valor_minimo = (float)plazo;
    if (ingreso_neto < valor_minimo) valor_minimo = ingreso_neto;
    if (capacidad_pago < valor_minimo) valor_minimo = capacidad_pago;
    if (pago_mensual < valor_minimo) valor_minimo = pago_mensual;
    if (contador_operaciones < valor_minimo) valor_minimo = (float)contador_operaciones;
    contador_operaciones++;

    // Impresión del análisis completo
    printf("Ingreso Neto: $%.2f\n", ingreso_neto);
    printf("Capacidad de Pago: $%.2f\n", capacidad_pago);
    printf("Pago Mensual Requerido: $%.2f\n", pago_mensual);
    printf("Credito Aprobado? %s\n", credito_aprobado ? "Verdadero (SI)" : "Falso (NO)");
    printf("Valor minimo de las variables utilizadas: %.2f\n", valor_minimo);
    printf("Contador de operaciones realizadas: %d\n", contador_operaciones);

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ3_058.c

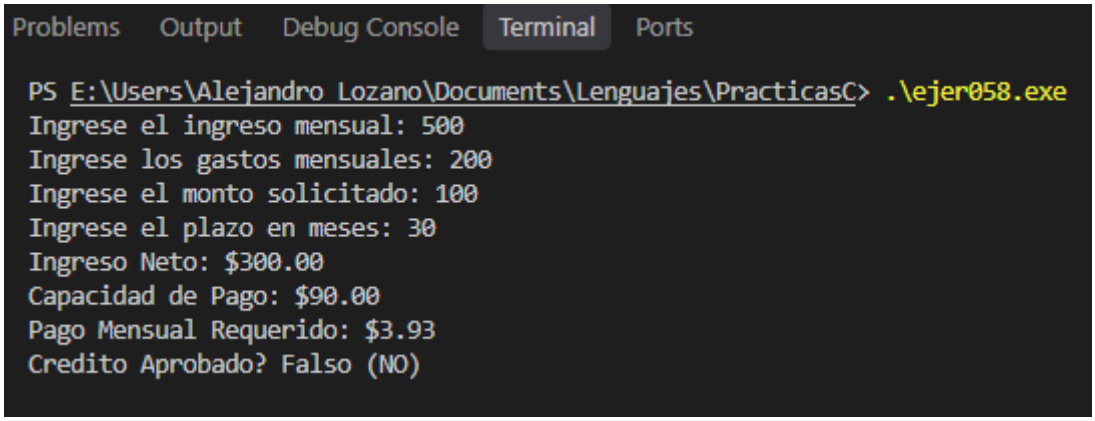
Salida del programa

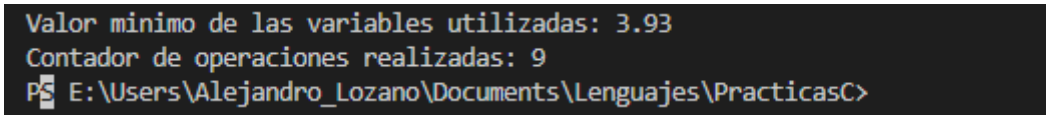
Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer058.exe
Ingrese el ingreso mensual: 1500
Ingrese los gastos mensuales: 500
Ingrese el monto solicitado: 1200
Ingrese el plazo en meses: 12
Ingreso Neto: $1000.00
Capacidad de Pago: $300.00
Pago Mensual Requerido: $118.00
Credito Aprobado? Verdadero (SI)
```

```
Valor minimo de las variables utilizadas: 8.00
Contador de operaciones realizadas: 9
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2





Conclusiones del algoritmo

- El algoritmo funciona correctamente: evalúa el crédito con dos condiciones simultáneas (ingreso mínimo ≥ 800 y pago mensual ≤ 30% del ingreso neto). Basta que una falle para rechazar la solicitud.
- Las constantes simbólicas facilitan el mantenimiento: si cambia la tasa de interés o el porcentaje máximo, solo se modifica una línea.
- El algoritmo es didáctico y cumple el 100% del enunciado, pero para producción requeriría validaciones adicionales de entrada y manejo de errores.

Enunciado del algoritmo

EJ-059 (Ejercicio 4)

En una empresa de mensajería, construya un algoritmo que simule un cajero automático. Lee el saldo de la cuenta (real) y el monto a retirar (real). Use constantes simbólicas SALDO_MINIMO = 10.0, LIMITE_RETIRO = 500.0, COMISION = 0.50. Evalué la expresión lógica (monto > 0 AND monto <= LIMITE_RETIRO AND (saldo – monto- COMISION) >= SALDO_MINIMO). SI la operación es válida, calcule el nuevo saldo con (saldo – monto - COMISION). Asigne los resultados a variables. Use una variable booleana transacción_exitosa. Muestre por pantalla el saldo anterior, el monto, la comisión y el nuevo saldo. Además, calcule el valor mínimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados

Análisis del problema
Datos de entrada:

- **Constantes**:
  - SALDO_MINIMO = 10.0
  - Constante LIMITE_RETIRO = 500.0
  - Constante COMISION = 0.50
- **Variables**: saldo, monto

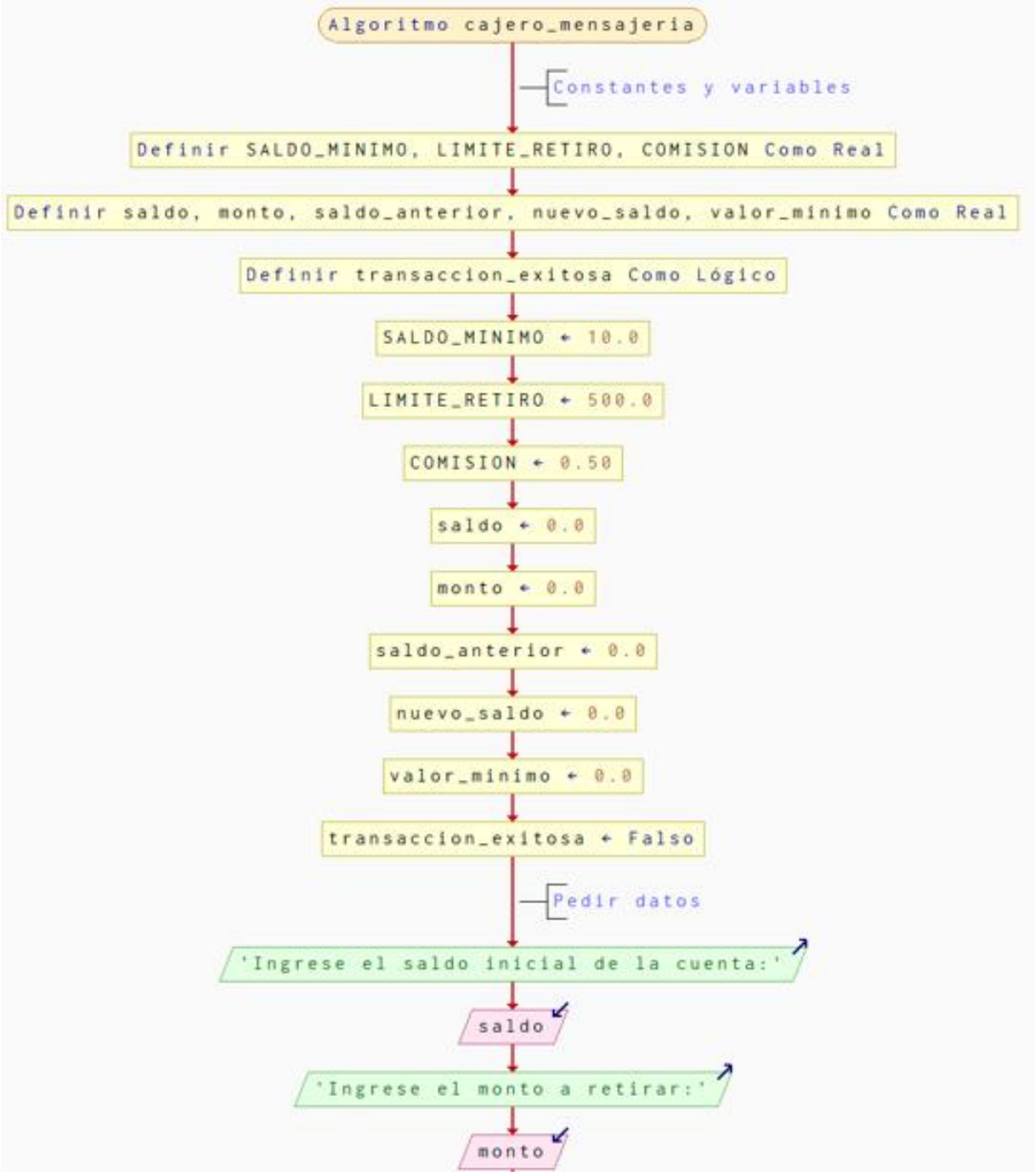
Proceso

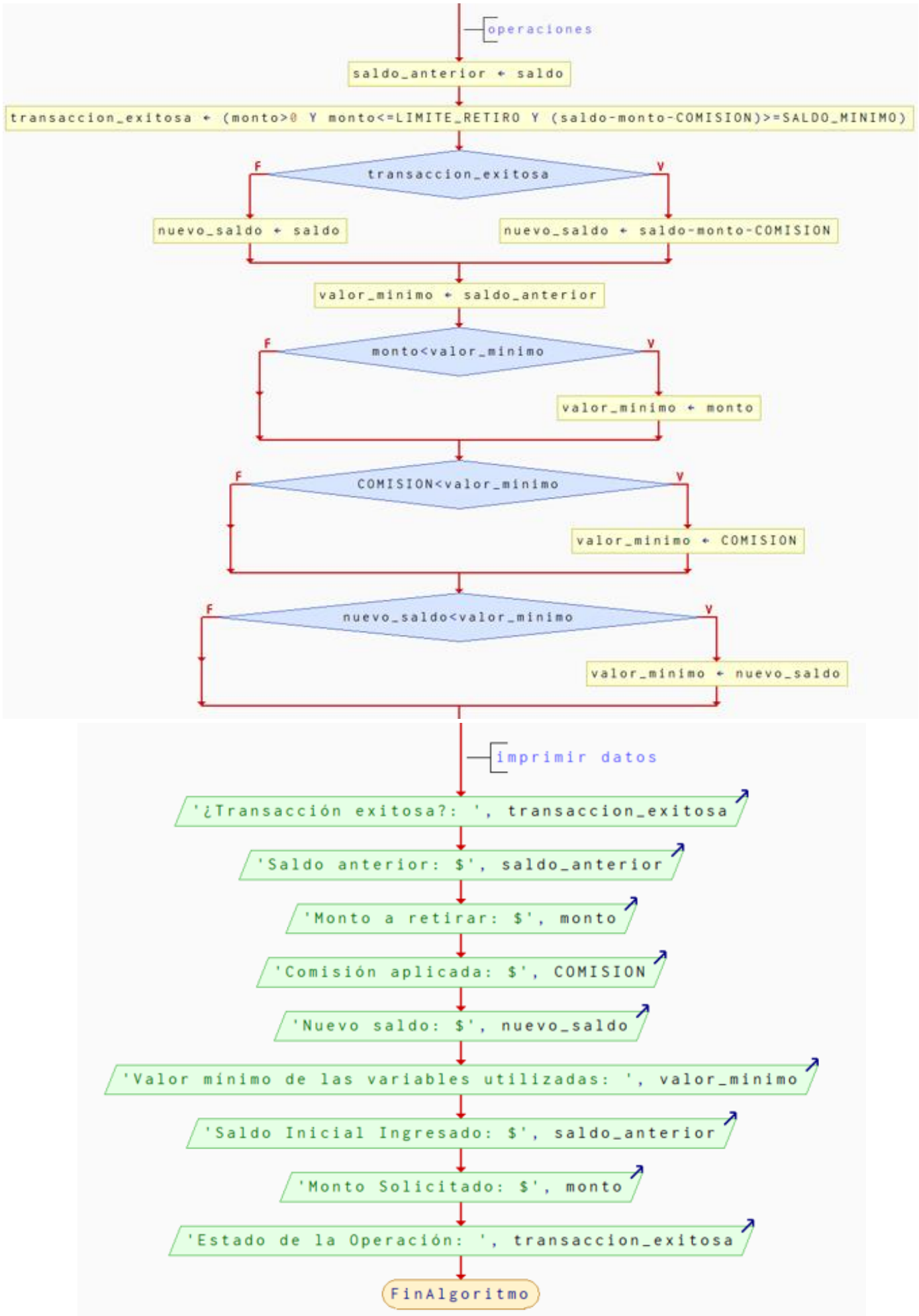
- transaccion_exitosa = (monto > 0 AND monto <= LIMITE_RETIRO AND (saldo - monto - COMISION) >= SALDO_MINIMO)
- nuevo_saldo = saldo - monto - COMISION

Datos de salida

- Transacción_exitosa
- Nuevo_saldo
- Comisión
- minimo

Diagrama de flujo




operaciones
saldo_anterior ← saldo
transaccion_exitosa ← (monto>0 Y monto<=LIMITE_RETIRO Y (saldo-monto-COMISION)>=SALDO_MINIMO)
transaccion_exitosa
F
V
nuevo_saldo ← saldo
nuevo_saldo ← saldo-monto-COMISION
valor_minimo ← saldo_anterior
monto<valor_minimo
valor_minimo ← monto
COMISION<valor_minimo
valor_minimo ← COMISION
nuevo_saldo<valor_minimo
valor_minimo ← nuevo_saldo
imprimir datos
'¿Transacción exitosa?: ', transaccion_exitosa
'Saldo anterior: $', saldo_anterior
'Monto a retirar: $', monto
'Comisión aplicada: $', COMISION
'Nuevo saldo: $', nuevo_saldo
'Valor mínimo de las variables utilizadas: ', valor_minimo
'Saldo Inicial Ingresado: $', saldo_anterior
'Monto Solicitado: $', monto
'Estado de la Operación: ', transaccion_exitosa
FinAlgoritmo

Pseudocódigo

```
Inicio
// Constantes y variables
SALDO_MINIMO = 10.0
LIMITE_RETIRO = 500.0
COMISION = 0.50
Definir saldo, monto, saldo_anterior, nuevo_saldo, valor_minimo Como Real
Definir transaccion_exitosa Como Logico
saldo = 0.0
monto = 0.0
saldo_anterior = 0.0
nuevo_saldo = 0.0
valor_minimo = 0.0
transaccion_exitosa = Falso
//Pedir datos
Escribir "Ingrese el saldo inicial de la cuenta:"
Leer saldo
Escribir "Ingrese el monto a retirar:"
Leer monto
//operaciones
saldo_anterior = saldo

transaccion_exitosa = (monto > 0 Y monto <= LIMITE_RETIRO Y (saldo - monto -
COMISION) >= SALDO_MINIMO)

Si (transaccion_exitosa) Entonces {
  nuevo_saldo = saldo - monto - COMISION
} Sino {
  nuevo_saldo = saldo
} FinSi
valor_minimo = saldo_anterior
Si (monto < valor_minimo) Entonces {
  valor_minimo = monto
} FinSi
Si (COMISION < valor_minimo) Entonces {
  valor_minimo = COMISION
} FinSi
Si (nuevo_saldo < valor_minimo) Entonces {
  valor_minimo = nuevo_saldo
} FinSi
//imprimir datos
Escribir "¿Transaccion exitosa?: ", transaccion_exitosa
```

Escribir "Saldo anterior: $", saldo_anterior
Escribir "Monto a retirar: $", monto
Escribir "Comision aplicada: $", COMISION
Escribir "Nuevo saldo: $", nuevo_saldo
Escribir "Valor minimo de las variables utilizadas: ", valor_minimo
Escribir "Saldo Inicial Ingresado: $", saldo_anterior
Escribir "Monto Solicitado: $", monto
Escribir "Estado de la Operacion: ", transaccion_exitosa
Fin

Prueba de escritorio

Prueba 1

Paso	**saldo**	**monto**	**Monto > 0**	**Monto <= 500**	**saldo - monto - 0.50 >= 10**	**transaccion_exitosa**
1	200	100	V	V	99.5 >= 10, V	Verdadero

Paso	**Nuevo saldo**	**minimo**
1	99.5	99.5

Paso	Salida de pantalla
2	**Transaccion EXITOSA Saldo anterior:** 200 **Monto retirado**: 100 **Comision:** 0.5 **Nuevo saldo:** 99.5 **Valor minimo de las variables**: 99.5 **Saldo ingresado:** 200 **Monto a retirar ingresado:** 100 **Estado de la operacion**: exitosa

Prueba 2

Paso	**saldo**	**monto**	**Monto > 0**	**Monto <= 500**	**saldo - monto - 0.50 >= 10**	**transaccion_exitosa**
1	100	150	V	V	-50.5 >= 10, **F**	falso

Paso	**Nuevo saldo**	**minimo**
1	100	100

Paso	**Salida de pantalla**
2	**Transacción:** RECHAZADA **Saldo actual (sin cambios):** 100

	Valor mínimo de las variables: 100 **Saldo ingresado:** 100 **Monto a retirar ingresado:** 150 **Estado de la operación:** rechazada

Codificación en lenguaje C

EJ4_059.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    const float SALDO_MINIMO = 10.0;
    const float LIMITE_RETIRO = 500.0;
    const float COMISION = 0.50;
    double saldo = 0, monto = 0, nuevo_saldo = 0, minimo = 0;
    bool transaccion_exitosa = false;

    printf("Ingrese el saldo de la cuenta: ");
    scanf("%lf", &saldo);
    printf("Ingrese el monto a retirar: ");
    scanf("%lf", &monto);

    nuevo_saldo = saldo;
    transaccion_exitosa = (monto > 0 && monto <= LIMITE_RETIRO && (saldo - monto - COMISION) >= SALDO_MINIMO);

    if (transaccion_exitosa) {
        nuevo_saldo = saldo - monto - COMISION;
    }

    minimo = saldo;
    if (monto < minimo)       minimo = monto;
    if (nuevo_saldo < minimo) minimo = nuevo_saldo;
```

```c

    if (transaccion_exitosa) {
        printf("Transaccion EXITOSA\n");
        printf("Saldo anterior: %.2f\n", saldo);
        printf("Monto retirado: %.2f\n", monto);
        printf("Comision: %.2f\n", COMISION);
        printf("Nuevo saldo: %.2f\n", nuevo_saldo);
    } else {
        printf("Transaccion RECHAZADA\n");
        printf("Saldo actual (sin cambios): %.2f\n", nuevo_saldo);
    }
    printf("Valor minimo de las variables: %.2f\n", minimo);

    printf("Saldo ingresado: %.2f\n", saldo);
    printf("Monto a retirar ingresado: %.2f\n", monto);
    printf("Estado de la operacion: %s\n", transaccion_exitosa ? "exitosa" : "rechazada");

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ4_059.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer059.exe
Ingrese el saldo de la cuenta: 200
Ingrese el monto a retirar: 100
Transaccion EXITOSA
Saldo anterior: 200.00
Monto retirado: 100.00
Comision: 0.50
Nuevo saldo: 99.50
```

```
Valor minimo de las variables: 99.50
Saldo ingresado: 200.00
Monto a retirar ingresado: 100.00
Estado de la operacion: exitosa
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>  .\ejer059.exe
Ingrese el saldo de la cuenta: 100
Ingrese el monto a retirar: 150
Transaccion RECHAZADA
Saldo actual (sin cambios): 100.00
Valor minimo de las variables: 100.00
Saldo ingresado: 100.00
Monto a retirar ingresado: 150.00
Estado de la operacion: rechazada
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El algoritmo simula correctamente un cajero automático aplicando tres validaciones simultáneas (monto positivo, límite de retiro y saldo mínimo tras comisión). Si la operación es válida, descuenta el monto y la comisión; si no, el saldo permanece intacto y no se cobra comisión.

Enunciado del algoritmo

EJ-060 (Ejercicio 5)

En un hospital, diseñe un algoritmo que valide una dirección de correo electrónico (cadena). Use constantes simbólicas MIN_LEN = 5, MAX_LEN = 50. Evalué la expresión lógica compuesta (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN AND contiene (correo '@') AND contiene (correo, '.') AND NOT empieza_con (correo, '@') AND NOT termina_con (correo, '@')). Asigne el resultado a una variable booleana correo_valido. Muestre por pantalla si el correo es válido y que criterios fallaron. Además, calcule el valor mínimo de todas las variables utilizadas. Asegúrese de que las expresiones lógicas estén bien parentizadas

Análisis del problema

Datos de entrada: correo

Constantes:

- MIN_LEN = 5
- MAX_LEN = 50

Variables:

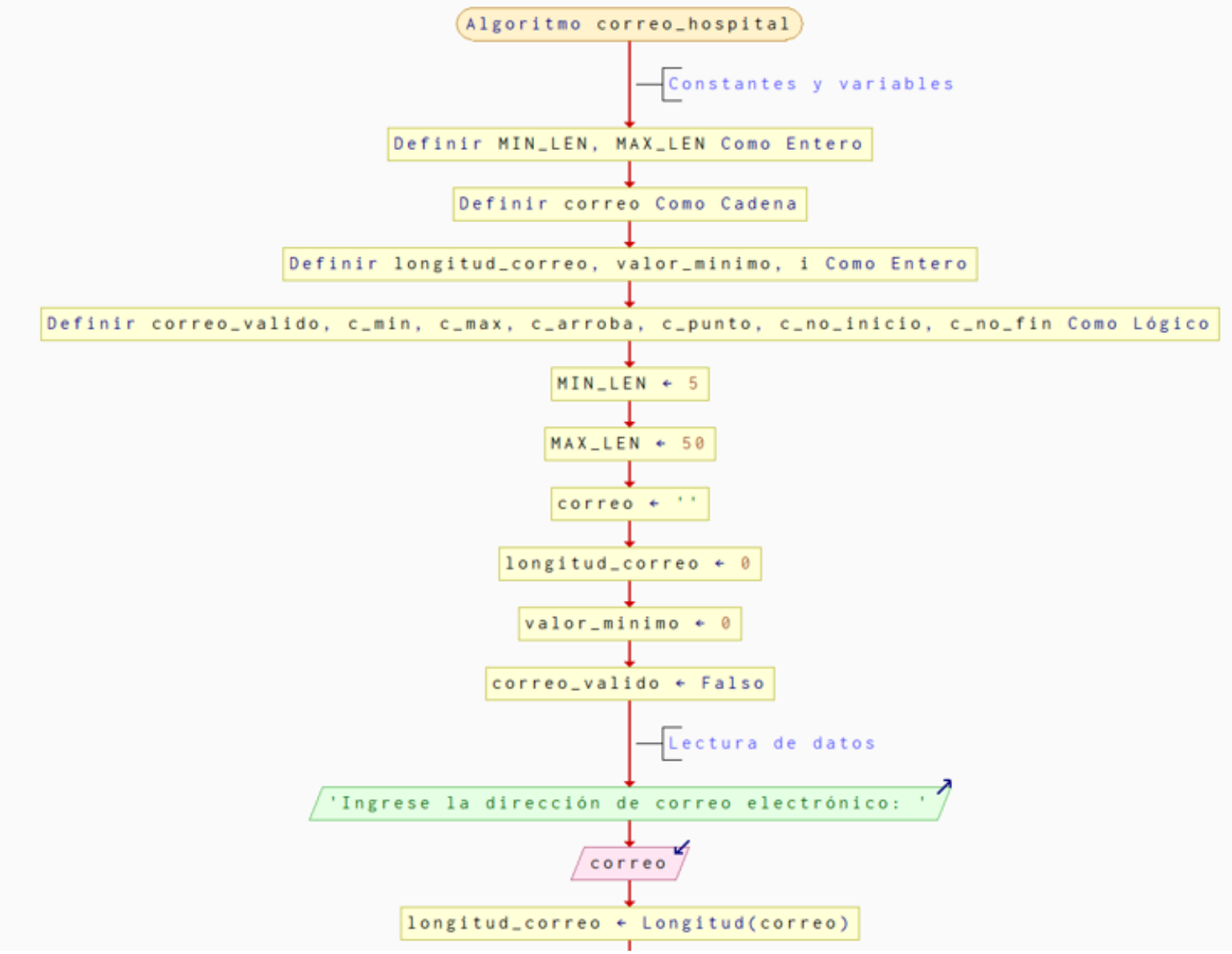
Proceso

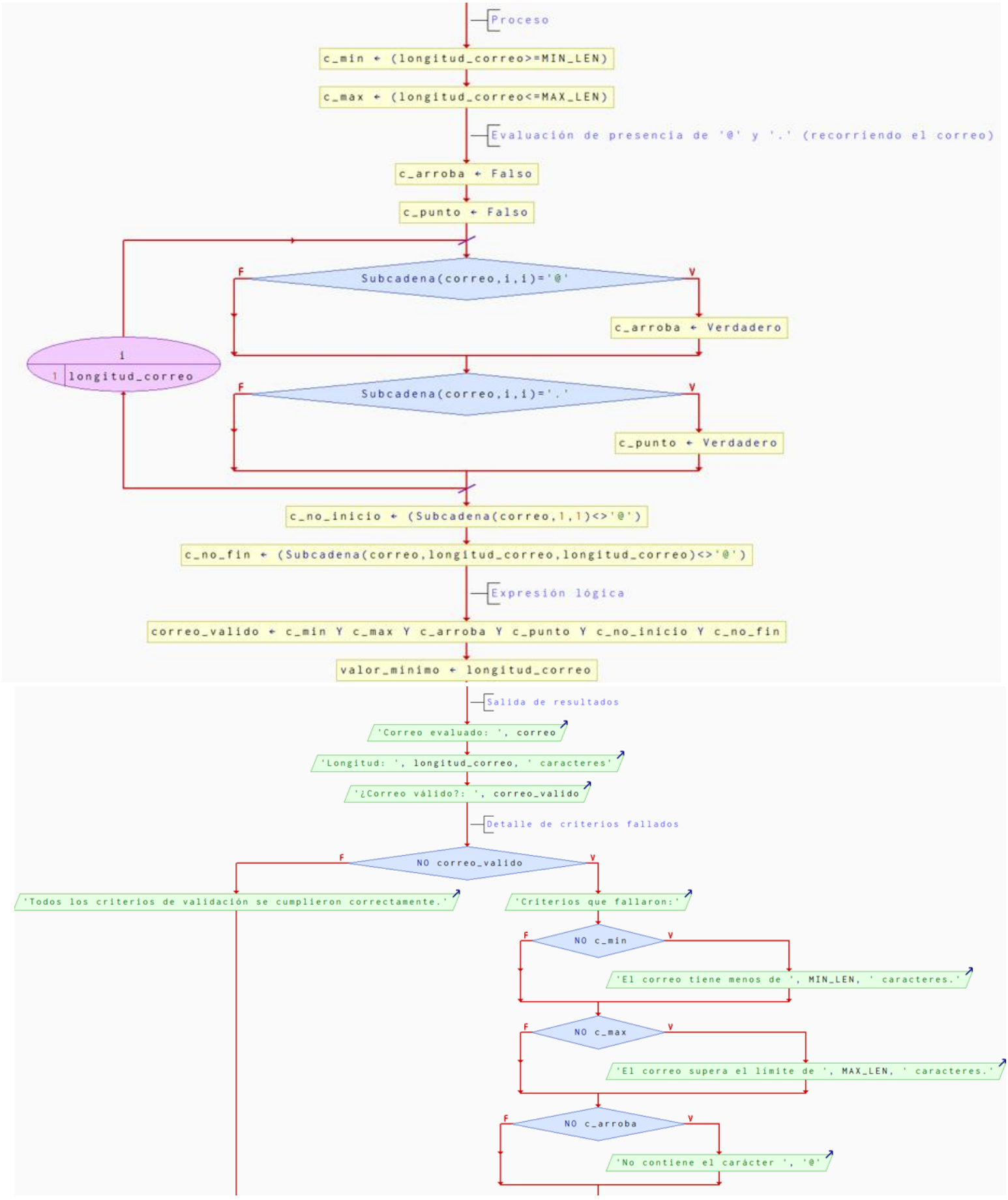
- longitud_correo = longitud(correo)
- correo_valido: (largo >= MIN_LEN) AND (largo <= MAX_LEN) AND (cant_arroba > 0) AND (cant_punto > 0) AND (NOT inicia_arroba) AND (NOT termina_arroba)

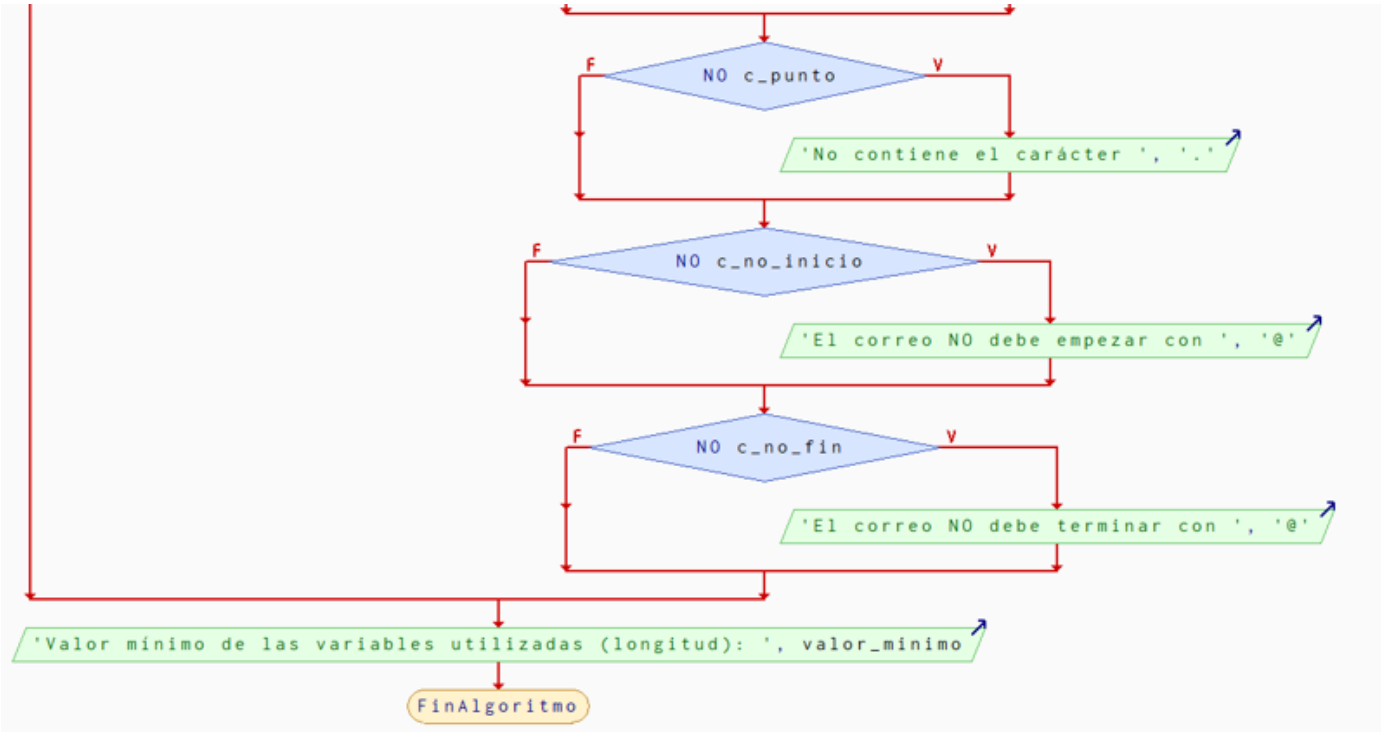
Datos de salida

- Correo_valido
- Criterios_fallidos
- minimo

Diagrama de flujo




Proceso
c_min ← (longitud_correo>=MIN_LEN)
c_max ← (longitud_correo<=MAX_LEN)
Evaluación de presencia de '@' y '.' (recorriendo el correo)
c_arroba ← Falso
c_punto ← Falso
F
Subcadena(correo,i,i)='@'
V
c_arroba ← Verdadero
i
1
longitud_correo
F
Subcadena(correo,i,i)='.'
V
c_punto ← Verdadero
c_no_inicio ← (Subcadena(correo,1,1)<>'@')
c_no_fin ← (Subcadena(correo,longitud_correo,longitud_correo)<>'@')
Expresión lógica
correo_valido ← c_min Y c_max Y c_arroba Y c_punto Y c_no_inicio Y c_no_fin
valor_minimo ← longitud_correo
Salida de resultados
'Correo evaluado: ', correo
'Longitud: ', longitud_correo, ' caracteres'
'¿Correo válido?: ', correo_valido
Detalle de criterios fallados
F
NO correo_valido
V
'Todos los criterios de validación se cumplieron correctamente.'
'Criterios que fallaron:'
F
NO c_min
V
'El correo tiene menos de ', MIN_LEN, ' caracteres.'
F
NO c_max
V
'El correo supera el límite de ', MAX_LEN, ' caracteres.'
F
NO c_arroba
V
'No contiene el carácter ', '@'



Pseudocódigo

```
Inicio
//Constantes y variables
MIN_LEN <- 5
MAX_LEN <- 50
Definir correo Como Cadena
Definir longitud_correo, valor_minimo Como Entero
Definir correo_valido, c_min, c_max, c_arroba, c_punto, c_no_inicio, c_no_fin Como Logico

correo = ""
longitud_correo = 0
valor_minimo = 0
correo_valido = Falso
//Lectura de datos
Escribir ("Ingrese la direccion de correo electronico: ")
Leer correo
longitud_correo = Longitud(correo)
//Proceso
c_min = (longitud_correo >= MIN_LEN)
c_max = (longitud_correo <= MAX_LEN)
c_arroba = (Subcadena (correo, 1, Longitud(correo)) contiene "@") // Evaluación funcional de presencia
c_punto = (Subcadena (correo, 1, Longitud(correo)) contiene ".")
c_no_inicio = (Subcadena (correo, 1, 1) != "@")
c_no_fin = (Subcadena (correo, longitud_correo, longitud_correo) != "@")
//Expresión lógica
correo_valido = ((c_min) Y (c_max) Y (c_arroba) Y (c_punto) Y (c_no_inicio) Y (c_no_fin))
```

```
valor_minimo = longitud_correo
// Salida de resultados
Escribir ("Correo evaluado: " + correo)
Escribir ("Longitud: " + longitud_correo + " caracteres")
Escribir ("¿Correo valido?: " + correo_valido)
// Detalle de criterios fallados
Si NO correo_valido Entonces
        Escribir ("Criterios que fallaron:")
Si NO c_min Entonces
        Escribir ("El correo tiene menos de " + MIN_LEN + " caracteres.")
FinSi
Si NO c_max Entonces
        Escribir ("El correo supera el límite de " + MAX_LEN + " caracteres.")
FinSi
Si NO c_arroba Entonces
        Escribir ("No contiene el caracter '@'.")
FinSi
Si NO c_punto Entonces
        Escribir ("No contiene el caracter '.'.")
FinSi
Si NO c_no_inicio Entonces
        Escribir ("El correo NO debe empezar con '@'.")
FinSi
Si NO c_no_fin Entonces
        Escribir ("El correo NO debe terminar con '@'.")
FinSi
Sino
  Escribir ("Todos los criterios de validación se cumplieron correctamente.")
FinSi
Escribir ("Valor mínimo de las variables utilizadas (longitud): " + valor_minimo)
Fin
```

Prueba de escritorio

Prueba 1

Paso	**Correo**	**Longitud**	**Posición @**	**Posición “.”**	**c1 (≥5)**	**c2 (≤50)**
1	daniela @gmail. com	17	7	13	V	V
1	**c3 (contie**	**c4 (contiene .)**	**c5 (NO empieza**	**c6 (NO termina**	**correo_valido**	**Criterios fallidos**

	ne @)		**@)**	**@)**		
	V	V	V	V	true	Ninguno
	Mínimo	**Resultado**				
1	1	Correo válido				

Prueba 2

Paso	**Correo**	**Longitud**	**Posición @**	**Posición “.”**	**c1 (≥5)**	**c2 (≤50)**
1	Hospital.com	12	No existe (0)	8	V	V
1	**c3 (contiene @)**	**c4 (contiene .)**	**c5 (NO empieza @)**	**c6 (NO termina @)**	**correo_valido**	**Criterios fallidos**
	V	V	V	V	false	No contiene “@”
	Mínimo	**Resultado**				
1	0	Cadena inválida				

Codificación en lenguaje C

C EJ5_060.c > main()

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define min_len 5
#define max_len 50

int main() {
    //Inicialización de todas las variables
    char correo[100] = "";
    char criterios_fallidos[300] = "";
    int  longitud = 0, pos_arroba = 0, pos_punto = 0, minimo = 0;
    bool correo_valido = false;
    bool c1 = false, c2 = false, c3 = false, c4 = false, c5 = false, c6 = false;

    //Datos
    printf("Ingrese el correo electronico: ");
    scanf("%99s", correo);

    // Proceso
    longitud = (int)strlen(correo);

    // Proceso - Evaluación de cada criterio por separado
    c1 = (longitud >= min_len);
    c2 = (longitud <= max_len);
    c3 = (strchr(correo, '@') != NULL);
    c4 = (strchr(correo, '.') != NULL);
    c5 = (correo[0] != '@');
    c6 = (longitud > 0 && correo[longitud - 1] != '@');
```

```c
    // Proceso - Expresión lógica compuesta (bien parentizada)
    correo_valido = (c1 && c2 && c3 && c4 && c5 && c6);

    // Proceso - Criterios fallidos
    if (!c1) strcat(criterios_fallidos, "Longitud < MIN_LEN. ");
    if (!c2) strcat(criterios_fallidos, "Longitud > MAX_LEN. ");
    if (!c3) strcat(criterios_fallidos, "No contiene '@'. ");
    if (!c4) strcat(criterios_fallidos, "No contiene '.'. ");
    if (!c5) strcat(criterios_fallidos, "Empieza con '@'. ");
    if (!c6) strcat(criterios_fallidos, "Termina con '@'. ");

    if (criterios_fallidos[0] == '\0')
        strcpy(criterios_fallidos, "Ninguno (correo valido)");

    // Posiciones para el mínimo (si no existen, quedan en 0)
    char *p_arroba = strchr(correo, '@');
    char *p_punto  = strchr(correo, '.');
    pos_arroba = p_arroba ? (int)(p_arroba - correo) : 0;
    pos_punto  = p_punto  ? (int)(p_punto  - correo) : 0;

    // Proceso
    float fv = correo_valido ? 1.0f : 0.0f;
    minimo = longitud;
    minimo = min_len    < minimo ? min_len    : minimo;
    minimo = max_len    < minimo ? max_len    : minimo;
    minimo = pos_arroba < minimo ? pos_arroba : minimo;
    minimo = pos_punto  < minimo ? pos_punto  : minimo;
    minimo = (fv < minimo) ? (int)fv : minimo;
```

```c
// Salida

printf("Correo ingresado:    %s\n", correo);
printf("Longitud:            %d\n", longitud);
printf("Correo valido?       %s\n", correo_valido ? "true" : "false");
printf("Criterios fallidos:  %s\n", criterios_fallidos);
printf("Valor minimo:        %d\n", minimo);

return 0;
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ5_060.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer060.exe
Ingrese el correo electronico: daniela@gmail.com

--- VALIDACION DE CORREO ---
Correo ingresado:    daniela@gmail.com
Longitud:            17
Correo valido?       true
Criterios fallidos:  Ninguno (correo valido)
Valor minimo:        1
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer060
Ingrese el correo electronico: Hospital.com

--- VALIDACION DE CORREO ---
Correo ingresado:    Hospital.com
Longitud:            12
Correo valido?       false
Criterios fallidos:  No contiene '@'.
Valor minimo:        0
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- Comprendí la importancia de parentizar correctamente. Cada condición va entre paréntesis para evitar ambigüedades de precedencia. Esto es especialmente importante cuando se combinan operadores como &&, || y !, donde un paréntesis mal puesto puede cambiar completamente el resultado.

Enunciado del algoritmo

EJ-001 (Ejercicio 6)

En un taller mecánico, declare tres variables enteras llamadas x, u y val2. Solicite por teclado tres números enteros (lectura) y almacénelos en esas variables. Calcule la suma, el producto y el promedio (expresión aritmética) usando una constante literal 3 como divisor. Asigne los resultados a variables suma, producto y promedio. Muestre por pantalla (escritura) un mensaje con los tres resultados. Justifique porque promedio debe ser tipo real mientras que suma y producto son enteros. Además del cálculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operación realizada.

Análisis del problema

Datos de entrada:

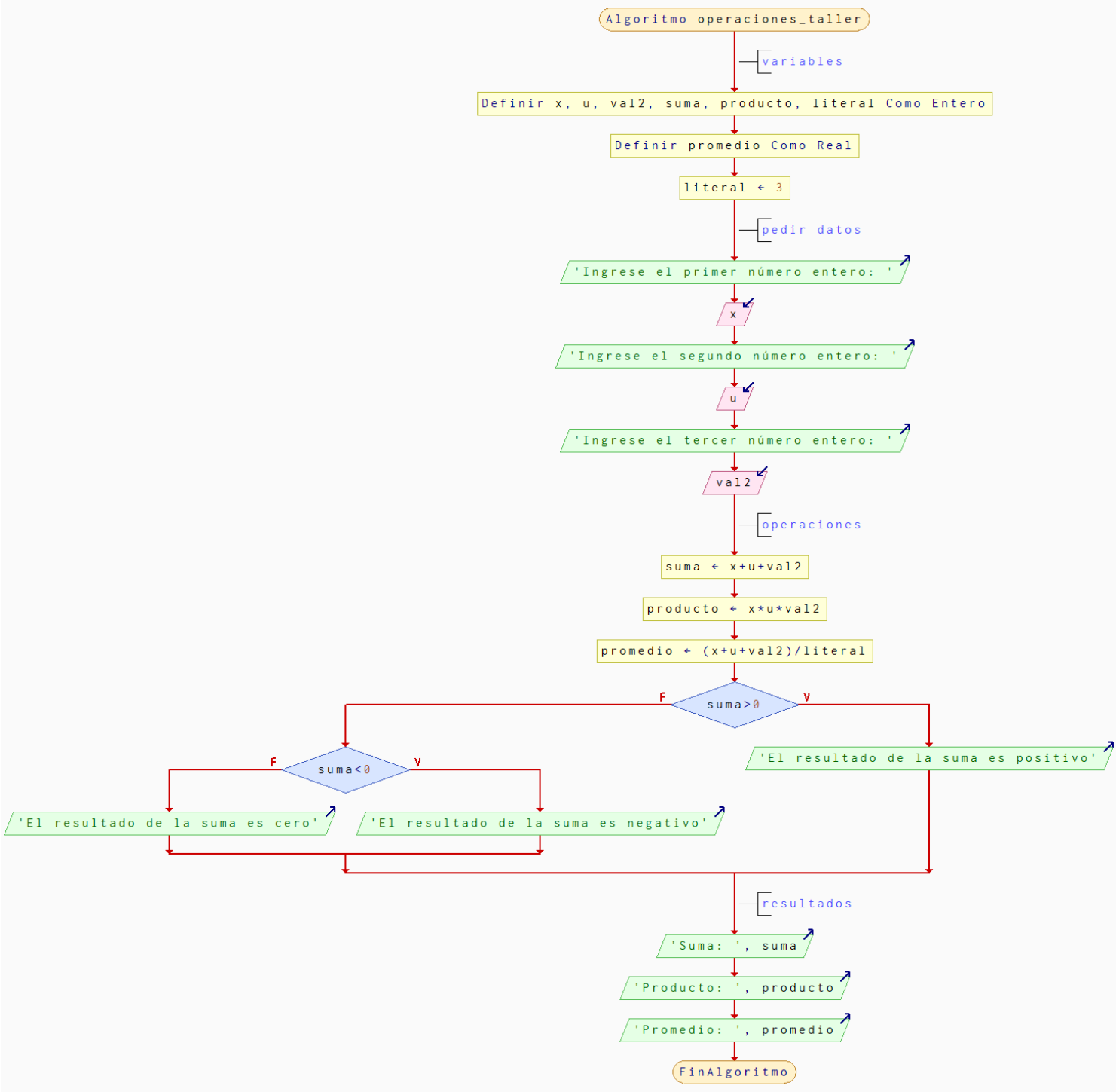
- **Variables:** x, u, val2
- **Constante:** literal =3

Proceso

- **suma** = x + u + val2
- **producto** = x * u * val2
- **promedio** = suma / literal

Datos de salida: suma, producto, promedio.

Diagrama de flujo



Pseudocódigo

Inicio

//variables

Definir x, u, val2, suma, producto, literal como entero

Definir promedio como real

//Pedir datos

Escribir ("Ingrese el primer número entero: ")

Leer x

Escribir ("Ingrese el segundo número entero: ")

Leer u

Escribir ("Ingrese el tercer número entero: ")

```
Leer val2
//operaciones
Suma = x + u + val2
Producto = x * u * val2
Promedio = (x + u + val2)/literal
Si (suma > 0) Entonces
        Imprimir (“El resultado de la suma es positivo”)
Sino
        Si (suma < 0) Entonces
                Imprimir (“El resultado de la suma es negativo”)
        Si no
                Imprimir (“El resultado de la suma es cero”)
        Fin si
Fin si
Escribir ("Suma: " + suma)
Escribir ("Producto: " + producto)
Escribir ("Promedio: " + promedio)
Fin
```

Prueba de escritorio

Prueba 1

Paso	x	u	Val2	suma	producto	promedio
1	---	---	---	---	---	---
2	10	---	---	---	---	---
3	10	10	---	---	---	---
4	10	10	10	30	100	10

Paso	Suma > 0	Suma <0	Salida de pantalla
4	Verdadero	falso	El resultado de la suma es positivo

Prueba 2

Paso	x	u	Val2	suma	producto	promedio
1	---	---	---	---	---	---
2	9	---	---	---	---	---
3	9	7	---	---	---	---
4	9	7	8	24	504	8

Paso	Suma > 0	Suma <0	Salida de pantalla
4	Verdadero	falso	El resultado de la suma es positivo

Codificación en lenguaje C

```c
#include <stdio.h>
int main(){
    int x, u, val2, suma, producto;
    const int literal = 3;
    float promedio;
    //pedir datos
    printf("Ingrese el primer numero entero: ");
    scanf("%d", &x);
    printf("Ingrese el segundo numero entero: ");
    scanf("%d",&u);
    printf("Ingrese el tercer numero entero: ");
    scanf("%d", &val2);
    //operaciones
    suma = x + u + val2;
    producto = x * u * val2;
    promedio = (x + u + val2)/literal;
    if (suma >0){
        printf("El resulatado de la suma es positivo.\n");
    }else{
        if (suma < 0){
            printf("El resultado de la suma es negativo.\n");
        }else{
            printf("El resultado de la suma es 0.\n");
        }
    }
    printf("Suma: %d.\n", suma);
    printf("Producto: %d.\n", producto);
    printf("Promedio: %.2f", promedio);
    return 0;
}
```

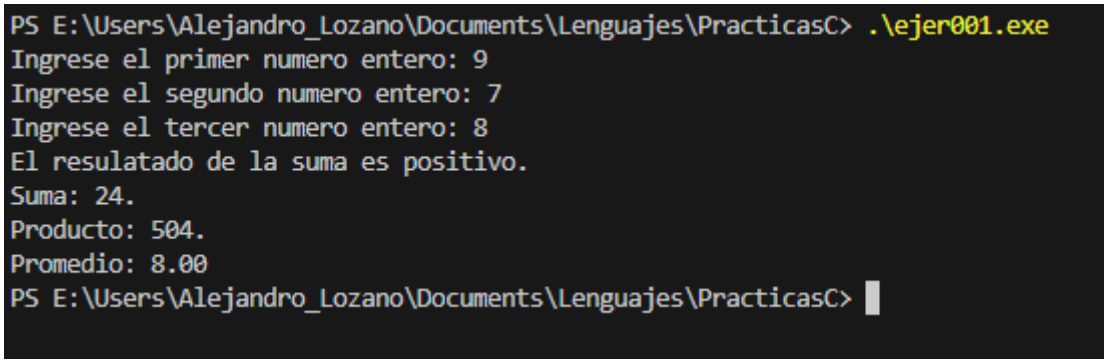
Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ6_001.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> gcc EJ6_001.c -o ejer001.exe
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer001.exe
Ingrese el primer numero entero: 10
Ingrese el segundo numero entero: 10
Ingrese el tercer numero entero: 10
El resulatado de la suma es positivo.
Suma: 30.
Producto: 1000.
Promedio: 10.00
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2



Conclusiones del algoritmo

Enunciado del algoritmo

EJ-008 (Ejercicio 7)

En una farmacia, cree un algoritmo que lea la base (real) y la altura (real) de un rectángulo. Use la constante literal 2 en el perímetro. Calcule área = base * altura y perímetro = 2 * (base + altura) usando expresiones aritméticas. Evalúe la expresión lógica (base > 0 AND altura > 0). Asigne los resultados a variables reales. Muestre por pantalla el área y el perímetro. Además del cálculo principal, determine el doble del resultado. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

Datos de entrada:

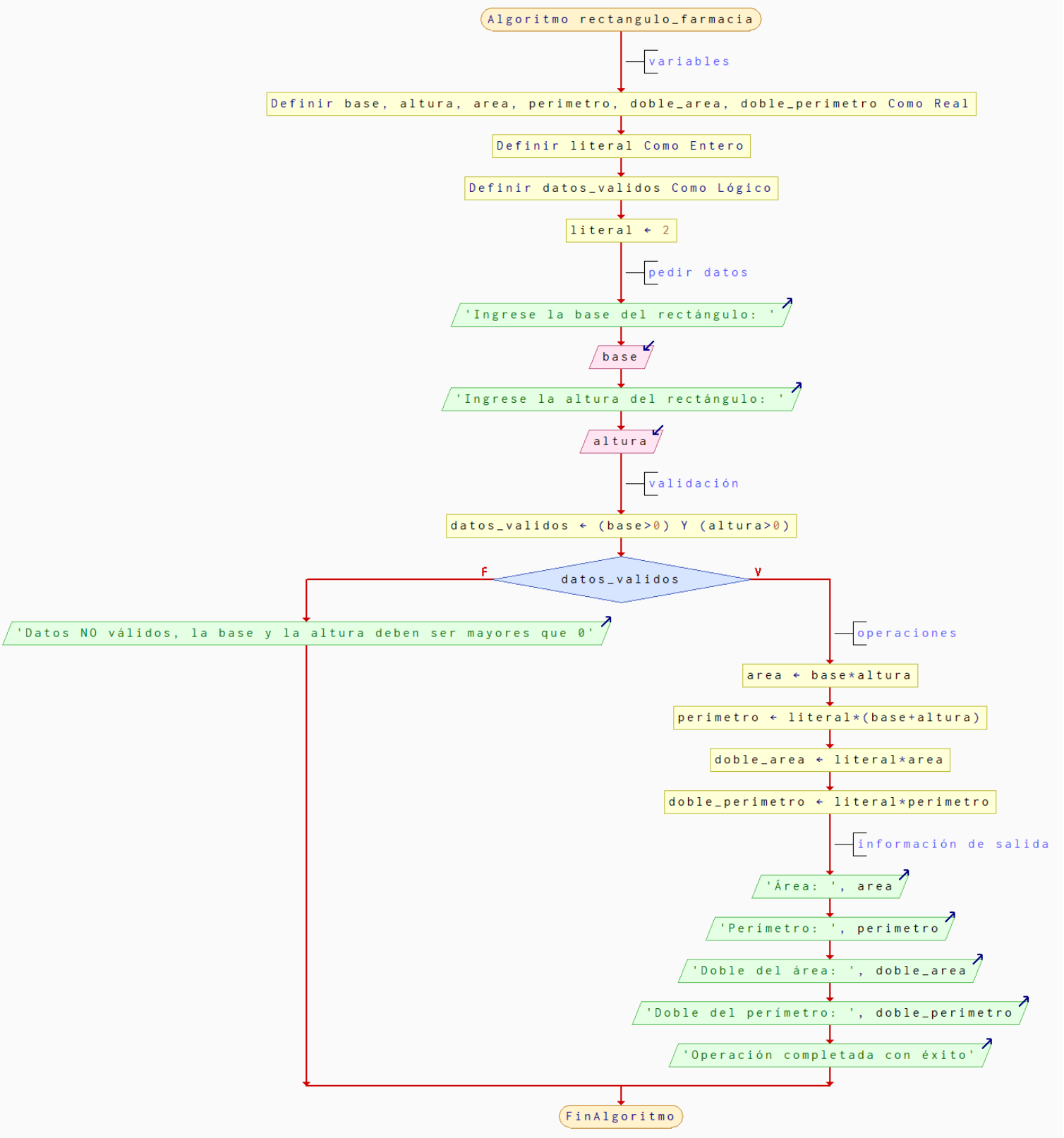
- **Variables:** base, altura
- **Constante:** literal =2

Proceso

- **Área =** base * altura
- **Perimetro =** 2 * (base + altura)
- (base > 0 AND altura > 0
- doble_area = 2 * área
- doble_perimetro = 2* perimetro

- **Datos de salida:** area, producto, perimetro, doble_area, doble_perimetro, datos_validos

Diagrama de flujo



Pseudocódigo

```
Inicio
Definir base, altura, perimetro como real
Definir datos_validos Como Logico
Literal = 2
Imprimir (“Ingrese la base del rectangulo: ”)
Leer base
Imprimir (“Ingrese la altura del rectangulo: ”)
Leer altura
datos_validos = (base > 0) Y (altura > 0)
Si (datos_validos) Entonces
        //operaciones
        area = base * altura
        perimetro = 2 * (base + altura)
        doble_area = 2 * area
        doble_perimetro = 2 * perimetro
        // Informacion de salida
        Escribir ("Area: " + area )
        Escribir ("Perimetro: " + perimetro)
        Escribir ("Doble del area: " + doble_area)
        (Escribir "Doble del perimetro: " + doble_perimetro)
        (Escribir "Operación completada con exito")
SiNo
        Escribir "Datos NO validos, la base y la altura deben ser mayores que 0"
FinSi
```

Prueba de escritorio

Prueba 1

Paso	Instrucción	base	altura	datos_validos
1	Leer base y altura	9	5	?
2	datos_validos = (9 > 0) AND (5 > 0)	9	5	Verdadero
3	area = 9 * 5 y perimetro = 2 * (9 + 5)	9	5	Verdadero
4	doble_area = 45 * 2 y doble_perimetro = 28 * 2; mostrar resultados	9	5	Verdadero

Pasos	area	perimetro	doble_area	doble_perimetro	Salida
1	---	---	---	---	---
2	---	---	---	---	---
3	45.0	28.0	---	---	---

4	45.0	28.0	90.0	56.0	**Área**: 45.00 **Perímetro:** 28.00 **Doble del área:** 90.00 **Doble del perímetro:** 56.00 **Operación completada con éxito**

Prueba 2

Paso	Instrucción	base	altura	datos_validos
1	Leer base y altura	30	20	?
2	datos_validos = (9 > 0) AND (5 > 0)	30	20	Verdadero
3	area = 9 * 5 y perimetro = 2 * (9 + 5)	30	20	Verdadero
4	doble_area = 45 * 2 y doble_perimetro = 28 * 2; mostrar resultados	30	20	Verdadero

Pasos	area	perimetro	doble_area	doble_perimetro	Salida
1	---	---	---	---	---
2	---	---	---	---	---
3	600.0	100.0	---	---	---
4	600.0	100.0	1200.0	200.0	**Área**: 600.00 **Perímetro:** 100.00 **Doble del área:** 1200.00 **Doble del perímetro:** 200.00 **Operación completada con éxito**

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
int main(){
    //declarar variables
    double base, altura, area, perimetro, doble_area, doble_perimetro;
    bool datos_validos = false;
    const int literal = 2;
    //pedir datos
    printf("Ingrese la base del rectangulo: ");
    scanf("%Lf", &base);
    printf("Ingrese la altura del rectangulo: ");
    scanf("%Lf", &altura);
    //operaciones
    datos_validos = (base > 0) && (altura > 0);

    if (datos_validos) {
        area = base * altura;
        perimetro = literal * (base + altura);
        doble_area = literal * area;
        doble_perimetro = literal * perimetro;

        printf("Area: %.2f\n", area);
        printf("Perimetro: %.2f\n", perimetro);
        printf("Doble del area: %.2f\n", doble_area);
        printf("Doble del perimetro: %.2f\n", doble_perimetro);
        printf("Operacion completada con exito\n");
    } else {
        printf("\nDatos no validos: la base y la altura deben ser mayores que 0\n");
    }
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ7_008.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer008.exe
Ingrese la base del rectangulo: 9
Ingrese la altura del rectangulo: 5
Area: 45.00
Perimetro: 28.00
Doble del area: 90.00
Doble del perimetro: 56.00
Operacion completada con exito
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer008.exe
Ingrese la base del rectangulo: 30
Ingrese la altura del rectangulo: 20
Area: 600.00
Perimetro: 100.00
Doble del area: 1200.00
Doble del perimetro: 200.00
Operacion completada con exito
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- Los resultados de la prueba de escritorio coincidieron con lo de la salida del programa
- Guardas los resultados en variables reales permitió que se pueda trabajar con decimales
- El mensaje final le confirma al usuario que todo salió bien, con el proceso de cálculo y ejecución del algoritmo

Enunciado del algoritmo

EJ-010 (Ejercicio 8)

En una joyería, declare dos variables enteras n y u. Lea ambos valores por teclado. Use la constante literal 2 para evaluar paridad. Calcule residuo1 = n % 2 y residuo2 = u % 2. Evalúe las expresiones lógicas (residuo1 == 0) y (residuo2 == 0). Muestre por pantalla si cada número es par o impar. Identifique el tipo de dato simple de cada variable. Además del cálculo principal, calcule el resultado elevado al cubo. Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

Datos de entrada:

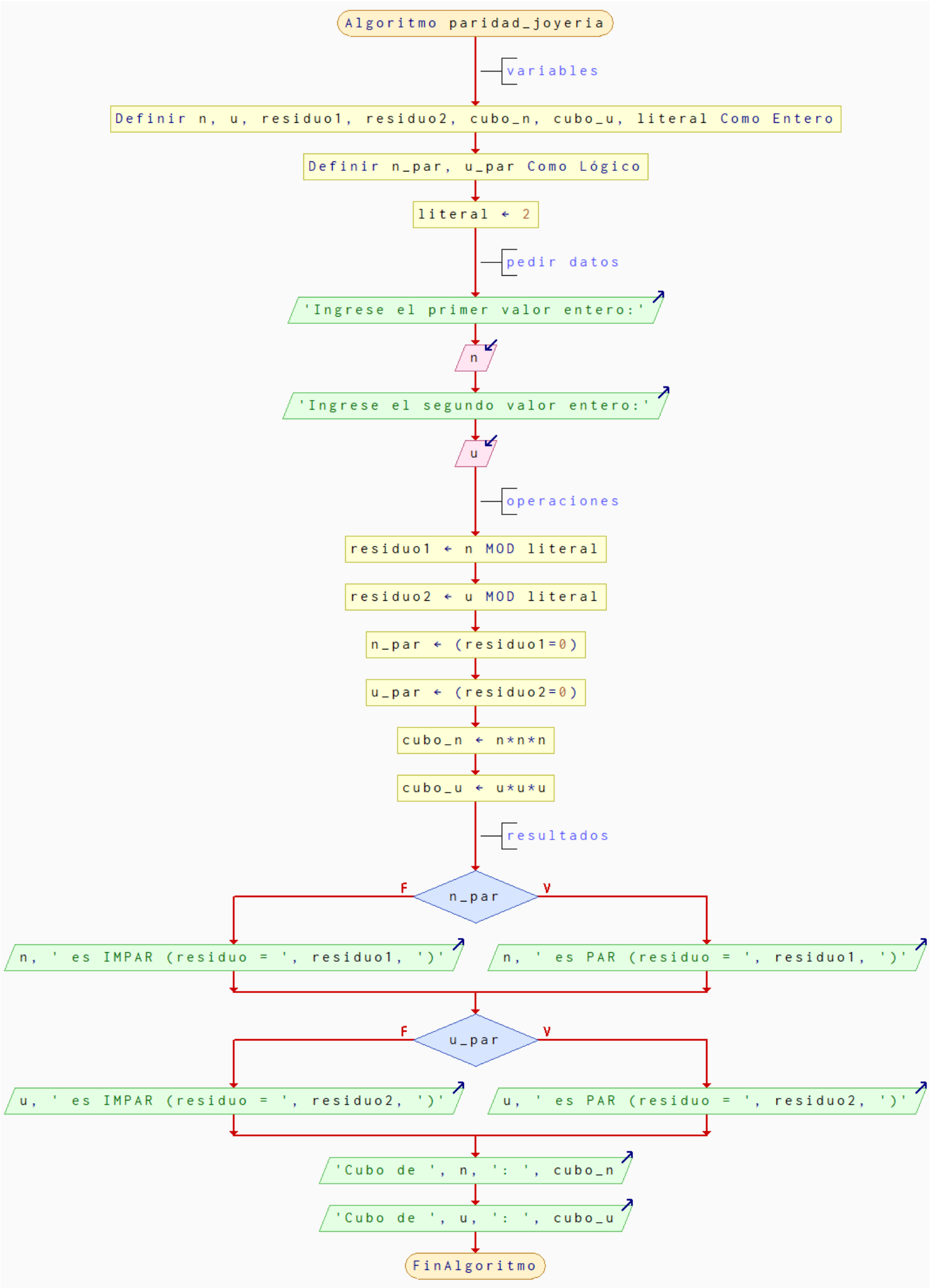
- **Variables:** n, u
- **Constante:** literal =2

Proceso

- **Residuo1 =** n % 2
- **Rediduo2 =** u % 2
- **n_par =** (residuo1 = 0)
- **u_par =** (residuo2 = 0)
- **cubo_n =** n * n * n
- **cubo_u =** u * u * u

Datos de salida: residuo1, residuo2, n_par, u_par, cubo_n, cubo_u.

Diagrama de flujo



Pseudocódigo

```
Inicio
Definir n, u, residuo1, residuo2, cubo_n, cubo_u Como Entero
Definir n_par, u_par Como Logico
Literal = 2
Imprimir (“Ingrese el primer valor entero:”)
Leer n
Imprimir (“Ingrese el segundo valor entero:”)
Leer u
//operaciones
residuo1 = n % 2
residuo2 = u % 2
n_par = (residuo1 == 0)
u_par = (residuo2 ==0)
cubo_n = n * n * n
cubo_u = u * u * u
Si (n_par) Entonces
      Escribir (n + " es PAR (residuo = " + residuo1 + ")")
Sino
      Escribir (n, + es IMPAR (residuo = " + residuo1 + ")")
FinSi
Si (u_par) Entonces
      Escribir (u + “es PAR (residuo = " + residuo2 + ")")
Sino
      Escribir (u + " es IMPAR (residuo = " + residuo2, ")")
FinSi
Escribir ("Cubo de " + n + ": " + cubo_n)
Escribir ("Cubo de " +u + ": " + cubo_u)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Instrucción	Calculo	Resultado
1	---	---	---
2	Leer n	---	4
3	Leer u	---	8
4	residuo1 = 4 % 2	4/2 = 2 residuo 0	0
5	residuo2 = 8 % 2	8/2 = 4 residuo 0	0
6	n_par = (0 == 0)	V	El numero 4 es par (residuo = 0)
7	u_par = (0 == 0)	V	El numero 8 es par (residuo = 0)
8	cubo_n = 4 * 4 * 4	64	**64**
9	cubo_u = 8 * 8 * 8	512	**512**

Prueba 2

Paso	Instrucción	Calculo	Resultado
1	---	---	---
2	Leer n	---	6
3	Leer u	---	3
4	residuo1 = 6 % 2	6/2 = 3 residuo 0	0
5	residuo2 = 3 % 2	3/2 = 1 residuo 1	1
6	n_par = (0 == 0)	V	El numero 6 es par (residuo = 0)
7	u_par = (1 == 0)	F	El numero 3 es impar (residuo = 1)
8	cubo_n = 6 * 6 * 6	**216**	**216**
9	cubo_u = 3 * 3 * 3	**27**	**27**

Codificación en lenguaje C

```c
C EJ8_010.c > main()
#include <stdio.h>
#include <stdbool.h>
void main(){
    //declarar variables y constantes
    int n, u, cubo_n, cubo_u, residuo1, residuo2;
    bool n_par, u_par;
    const int literal = 2;
    printf("Ingrese el primer valor entero: ");
    scanf("%d", &n);
    printf("Ingrese el segundo valor entero: ");
    scanf("%d", &u);
    //proceso
    residuo1 = n % literal;
    residuo2 = u % literal;
    n_par = residuo1 == 0;
    u_par = residuo2 == 0;
    cubo_n = n * n * n;
    cubo_u = u * u * u;
    if(n_par){
        printf("El numero %d es par (residuo = %d)\n", n, residuo1);
    }else{
        printf("El numero %d es impar (residuo = %d)\n", n, residuo1);
    }
    if(u_par){
        printf("El numero %d es par (residuo = %d)\n", u, residuo2);
    }else{
        printf("El numero %d es impar (residuo = %d)\n", u, residuo2);
    }
    printf("Cubo de %d: %d\n", n, cubo_n);
    printf("Cubo de %d: %d\n", u, cubo_u);
}
```

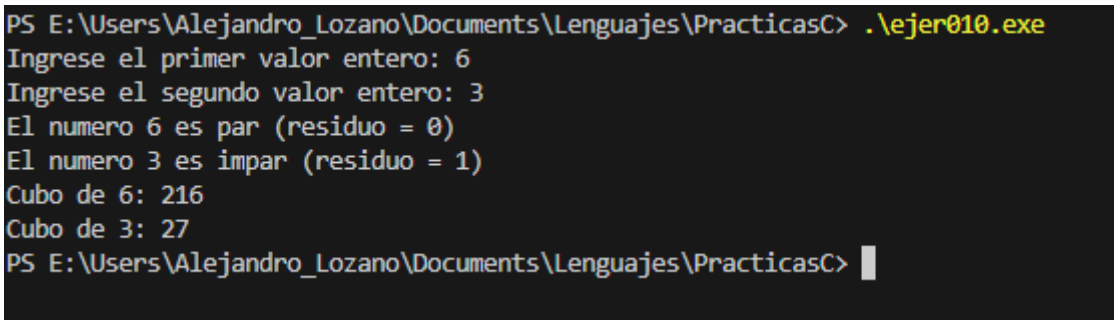
Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ8_010.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> gcc EJ8_010.c -o ejer010.exe
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer010.exe
Ingrese el primer valor entero: 4
Ingrese el segundo valor entero: 8
El numero 4 es par (residuo = 0)
El numero 8 es par (residuo = 0)
Cubo de 4: 64
Cubo de 8: 512
```

Prueba 2



Conclusiones del algoritmo

- Se uso la siguiente operación para calcular el cubo (n * n * n), ya que resultaba ser una opción más precisa, a diferencia de la función pow().

- La ejecución real (con n = 6 y u = 3) coincidió exactamente con la prueba de escritorio teórica.

Enunciado del algoritmo

EJ-039 (Ejercicio 9)

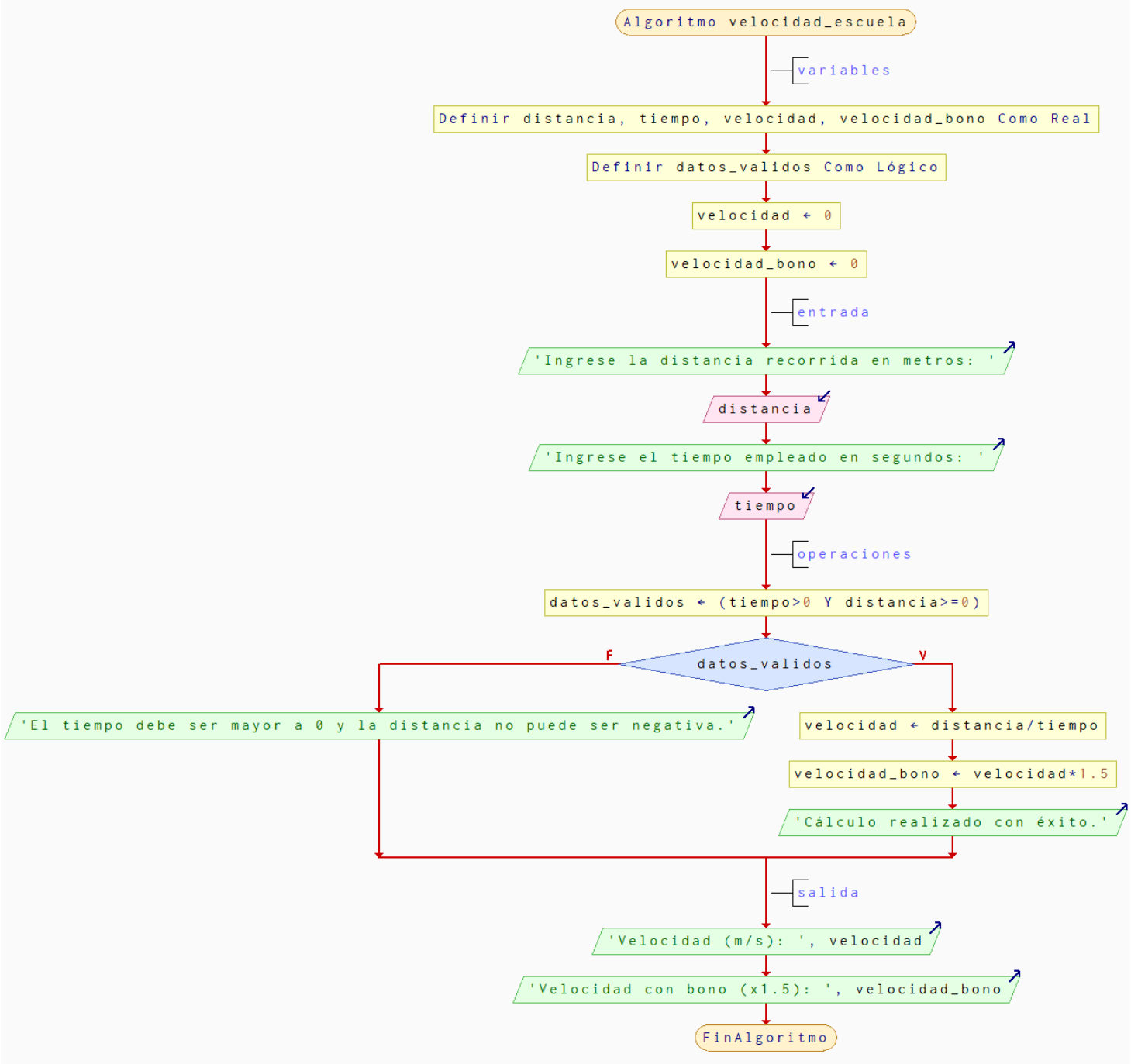
En una escuela, diseñe un algoritmo que lea la distancia recorrida (real, en metros) y el tiempo (real, en segundos). Calcule la velocidad con la expresión aritmética (distancia / tiempo) y asígnela a velocidad. Evalúe la expresión lógica (tiempo > 0 AND distancia >= 0). Muestre por pantalla la velocidad en m/s. Justifique el tipo de dato de cada variable. Además del cálculo principal, multiplique el resultado por 1.5 como bono. Agregue validaciones para que no haya resultados negativos.

Análisis del problema
Datos de entrada:

- **Variables:** distancia, tiempo

Proceso
Datos de salida: velocidad, velocidad_bono, datos_validos, mensaje
No se usan enteros porque el cálculo físico de velocidad involucra magnitudes continuas que requieren representación decimal.

Diagrama de flujo



Pseudocódigo

Inicio
//Variables
Definir distancia, tiempo, velocidad, velocidad_bono como real
Definir datos_validos como booleano
//Entrada
Imprimir ("Ingrese la distancia recorrida en metros: ")
Leer distancia
Imprimir ("Ingrese el tiempo empleado en segundos): "
Leer tiempo
//operaciones
datos_validos = (tiempo > 0 Y distancia >= 0)
Si datos_validos Entonces

```
        velocidad = distancia / tiempo
        velocidad_bono = velocidad * 1.5
        Imprimir = "Cálculo realizado con éxito."
Sino
        Imprimir (“El tiempo debe ser mayor a 0 y la distancia no puede ser
        negativa.")
FinSi
//Salida
Imprimir ("Velocidad (m/s): " + velocidad)
Imprimir ("Velocidad con bono (x1.5): " + velocidad_bono)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida en pantalla
1	distancia	-10	---
2	tiempo	0	---
3	tiempo > 0	Falso.	---
4	distancia >= 0	Falso	---
5	datos_validos	Falso	---
6	velocidad	0 (no se calcula)	---
7	velocidad_bono	0 (no se calcula)	El tiempo debe ser mayor que 0 La distancia no puede ser negativa

Prueba 2

Paso	Variable / expresión	Valor	Salida en pantalla
1	distancia	20	---
2	tiempo	5	---
3	tiempo > 0	Verdadero.	---
4	distancia >= 0	Verdadero	---
5	datos_validos	Falso	---
6	velocidad	4.00	---
7	velocidad_bono	6.00	Calculo realizado con éxito **Velocidad**: 4.00m/s **Velocidad con bono (x1.5):** 6.00 m/s

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    //declarar variables
    float distancia, tiempo, velocidad, velocidad_bono;
    bool datos_validos;
    //pedir datos
    printf("Ingrese la distancia recorrida en metros: ");
    scanf("%f", &distancia);
    printf("Ingrese el tiempo empleado en segundos: ");
    scanf("%f", &tiempo);
    //proceso
    datos_validos = (tiempo > 0 && distancia >= 0);
    if (datos_validos) {
        velocidad = distancia / tiempo;
        velocidad_bono = velocidad * 1.5;
        printf("Calculo realizado con exito\n");
    } else {
        printf("El tiempo debe ser mayor a 0 y la distancia no puede ser negativa\n");
    }
    //salida de resultados
    printf("Velocidad (m/s): %.2f\n", velocidad);
    printf("Velocidad con bono (x1.5): %.2f\n", velocidad_bono);
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ9_039.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer09.exe
Ingrese la distancia recorrida en metros: -10
Ingrese el tiempo empleado en segundos: 0
El tiempo debe ser mayor a 0 y la distancia no puede ser negativa
Velocidad (m/s): 0.00
Velocidad con bono (x1.5): 0.00
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer09.exe
Ingrese la distancia recorrida en metros: 20
Ingrese el tiempo empleado en segundos: 5
Calculo realizado con exito
Velocidad (m/s): 4.00
Velocidad con bono (x1.5): 6.00
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- Distancia, tiempo, velocidad y bono son reales porque pueden tener decimales. datos_validos es lógico porque solo toma dos valores.

- El 1.5 del bono está escrito directamente en la fórmula, como pide el enunciado. En un programa real sería mejor una constante con nombre, para cambiarlo en un solo lugar.

Enunciado del algoritmo

EJ-050 (Ejercicio 10)

En una cooperativa, construya un algoritmo que lea una altura en pies (real). Use una constante simbólica PIE_METRO = 0.3048. Calcule la equivalencia en metros con la expresión aritmética (pies * PIE_METRO) y asígnela a metros. Evalúe la expresión relacional (metros > 0). Muestre por pantalla 'X pies equivalen a Y metros'. Además, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

Datos de entrada:

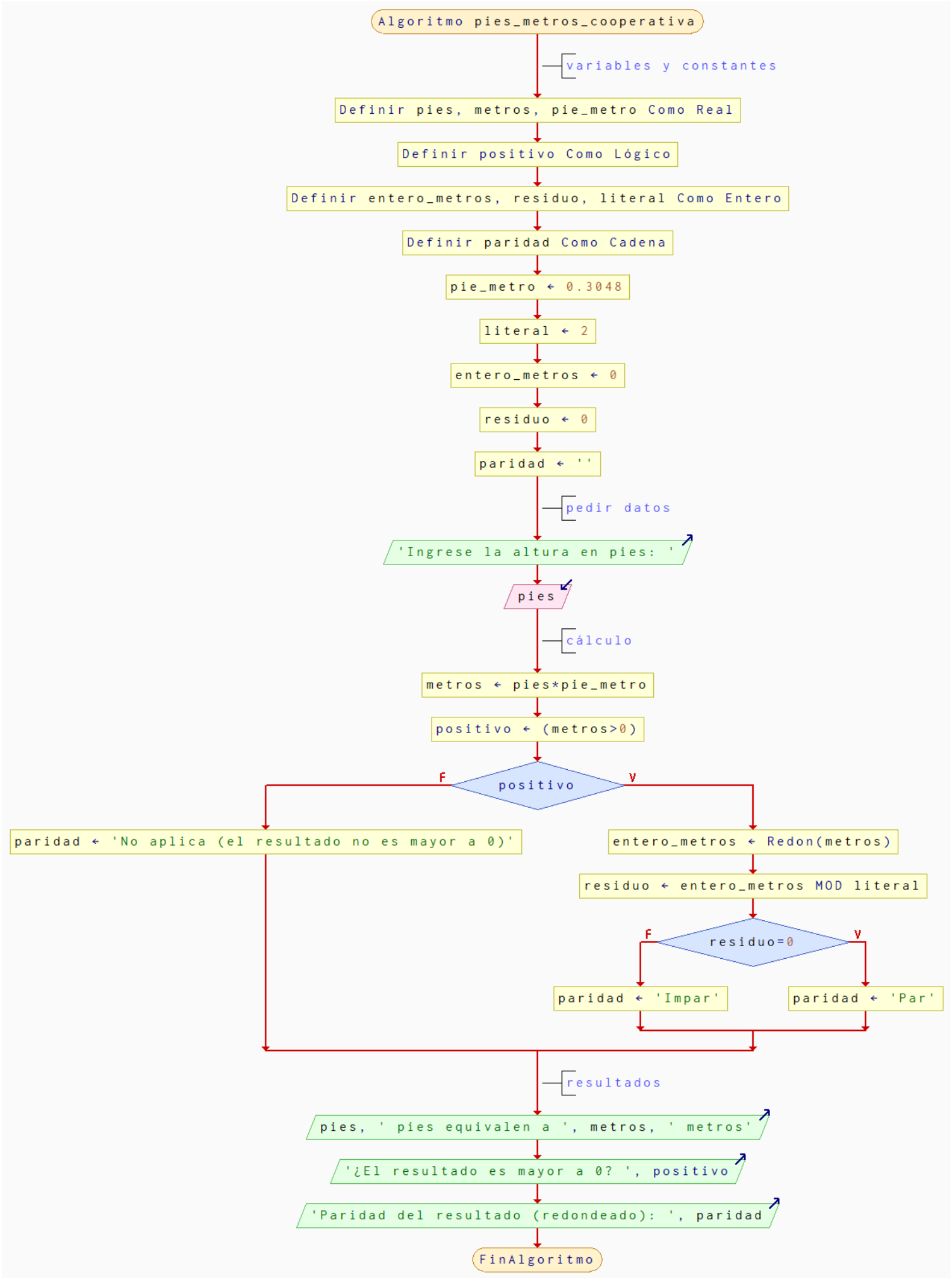
- **Variables:** pies
- **Constante:** pie_metro = 0.3048

Proceso

- **metros =** pies * pie_metro
- positivo = (metros > 0).

Datos de salida: metros, positivo, paridad

Diagrama de flujo



Pseudocódigo

```
Inicio
//variables y constantes
Definir pies, metros como real
Definir positivo como booleano
Definir entero_metros, residuo Como Entero
Definir paridad como cadena
pie_metro = 0.3048
//pedir datos
Imprimir ("Ingrese la altura en pies.")
Leer pies
 //Cálculo
metros = pies * pie_metro
positivo = (metros > 0)
Si (positivo) Entonces
      metros_entero = redondear(metros)
      Si (metros_entero % 2 == 0) Entonces
            paridad = "Par"
      Sino
            paridad = "Impar"
      FinSi
Sino
      paridad = "No aplica (el resultado no es mayor a 0)"
FinSi
Imprimir (pies + " pies equivalen a " + metros + " metros")
   Escribir ("¿El resultado es mayor a 0? " + positivo)
   Escribir ("Paridad del resultado (redondeado): " + paridad)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Instrucción	Cálculo	Resultado
1	Leer pies	---	20.0
2	metros = 20.0 * 0.3048	---	6.096
3	es_positivo = (6.096 > 0)	V	true
4	metros_entero = round(6.096)	---	6
5	6 % 2 == 0	0==0	Par

Prueba 2

Paso	Instrucción	Cálculo	Resultado
1	Leer pies	---	30.0
2	metros = 30.0 * 0.3048	---	9.144

3	es_positivo = (9.144 > 0)	V	true
4	metros_entero = round(9.144)	---	9
5	9 % 2 == 0	1==0	Impar

Codificación en lenguaje C

```c
EJ10_050.c > main()
#include <stdio.h>
#include <stdbool.h>
#include <math.h>
int main() {
    //variables y constantes
    float pies, metros;
    bool es_positivo;
    int metros_entero;
    const float pie_metro = 0.3048;
    const char *paridad = "No aplica";
    //pedir informacion
    printf("Altura en pies: ");
    scanf("%f", &pies);
    // Cálculo
    metros = pies * pie_metro;
    es_positivo = (metros > 0);
    if (es_positivo) {
        metros_entero = (int)roundf(metros); // Redondeo al entero más cercano
        if (metros_entero % 2 == 0) {
            paridad = "Par";
        } else {
            paridad = "Impar";
        }
    } else {
        paridad = "No aplica, el resultado no es mayor a 0";
    }
    // Salida
    printf("%.2f pies equivalen a %.2f metros\n", pies, metros);
    printf("El resultado es mayor a 0? %s\n", es_positivo ? "true" : "false");
    printf("Paridad del resultado (redondeado): %s\n", paridad);
    return 0;
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ10_050.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> gcc EJ10_050.c
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer10.exe
Altura en pies: 20
20.00 pies equivalen a 6.10 metros
El resultado es mayor a 0? true
Paridad del resultado (redondeado): Par
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer10.exe
Altura en pies: 30
30.00 pies equivalen a 9.14 metros
El resultado es mayor a 0? true
Paridad del resultado (redondeado): Impar
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- Para lograr calcular la paridad fue necesario redondear el valor real a entero ya que un numero par o impar solo tiene sentido en valores enteros

- Los mensajes informativos hacen visible el flujo, ya que con ellos se ve qué está haciendo el programa en cada momento (leer, convertir, evaluar, verificar). Eso ayuda a probar el algoritmo y a explicarlo en la prueba de escritorio.

Enunciado del algoritmo

EJ-042 (Ejercicio 11)

En una juguetería, diseñe un algoritmo que lea el peso (real, en kg) y la altura (real, en metros). Calcule el IMC con la expresión aritmética (peso / (altura * altura)) y asígnelo a imc. Evalúe las expresiones lógicas (imc < 18.5) y (imc > 25.0). Muestre por pantalla el IMC y un mensaje indicando si está bajo o sobre el rango normal. Además, calcule el cuadrado del resultado principal. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos de entrada:

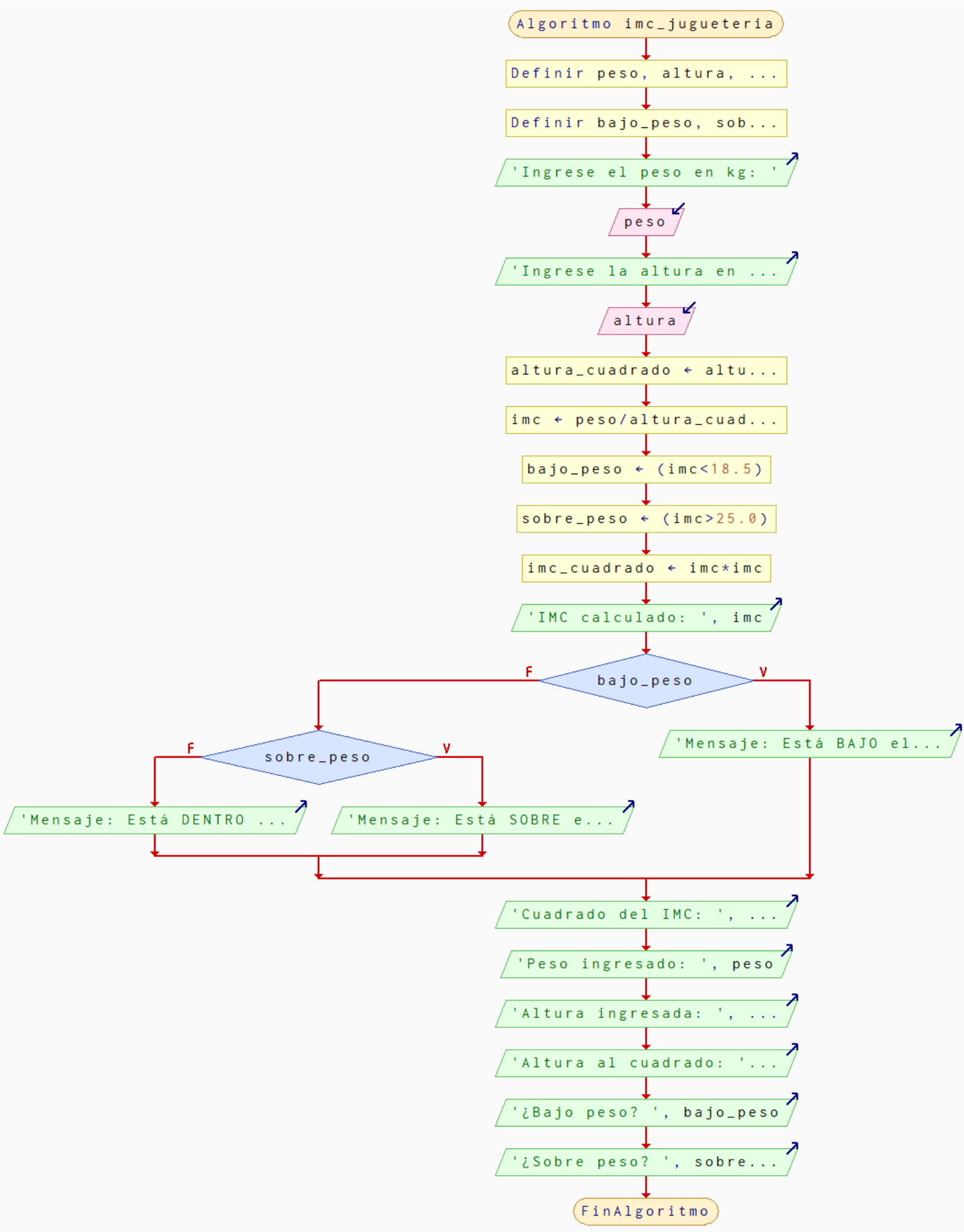
- **Variables:** peso, altura

Proceso

- datos_validos = (peso > 0 Y altura >0)
- imc = peso / (altura * altura)
- bajo_peso = (imc < 18.5) y sobre_peso = (imc > 25.0)
- imc_cuadrado = imc * imc
- altura_cuadrado = altura * altura * altura

Datos de salida: imc, bajo_peso, sobre_peso, imc_cuadrado, altura_cuadrado.

Diagrama de flujo



Pseudocódigo

```
Inicio
//Variables y constantes
Definir peso, altura, imc, imc_cuadrado, altura_cuadrado como real
Definir bajo_peso, sobre_peso como booleano
//Pedir información
Imprimir ("Ingrese el peso en kg: ")
Leer peso
Imprimir ("Ingrese la altura en metros: ")
Leer altura
//Proceso
altura_cuadrado = altura * altura
imc = peso / altura_cuadrado
bajo_peso = (imc < 18.5)
sobre_peso = (imc > 25.0)
imc_cuadrado <- imc * imc
Imprimir ("IMC calculado: " + imc)
Si (bajo_peso) Entonces
                Imprimir ("Está BAJO el rango normal (IMC < 18.5)")
Sino
    Si (sobre_peso) Entonces
                Imprimir ("Mensaje: Está SOBRE el rango normal (IMC > 25.0)")
    Sino
                Imprimir ("Mensaje: Está DENTRO del rango normal (18.5 - 25.0)")
    FinSi
FinSi

Imprimir ("Cuadrado del IMC: " + imc_cuadrado)
Imprimir ("Peso ingresado: " + peso)
Imprimir ("Altura ingresada: " + altura)
Imprimir ("Altura al cuadrado: " + altura_cuadrado)
Imprimir ("¿Bajo peso? " + bajo_peso)
Imprimir ("¿Sobre peso? " + sobre_peso)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Instrucción	Cálculo	Resultado
1	Leer peso	---	51.0
2	Leer altura	---	1.53
3	altura_cuadrado = 1.53 * 1.53	---	2.3409
4	imc = 51.0 / 2.3409	---	21.78
5	bajo_peso = (21.78 < 18.5)	Falso	False

6	sobre_peso = (21.78 > 25.0)	Falso	false
7	imc_cuadrado = 21.78 * 21.78	---	474.65

Prueba 2

Paso	Instrucción	Cálculo	Resultado
1	Leer peso	---	70
2	Leer altura	---	1.57
3	altura_cuadrado = 1.57 * 1.57	---	2.4649
4	imc = 70 / 2.4649	---	28.40
5	bajo_peso = (28.40 < 18.5)	Falso	False
6	sobre_peso = (28.40> 25.0)	Verdadero	True
7	imc_cuadrado = 28.40 * 28.40	---	806.49

Codificación en lenguaje C

```c
int main() {
    //variables
    float peso, altura, imc, imc_cuadrado, altura_cuadrado;
    bool bajo_peso, sobre_peso;
    //pedir informacion
    printf("Ingrese el peso en kg: ");
    scanf("%f", &peso);
    printf("Ingrese la altura en metros: ");
    scanf("%f", &altura);
    //Proceso
    altura_cuadrado = altura * altura;
    imc = peso / altura_cuadrado;
    bajo_peso = (imc < 18.5);
    sobre_peso = (imc > 25.0);
    imc_cuadrado = imc * imc;
    //salida
    printf("IMC calculado: %.2f\n", imc);
    if (bajo_peso) {
        printf("Esta BAJO el rango normal (IMC < 18.5)\n");
    } else if (sobre_peso) {
        printf("Esta SOBRE el rango normal (IMC > 25.0)\n");
    } else {
        printf("Esta DENTRO del rango normal (18.5 - 25.0)\n");
    }
    printf("Cuadrado del IMC: %.2f\n", imc_cuadrado);
    printf("Peso ingresado:      %.2f kg\n", peso);
    printf("Altura ingresada:    %.2f m\n", altura);
    printf("Altura al cuadrado:  %.4f m2\n", altura_cuadrado);
    printf("Bajo peso?           %s\n", bajo_peso ? "true" : "false");
    printf("Sobre peso?          %s\n", sobre_peso ? "true" : "false");
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_fin

al/EJ11_042.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer11.exe
Ingrese el peso en kg: 51
Ingrese la altura en metros: 1.53
IMC calculado: 21.79
Esta DENTRO del rango normal (18.5 - 25.0)
Cuadrado del IMC: 474.65
Peso ingresado:      51.00 kg
Altura ingresada:    1.53 m
Altura al cuadrado:  2.3409 m2
```

```
Bajo peso?           false
Sobre peso?          false
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer11.exe
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer11.exe
Ingrese el peso en kg: 70
Ingrese la altura en metros: 1.57
IMC calculado: 28.40
Esta SOBRE el rango normal (IMC > 25.0)
Cuadrado del IMC: 806.49
Peso ingresado:      70.00 kg
Altura ingresada:    1.57 m
Altura al cuadrado:  2.4649 m2
```

```
Altura al cuadrado:  2.4649 m2
Bajo peso?           false
Sobre peso?          true
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\Pra
```

Conclusiones del algoritmo

- El algoritmo cumple lo que pide el enunciado. Lee el peso y la altura, calcula el IMC, indica si está bajo, sobre o dentro del rango normal, calcula su cuadrado y muestra las variables auxiliares.

- Las variables lógicas nos permiten saber de forma rápida en qué situación se encuentra la persona. Al comparar el IMC con los límites de 18.5 y 25.0, el algoritmo puede identificar si la persona tiene bajo peso, peso normal o sobrepeso de una manera muy clara.

Enunciado del algoritmo

EJ-037 (Ejercicio 12)

En un hospital, cree un algoritmo que lea un numero entero y por teclado. Use constantes literales 2 y 3 como factores. Calcule el doble con (2 * y) y el triple con (3 * y). Asigne los resultados a variables doblo y triple. Evalúe la expresión lógica (doble < triple AND doble > 0). Muestre por pantalla ambos resultados. Además, calcule el inverso multiplicativo del resultado si no es cero. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos de entrada:

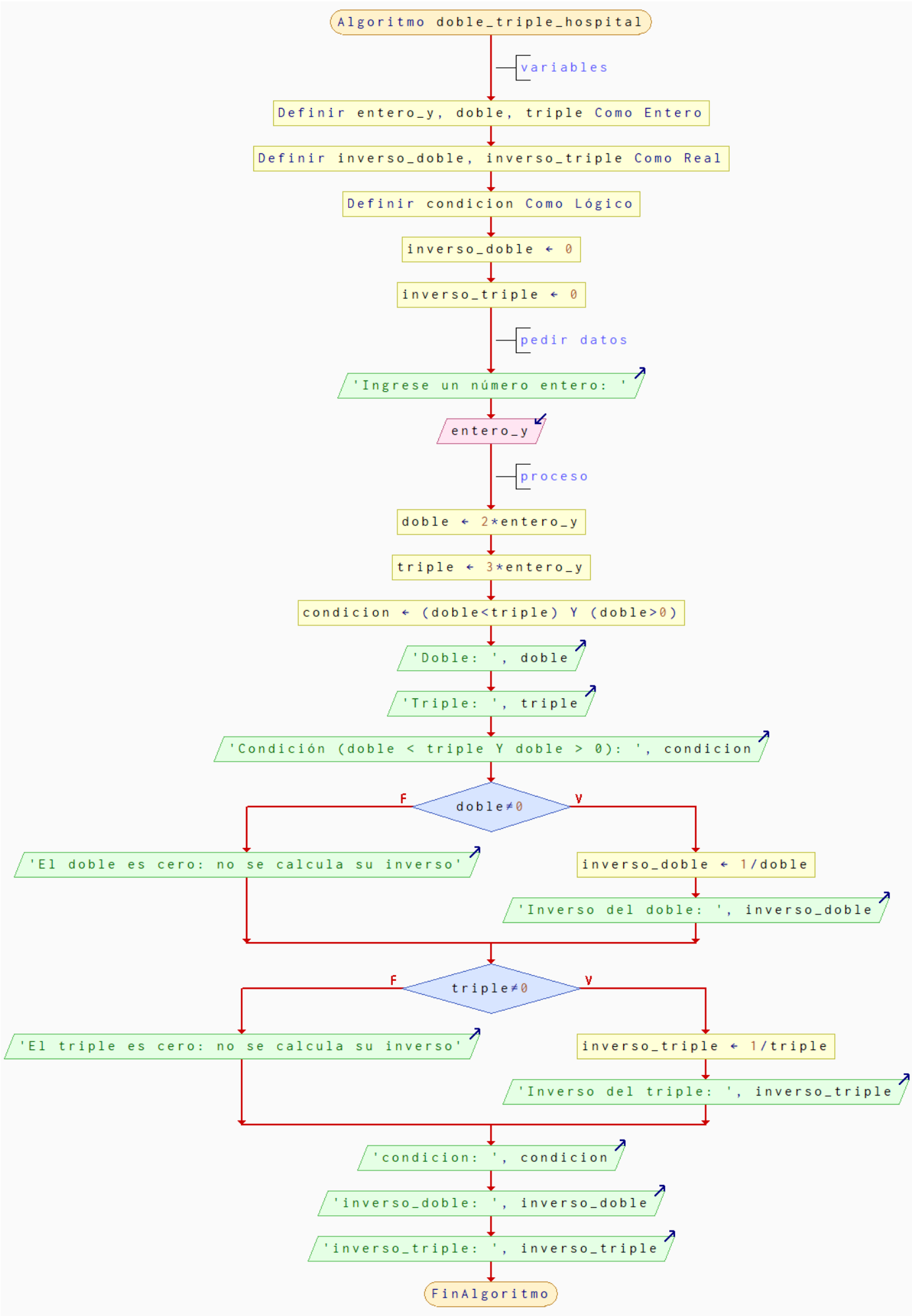
- **Variables:** y

Proceso

- doble = 2 * y
- triple =3 * y
- condicion = (doble < triple AND doble > 0)

Datos de salida: doble, triple, condición, inverso_doble, inverso_triple

Diagrama de flujo



Pseudocódigo

```
Inicio
//variables
Definir y, doble, triple Como Entero
Definir inverso_doble, inverso_triple Como Real
Definir condicion Como Logico
//pedir datos
Imprimir ("Ingrese un numero entero: ")
Leer y
//Proceso
doble = 2 * y
triple = 3 * y
condicion = (doble < triple) Y (doble > 0)
Imprimir ("Doble: " + doble)
Imprimir ("Triple: " + triple)
Imprimir ("Condicion (doble < triple Y doble > 0): " + condición)
Si (doble <> 0) Entonces
        inverso_doble = 1 / doble
        Imprimir ("Inverso del doble: " + inverso_doble)
SiNo
        Imprimir ("El doble es cero: no se calcula su inverso")
FinSi
Si (triple <> 0) Entonces
        inverso_triple = 1 / triple
        Imprimir ("Inverso del triple: " + inverso_triple)
SiNo
        Imprimir ("El triple es cero: no se calcula su inverso")
FinSi
Imprimir ("condicion: ", condición)
Imprimir ("inverso_doble: " + inverso_doble)
Imprimir ("inverso_triple: " + inverso_triple)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida en pantalla
1	y	2	---
2	doble (2 * 2)	4	---
3	triple (3 * 2)	6	---
4	doble < triple	Verdadero	---
5	doble > 0	Verdadero	---
6	Condición (doble < triple Y doble > 0)	Verdadero	---

7	inverso_doble (1 / 4)	0.25	---
8	inverso_triple (1 / 6)	0.1667	**Doble:** 4 **Triple:** 6 **Condicion**: verdadero **Inverso del doble:** 0.2500 **Inverso del triple:** 0.1667 **condicion:** verdadero **inverso_doble**: 0.2500

Prueba 2

Paso	Variable / expresión	Valor	Salida en pantalla
1	y	7	---
2	doble (2 * 7)	14	---
3	triple (3 * 7)	21	---
4	doble < triple	Verdadero	---
5	doble > 0	Verdadero	---
6	Condición (doble < triple Y doble > 0)	Verdadero	---
7	inverso_doble (1 / 14)	0.0714	---
8	inverso_triple (1 / 21)	0.0476	**Doble:** 14 **Triple:** 21 **Condicion**: verdadero **Inverso del doble:** 0.0714 **Inverso del triple:** 0.0476 **condicion:** 0.0714 **inverso_doble**: 0.0476

Codificación en lenguaje C

EJ12_037.c > main()

```c
#include <stdio.h>
#include <stdbool.h>
void main() {
    //variables
    int y, doble, triple;
    double inverso_doble, inverso_triple;
    bool condicion = false;
    //pedir informacion
    printf("Ingrese un numero entero : ");
    scanf("%d", &y);
    doble = 2 * y;
    triple = 3 * y;
    condicion = (doble < triple) && (doble > 0);
    //salida
    printf("Doble: %d\n", doble);
    printf("Triple: %d\n", triple);
    printf("Condicion (doble < triple Y doble > 0): %s\n", condicion ? "verdadero" : "falso");

    if (doble != 0) {
        inverso_doble = 1.0 / doble;
        printf("Inverso del doble: %.4f\n", inverso_doble);
    } else {
        printf("El doble es cero: no se calcula su inverso\n");
    }
```

```c
    if (triple != 0) {
        inverso_triple = 1.0 / triple;
        printf("Inverso del triple: %.4f\n", inverso_triple);
    } else {
        printf("El triple es cero: no se calcula su inverso\n");
    }
    printf("condicion: %s\n", condicion ? "verdadero" : "falso");
    printf("inverso_doble: %.4f\n", inverso_doble);
    printf("inverso_triple: %.4f\n", inverso_triple);

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ12_037.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer12.exe
Ingrese un numero entero y: 2
Doble: 4
Triple: 6
Condicion (doble < triple Y doble > 0): verdadero
Inverso del doble: 0.2500
Inverso del triple: 0.1667
condicion: verdadero
```

```
condicion: verdadero
inverso_doble: 0.2500
inverso_triple: 0.1667
```

Prueba 2

```
Ingrese un numero entero : 7
Doble: 14
Triple: 21
Condicion (doble < triple Y doble > 0): verdadero
Inverso del doble: 0.0714
Inverso del triple: 0.0476
condicion: verdadero
inverso_doble: 0.0714
inverso_triple: 0.0476
```

Conclusiones del algoritmo

- El inverso solo se calcula cuando el resultado no es cero. Dividir 1 entre 0 no es posible, así que el algoritmo lo evita con una comprobación. Esto solo ocurre cuando y es 0.

- El algoritmo cumple lo que pide el enunciado. Lee un entero, calcula el doble y el

triple con los factores 2 y 3, evalúa la expresión lógica, calcula los inversos y muestra las variables auxiliares.

Enunciado del algoritmo

EJ-034 (Ejercicio 13)

En una zapatería, cree un algoritmo que lea un año (entero) por teclado. Use constantes literales 4, 100 y 400. Evalúe la expresión lógica compuesta ((año % 4 == 0) AND (año % 100 != 0 OR año % 400 == 0)). Asigne el resultado booleano a es bisiesto. Muestre por pantalla si el año es o no bisiesto. Justifique el uso de los operadores AND y OR en la expresión. Además, calcule el valor mínimo de todas las variables utilizadas. Agregue validaciones para que no haya resultados negativos.

Análisis del problema

Datos de entrada:

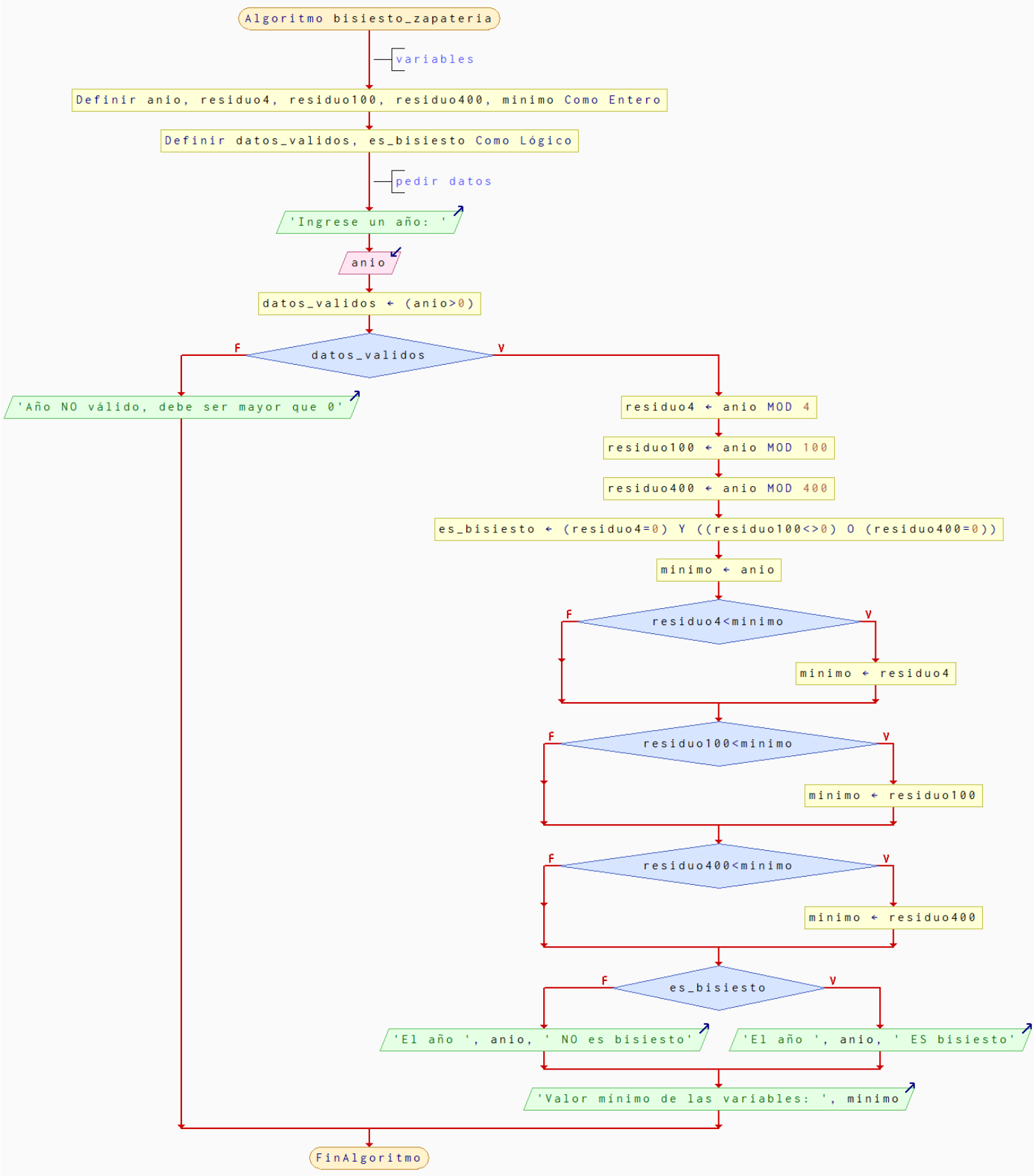
- **Variables:** anio

Proceso

- datos_validos = (anio > 0).
- residuo4 = anio % 4
- residuo100 = anio % 100
- residuo400 = anio % 400.
- es_bisiesto = ((residuo4 == 0) AND ((residuo100 != 0) OR (residuo400 == 0))).

Datos de salida: es_bisiesto, minimo.

Diagrama de flujo



Pseudocódigo

Inicio

//variables

Definir anio, residuo4, residuo100, residuo400, minimo Como Entero

Definir datos_validos, es_bisiesto Como Logico

//pedir datos

```
Imprimir ("Ingrese un año: ")
Leer anio
datos_validos = (anio > 0)
Si (datos_validos) Entonces
        residuo4 = anio % 4
        residuo100 = anio % 100
        residuo400 = anio % 400
        es_bisiesto = (residuo4 = 0) Y ((residuo100 != 0) O (residuo400 = 0))
        minimo = anio
        Si (residuo4 < mínimo) Entonces
                minimo = residuo4
        FinSi
        Si (residuo100 < mínimo) Entonces
                minimo = residuo100
        FinSi
        Si (residuo400 < mínimo) Entonces
                minimo = residuo400
        FinSi

        Si (es_bisiesto) Entonces
                Imprimir ("El año " + anio + " ES bisiesto")
        SiNo
                Imprimir ("El año " + anio + " NO es bisiesto")
        FinSi
        Imprimir ("Valor minimo de las variables: " + mínimo)
SiNo
        Imprimir ("Año NO valido, debe ser mayor que 0")
FinSi
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable	Valor	Salida
1	anio	2026	---
2	datos_validos	Verdadero	---
3	residuo4 (2026 % 4)	2	---
4	residuo100 (2026 % 100)	26	---
5	residuo400 (2026 % 400)	26	---
6	residuo4 == 0	Falso	---
7	residuo100 != 0	Verdadero	---
8	residuo400 == 0	Falso	---
9	es_bisiesto	Falso	---
10	minimo	2	**Ingrese un anio:** 2026

			El anio 2026 NO es bisiesto **Valor mínimo de las variables:** 2

Prueba 2

Paso	Variable	Valor	Salida
1	anio	2000	---
2	datos_validos	Verdadero	---
3	residuo4 (2026 % 4)	0	---
4	residuo100 (2026 % 100)	0	---
5	residuo400 (2026 % 400)	0	---
6	residuo4 == 0	Verdadero	---
7	residuo100 != 0	Falso	---
8	residuo400 == 0	Verdadero	---
9	es_bisiesto	Verdadero	---
10	minimo	0	**Ingrese un anio:** 2000 **El anio 2000 ES bisiesto** **Valor mínimo de las variables:** 0

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
void main (){
    int anio, residuo4, residuo100, residuo400, minimo;
    bool datos_validos, es_bisiesto;
    printf("Ingrese el anio: ");
    scanf("%d", &anio);
    datos_validos = anio > 0;
    if (datos_validos){
        residuo4 = anio % 4;
        residuo100 = anio % 100;
        residuo400 = anio % 400;
        es_bisiesto = (residuo4 == 0) && ((residuo100 != 0) || (residuo400 == 0));
        minimo = anio;
        if (residuo4 < minimo){
            minimo = residuo4;
        }
        if (residuo100 < minimo){
            minimo = residuo100;
        }
        if (residuo400 < minimo){
            minimo = residuo400;
        }
```

```
        if (es_bisiesto){
            printf("El anio %d ES bisiesto\n", anio);
        }else{
            printf("El anio %d NO es bisiesto\n", anio);
        }
        printf("Valor minimo de las variables: %d", minimo);
    }else{
        printf("Anio NO valido, debe ser mayor a 0");
    }
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ13_034.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer13.exe
Ingrese el anio: 2026
El anio 2026 NO es bisiesto
Valor minimo de las variables: 2
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ej
Ingrese el anio: 2000
El anio 2000 ES bisiesto
Valor minimo de las variables: 0
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El AND y el OR trabajan juntos. El AND exige que el año sea divisible entre 4. El OR permite que un año de siglo (divisible entre 100) sea bisiesto solo si también es divisible entre 400.

- La validación evita resultados negativos. Al exigir que el año sea mayor que 0, no se calculan residuos de números negativos, que en C pueden dar valores negativos.

Enunciado del algoritmo

EJ-401 (Ejercicio 14)

En una empresa de mensajería, diseñe un algoritmo que lea dos números reales por teclado. Use la constante literal 2 como divisor. Calcule el promedio con la expresión aritmética ((num1 + num2) / 2) y asígnelo a promedio. Evalúe la expresión relacional (promedio > 0).

Muestre por pantalla el promedio calculado. Identifique el tipo de dato de cada variable. Además del cálculo principal, calcule el logaritmo en base 10 del resultado. Muestre una tabla simulada con los datos y resultados al final.

Análisis del problema

Datos de entrada:

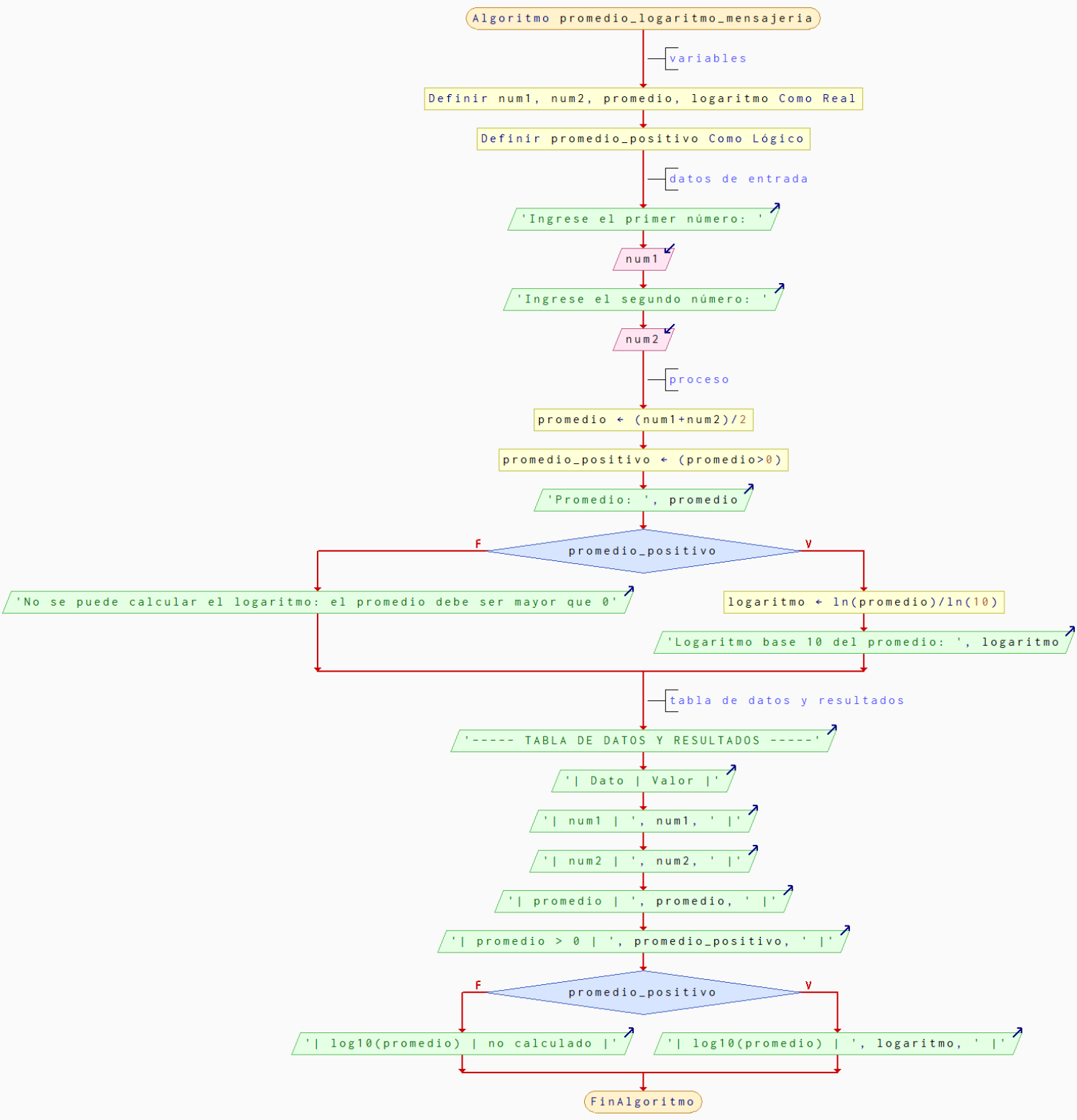
- **Variables:** num1, num2

Proceso

- promedio = (num1 + num2) / literal.
- promedio_positivo = (promedio > 0).
- logaritmo = log10(promedio)

Datos de salida: promedio, logaritmo

Diagrama de flujo



Pseudocódigo

Inicio

//variables

Definir num1, num2, promedio, logaritmo Como Real

Definir promedio_positivo Como Logico

//Datos de entrada

Imprimir ("Ingrese el primer número: ")

Leer num1

Imprimir (“Ingrese el segundo número: ")

```
Leer num2
//Proceso
promedio = (num1 + num2) / 2
promedio_positivo = (promedio > 0)
Imprimir ("Promedio: " + promedio)
Si (promedio_positivo) Entonces
        logaritmo = LN(promedio) / LN(10)
        Imprimir ("Logaritmo base 10 del promedio: " + logaritmo)
SiNo
        Imprimir ("No se puede calcular el logaritmo: el promedio debe ser mayor que 0")
FinSi
Imprimir ("----- TABLA DE DATOS Y RESULTADOS -----")
Imprimir ("| Dato | Valor |")
Imprimir ("| num1 | " + num1, " |")
Imprimir ("| num2 | " + num2, " |")
Imprimir ("| promedio | " + promedio, " |")
Imprimir ("| promedio > 0 | " + promedio_positivo, " |")
Si (promedio_positivo) Entonces
        Imprimir ("| log10(promedio) | " + logaritmo, " |")
SiNo
        Imprimir ("| log10(promedio) | no calculado |")
FinSi
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida
1	num1	10	---
2	num2	9	---
3	promedio ((10 + 9) / 2)	9.5	---
4	promedio > 0	Verdadero	---
5	logaritmo (log10 de 9.5)	0.9777	**Ingrese el primer número**: 10 **Ingrese el segundo número:** 9 **Promedio:** 9.50 **Logaritmo base 10 del promedio:** 0.9777

Prueba 2

Paso	Variable / expresión	Valor	Salida

1	num1	7	---
2	num2	8	---
3	promedio ((7 + 8) / 2)	7.5	---
4	promedio > 0	Verdadero	---
5	logaritmo (log10 de 7.5)	0.8751	**Ingrese el primer número**: 7 **Ingrese el segundo número:** 8 **Promedio:** 7.5 **Logaritmo base 10 del promedio:** 0.8751

Codificación en lenguaje C

EJ14_401.c > main()

```c
#include <stdio.h>
#include <math.h>
#include <stdbool.h>
void main() {
    //variables
    double num1, num2, promedio, logaritmo;
    bool promedio_positivo;
    //pedir informacion
    printf("Ingrese el primer numero: ");
    scanf("%lf", &num1);
    printf("Ingrese el segundo numero: ");
    scanf("%lf", &num2);
    //calculos
    promedio = (num1 + num2) / 2;
    promedio_positivo = (promedio > 0);
    printf("\nPromedio: %.2f\n", promedio);
    if (promedio_positivo) {
        logaritmo = log10(promedio);
        printf("Logaritmo base 10 del promedio: %.4f\n", logaritmo);
    } else {
        printf("No se puede calcular el logaritmo, el promedio debe ser mayor que 0\n");
    }
```

```c
    printf("\n----- TABLA DE DATOS Y RESULTADOS -----\n");
    printf("| %-20s | %11s |\n", "Dato", "Valor");
    printf("|%s|%s|\n", "----------------------", "-------------");
    printf("| %-20s | %11.4f |\n", "num1", num1);
    printf("| %-20s | %11.4f |\n", "num2", num2);
    printf("| %-20s | %11.4f |\n", "promedio", promedio);
    printf("| %-20s | %11s |\n", "promedio > 0", promedio_positivo ? "verdadero" : "falso");
    if (promedio_positivo) {
        printf("| %-20s | %11.4f |\n", "log10(promedio)", logaritmo);
    } else {
        printf("| %-20s | %11s |\n", "log10(promedio)", "no calculado");
    }

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ14_401.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer14.exe
Ingrese el primer numero: 10
Ingrese el segundo numero: 9
Promedio: 9.50
Logaritmo base 10 del promedio: 0.9777

----- TABLA DE DATOS Y RESULTADOS -----
| Dato                 |       Valor |
|----------------------|-------------|
| num1                 |     10.0000 |
| num2                 |      9.0000 |
| promedio             |      9.5000 |
| promedio > 0         |   verdadero |
| log10(promedio)      |      0.9777 |
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer1
Ingrese el primer numero: 7
Ingrese el segundo numero: 8
Promedio: 7.50
Logaritmo base 10 del promedio: 0.8751

----- TABLA DE DATOS Y RESULTADOS -----
| Dato                 |       Valor |
|----------------------|-------------|
| num1                 |      7.0000 |
| num2                 |      8.0000 |
| promedio             |      7.5000 |
| promedio > 0         |   verdadero |
| log10(promedio)      |      0.8751 |
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El algoritmo cumple lo que pide el enunciado, lee dos números reales, calcula el promedio dividiendo entre 2, evalúa si es positivo, calcula su logaritmo en base 10 y muestra la tabla final.

- La tabla final resume todo el proceso. Muestra los datos de entrada, el promedio, el resultado de la comparación y el logaritmo en un solo lugar, lo que facilita comprobar el resultado.

Enunciado del algoritmo

EJ-447 (Ejercicio 15)

En una tienda de ropa, cree un algoritmo que lea un número real a por teclado. Use la constante literal 2 como exponente. Calcule la potencia con la expresión aritmética (a ^ 2) y asígnela a cuadrado. Evalúe la expresión relacional (cuadrado >= 0). Muestre por pantalla 'El cuadrado de X es Y'. Además del cálculo principal, calcule el resto de una división adicional. Incluya un mensaje motivacional al final del algoritmo.

Análisis del problema

Datos de entrada:

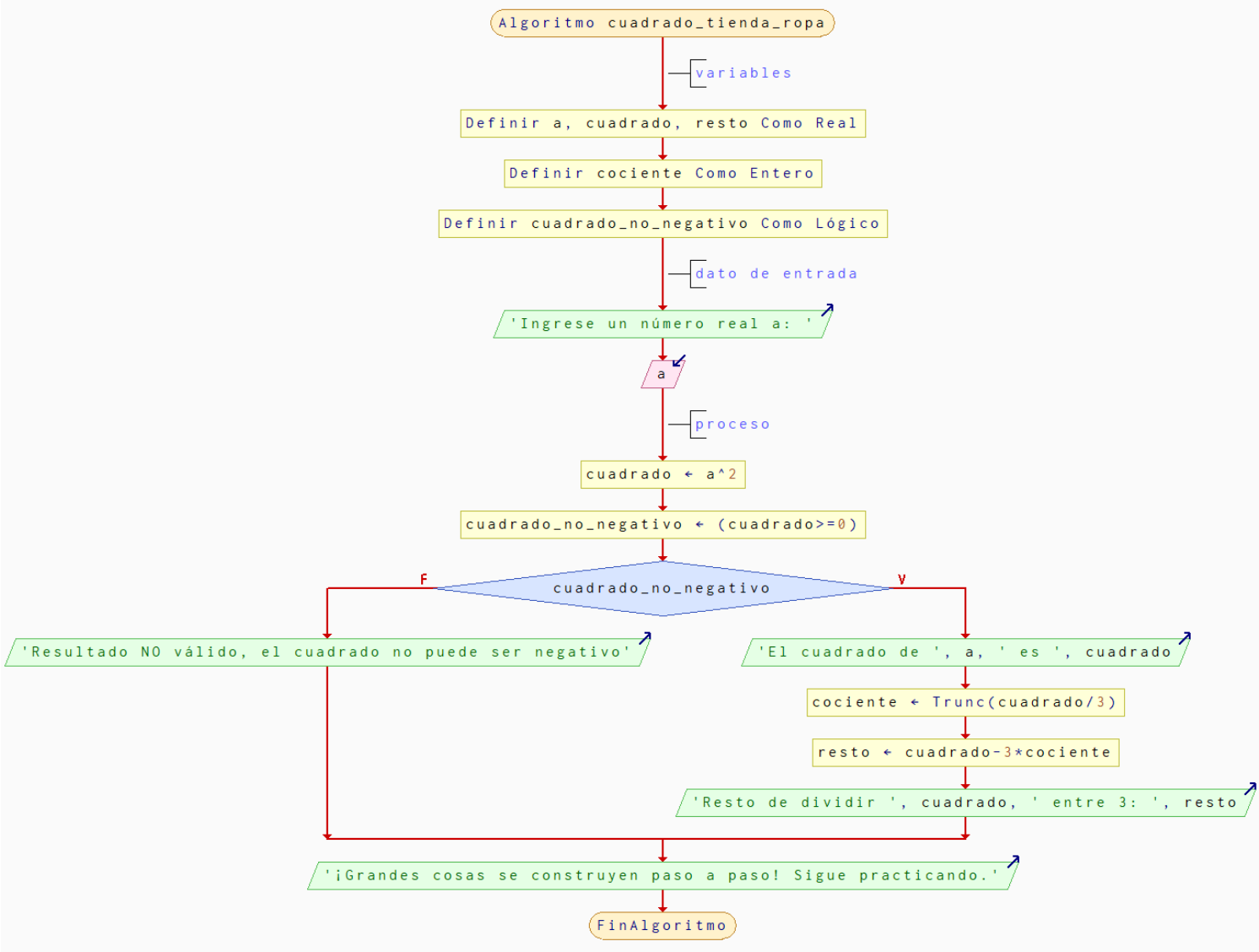
- **Variables:** a

Proceso

- Cuadrado = a ^ 2
- cuadrado_no_negativo = (cuadrado >= 0).

Datos de salida: resto, cuadrado, cuadrado_no_negativo, cociente.

Diagrama de flujo



Pseudocódigo

Inicio
//variables
Definir a, cuadrado, resto Como Real
Definir cociente Como Entero
Definir cuadrado_no_negativo Como Logico
//Dato de entrada
Imprimir (“Ingrese un número real a: ")
Leer a
//Proceso
cuadrado = a ^ 2
cuadrado_no_negativo = (cuadrado >= 0)
Si (cuadrado_no_negativo) Entonces

```
        Escribir ("El cuadrado de " + a + " es ", +cuadrado)
        cociente = Trunc(cuadrado / 3)
        resto = (cuadrado - 3 * cociente)
        Imprimir ("Resto de dividir " + cuadrado + " entre 3: " + resto)
SiNo
        Imprimir ("Resultado NO valido, el cuadrado no puede ser negativo")
FinSi
Escribir (" ¡Grandes cosas se construyen paso a paso! Sigue practicando.")
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida
1	a	4	
2	cuadrado (4 ^ 2)	16	
3	cuadrado >= 0	Verdadero	
4	cociente (16 / 3, parte entera)	5	
5	resto (16 - 3 × 5)	1	**Ingrese un número real a:** 4 **El cuadrado de** 4.00 **es** 16.00 **Resto de dividir** 16.00 **entre 3:** 1.00 ¡Excelente trabajo! Sigue practicando, cada algoritmo te acerca más a tus metas.

Prueba 2

Paso	Variable / expresión	Valor	Salida
1	a	93	
2	cuadrado (93^ 2)	8649	
3	cuadrado >= 0	Verdadero	
4	cociente (8649/ 3, parte entera)	2883	
5	resto (8649- 3 × 2883)	0	**Ingrese un número real a:** 93 **El cuadrado de** 93.00 **es** 8649.00 **Resto de dividir 8649.00 entre 3:** 0.00 ¡Excelente trabajo! Sigue practicando, cada algoritmo te acerca más a tus metas.

Codificación en lenguaje C

EJ15_447.c > main(void)

```c
#include <stdio.h>
#include <stdbool.h>
int main(void) {
    //variables
    double a, cuadrado, resto;
    long long cociente;
    bool cuadrado_no_negativo;
    //pedir informacion
    printf("Ingrese un numero real: ");
    scanf("%lf", &a);
    //calculos
    cuadrado = a * a;
    cuadrado_no_negativo = (cuadrado >= 0);
    if (cuadrado_no_negativo) {
        printf("El cuadrado de %.2f es %.2f\n", a, cuadrado);
        cociente = (long long) (cuadrado / 3);
        resto = cuadrado - 3 * cociente;
        printf("Resto de dividir %.2f entre 3: %.2f\n", cuadrado, resto);
    } else {
        printf("Resultado NO valido: el cuadrado no puede ser negativo\n");
    }
    printf("Grandes cosas se construyen paso a paso! Sigue practicando.\n");
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ15_447.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer1
Ingrese un numero real: 4
El cuadrado de 4.00 es 16.00
Resto de dividir 16.00 entre 3: 1.00
Grandes cosas se construyen paso a paso! Sigue practicando.
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer
Ingrese un numero real: 93
El cuadrado de 93.00 es 8649.00
Resto de dividir 8649.00 entre 3: 0.00
Grandes cosas se construyen paso a paso! Sigue practicando.
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El mensaje motivacional cierra el algoritmo. Se muestra siempre al final, sin importar el resultado, y su texto puede ser el que prefieras.

- El resto depende del número ingresado. Con 93, que es múltiplo de 3, su cuadrado también lo es y el resto fue 0. Con 4, que no es múltiplo de 3, el resto fue 1.

Enunciado del algoritmo

EJ-449 (Ejercicio 16)

En una librería, diseñe un algoritmo que lea una temperatura en grados Fahrenheit (real). Use constantes simbólicas FACTOR = 0.5556 y OFFSET_K = 273.15. Calcule la temperatura en Kelvin con la expresión aritmética ((Fahrenheit - 32) * FACTOR + OFFSET_K) y asígnela a kelvin. Evalúe la expresión relacional (kelvin >= 0). Muestre por pantalla 'X F equivalen a Y K'. Además, determine si el resultado es par o impar (cuando aplique). Incluya un encabezado descriptivo al iniciar la ejecución.

Análisis del problema

Datos de entrada:

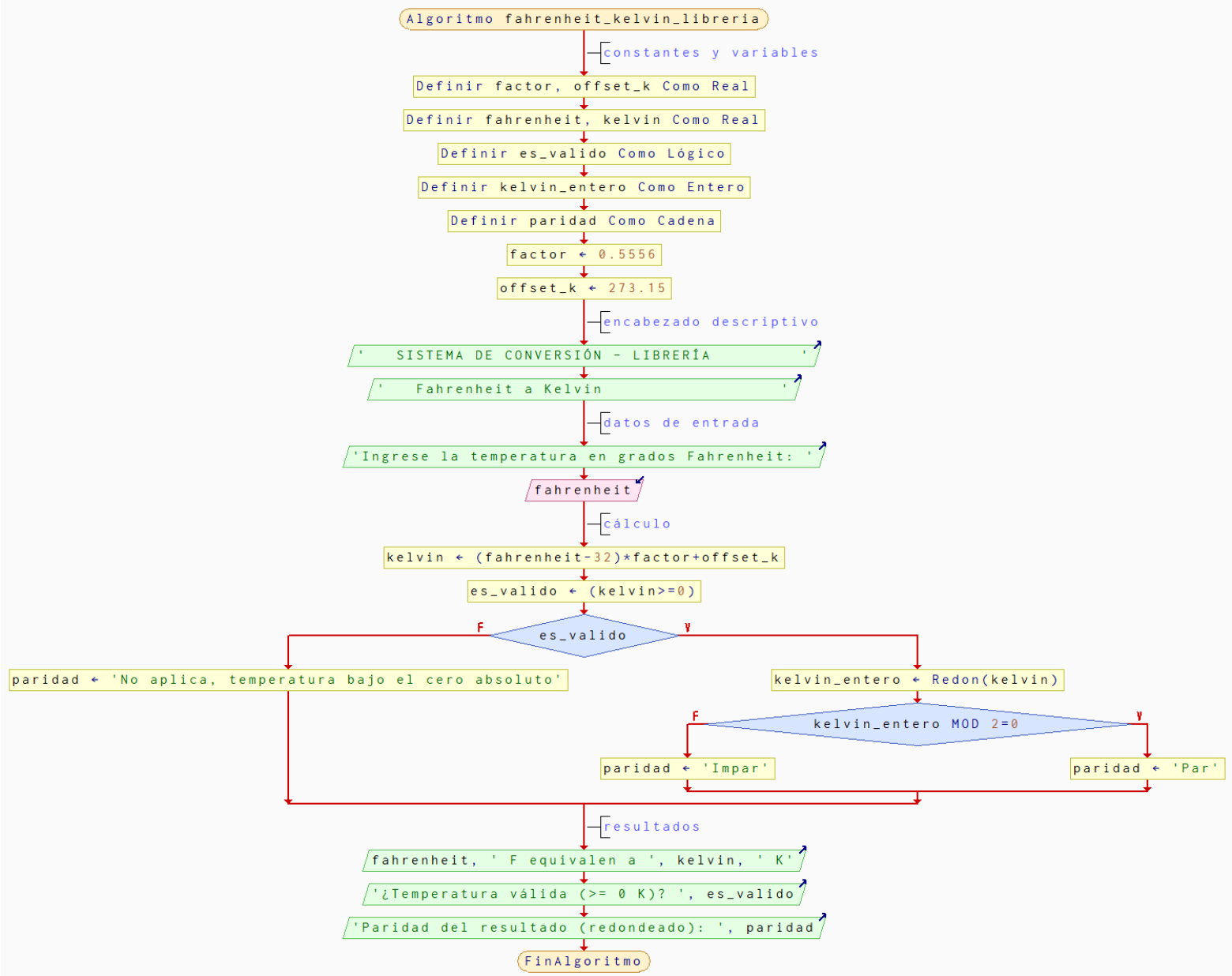
- **Variables:** fahrenheit
- **Constante:** factor = 0. 5556, offset_k = 273.15

Proceso

- kelvin = (fahrenheit - 32) * factor + offset_k.
- kelvin_valido = (kelvin >= 0).

Datos de salida: kelvin, kelvin_valido, paridad.

Diagrama de flujo



Pseudocódigo

Inicio
//Constantes y variables
factor = 0.5556
offset_k = 273.15
Definir fahrenheit, kelvin como real
Definir es_valido como booleano
Definir kelvin_entero como entero
Definir paridad como cadena
// Encabezado descriptivo
Imprimir "=="
Imprimir " SISTEMA DE CONVERSIÓN - LIBRERÍA "
Imprimir " Fahrenheit a Kelvin "
Imprimir "=="
//datos de entrada

```
Imprimir ("Ingrese la temperatura en grados Fahrenheit: ")
Leer fahrenheit
// Cálculo
kelvin = (fahrenheit - 32) * factor + offset_k
es_valido = (kelvin >= 0)
Si (es_valido) Entonces
    kelvin_entero = redondear(kelvin)
    Si (kelvin_entero % 2 == 0) Entonces
        paridad = "Par"
    Sino
       paridad = "Impar"
    FinSi
Sino
    paridad = "No aplica, temperatura bajo el cero absoluto"
FinSi
Imprimir "-----------------------------------------"
Imprimir (fahrenheit + " F equivalen a " + kelvin + " K")
Imprimir ("¿Temperatura válida (>= 0 K)? " + es_valido)
Imprimir ("Paridad del resultado (redondeado): " +paridad)
Imprimir "========================================="
Fin
```

Prueba de escritorio

Prueba 1

Paso	Instrucción	Calculo	Resultado
1	---	Se lee la temperatura	fahrenheit = 25.5
2	kelvin = (fahrenheit - 32.0f) * factor + offset_k;	(25.5 - 32) = -6.5 -6.5 * 0.5556 = -3.6114 -3.6114 + 273.15	269.5386
3	es_valido = (kelvin >= 0);	(269.5386 >= 0) es Verdadero	True
4	kelvin_entero = (int)roundf(kelvin);	Redondeo de 269.5386	270
5	kelvin_entero % 2 == 0	270 % 2 (residuo 0) -> Verdadero	Par

Prueba 2

Paso	Instrucción	Calculo	Resultado
1	---	Se lee la temperatura	fahrenheit = 20.4
2	kelvin = (fahrenheit -	(20.4 - 32) = -11.6	266.70504

	32.0f) * factor + offset_k;	-11.6 * 0.5556 = -6.44496 -6.44496 + 273.15	
3	es_valido = (kelvin >= 0);	(266.70504 >= 0) es Verdadero	True
4	kelvin_entero = (int)roundf(kelvin);	Redondeo de 266.70504	267
5	kelvin_entero % 2 == 0	267 % 2 (residuo 1) -> Falso	Impar

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
#include <math.h> // Para roundf
#define factor 0.5556
#define offset_k 273.15
int main() {
    // Inicialización de variables
    float fahrenheit, kelvin;
    bool es_valido;
    int kelvin_entero;
    const char *paridad = "No aplica";
    //Encabezado descriptivo
    printf("=============================================\n");
    printf("   SISTEMA DE CONVERSION - LIBRERIA         \n");
    printf("   Fahrenheit a Kelvin                      \n");
    printf("=============================================\n");
    //Entrada
    printf("Ingrese la temperatura en grados Fahrenheit: ");
    scanf("%f", &fahrenheit);
    //Cálculo
    kelvin = (fahrenheit - 32.0f) * factor + offset_k;
    es_valido = (kelvin >= 0);
    if (es_valido) {
        kelvin_entero = (int)roundf(kelvin);
        if (kelvin_entero % 2 == 0) {
            paridad = "Par";
        } else {
            paridad = "Impar";
        }
    } else {
        paridad = "No aplica (temperatura bajo el cero absoluto)";
    }
    printf("---------------------------------------------\n");
    printf("%.2f F equivalen a %.2f K\n", fahrenheit, kelvin);
    printf("Temperatura valida (>= 0 K)? %s\n", es_valido ? "true" : "false");
    printf("Paridad del resultado (redondeado): %s\n", paridad);
    printf("=============================================\n");
    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ16_449.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer16.exe
==========================================
   SISTEMA DE CONVERSION - LIBRERIA
   Fahrenheit a Kelvin
==========================================
Ingrese la temperatura en grados Fahrenheit: 25.5
------------------------------------------
25.50 F equivalen a 269.54 K
Temperatura valida (>= 0 K)? true
Paridad del resultado (redondeado): Par
==========================================
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\eje
==========================================
   SISTEMA DE CONVERSION - LIBRERIA
   Fahrenheit a Kelvin
==========================================
Ingrese la temperatura en grados Fahrenheit: 20.4
------------------------------------------
20.40 F equivalen a 266.71 K
Temperatura valida (>= 0 K)? true
Paridad del resultado (redondeado): Impar
==========================================
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El encabezado mejora la presentación. Dice qué hace el programa desde el inicio, lo que ayuda a quien lo ejecuta y facilita las capturas de la prueba

- Aprendí que la paridad (par o impar) es un concepto exclusivo de los números enteros. Como las temperaturas son números reales (con decimales), fue necesario usar la función roundf() para redondear el resultado

Enunciado del algoritmo

EJ-110 (Ejercicio 17)

En una agencia de viajes, diseñe un algoritmo que lea el ingreso mensual (real) y el gasto mensual (real). Calcule el ahorro con la expresión aritmética (ingreso - gasto) y asígnelo a ahorro. Use una constante simbólica PORCENTAJE_IDEAL = 0.20. Calcule el ahorro ideal con (ingreso * PORCENTAJE_IDEAL). Evalúe la expresión lógica (ahorro >= ahorro_ideal). Muestre por pantalla el ahorro y si cumple con el ideal. Adicionalmente, calcule la raíz cuadrada del valor absoluto del resultado. Verifique que las constantes estén declaradas al inicio del algoritmo.

Análisis del problema

Datos de entrada:

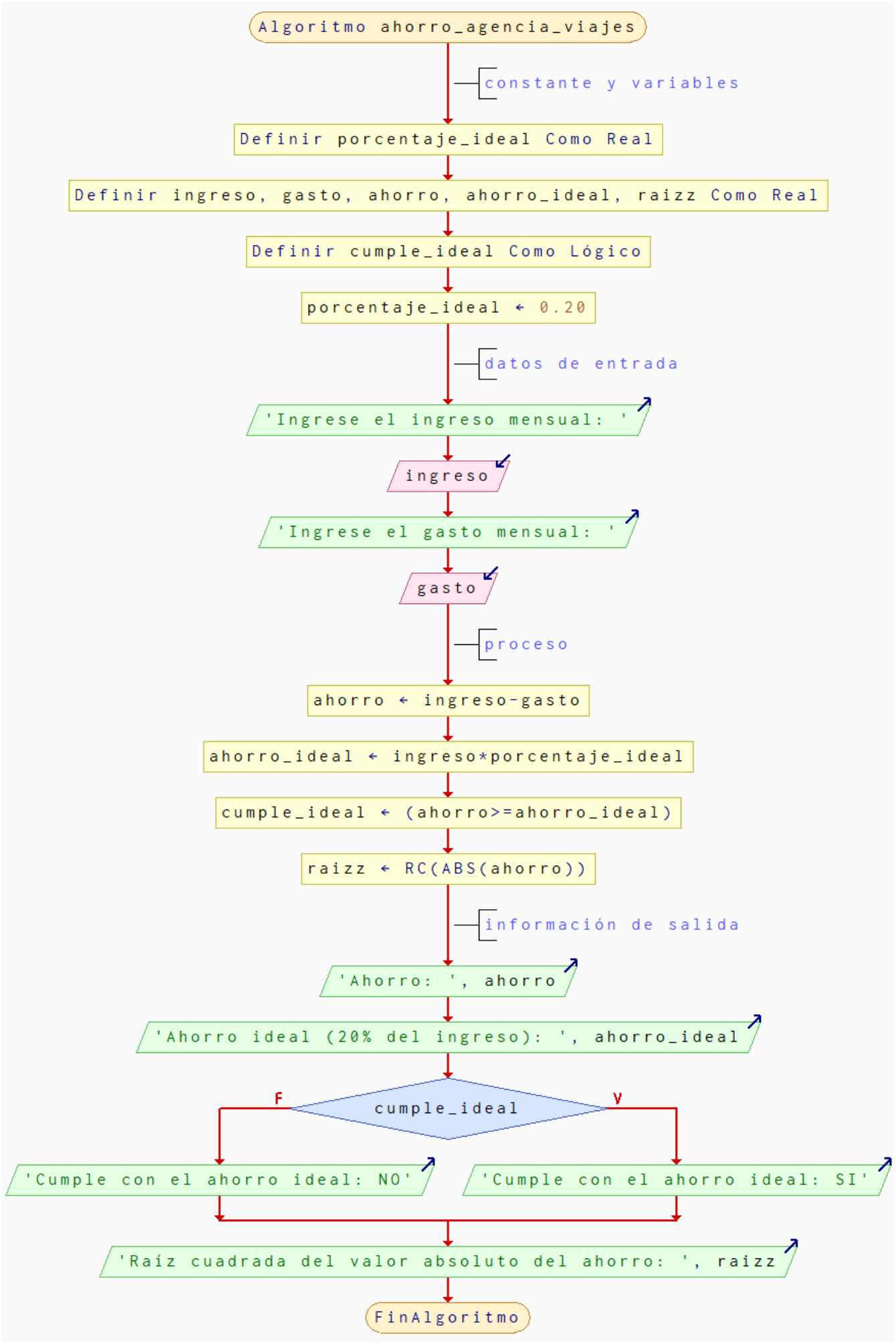
- **Variables:** ingreso, gasto
- **Constante:** porcentaje_ideal = 0.20

Proceso

- ahorro = ingreso - gasto.
- ahorro_ideal = ingreso * PORCENTAJE_IDEAL.
- cumple_ideal = (ahorro >= ahorro_ideal).

Datos de salida: ahorro, ahorro_ideal, cumple_ideal, raiz_ahorro.

Diagrama de flujo



Pseudocódigo

```
Inicio
//Constante y variables
porcentaje_ideal = 0.20
Definir ingreso, gasto, ahorro, ahorro_ideal, raiz Como Real
Definir cumple_ideal Como Logico
//Datos de entrada
Imprimir ("Ingrese el ingreso mensual: ")
Leer ingreso
Imprimir ("Ingrese el gasto mensual: ")
Leer gasto
//Proceso
ahorro = ingreso - gasto
ahorro_ideal = ingreso * porcentaje_ideal
cumple_ideal = (ahorro >= ahorro_ideal)
raiz = RC(ABS(ahorro))
//Informacion de salida
Imprimir ("Ahorro: " + ahorro)
Imprimir ("Ahorro ideal (20% del ingreso): " + ahorro_ideal)
Si (cumple_ideal) Entonces
        Imprimir ("Cumple con el ahorro ideal: SI")
SiNo
        Imprimir ("Cumple con el ahorro ideal: NO")
FinSi
Imprimir ("Raíz cuadrada del valor absoluto del ahorro: " + raiz)
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida
1	ingreso	400	---
2	gasto	20	---
3	ahorro (400 - 20)	380	---
4	ahorro_ideal (400 * 0.20)	80	---
5	ahorro >= ahorro_ideal	Verdadero	---
	raiz (raíz de |380|)	19.4936	**Ingrese el ingreso mensual:** 400 **Ingrese el gasto mensual:** 20 **Ahorro**: 380.00 **Ahorro ideal (20% del**

			ingreso): 80.00 **Cumple con el ahorro ideal**: SI **Raiz cuadrada del valor absoluto del ahorro:** 19.49

Prueba 2

Paso	Variable / expresión	Valor	Salida
1	ingreso	800	---
2	Gasto	900	---
3	ahorro (800 - 900)	-100	---
4	ahorro_ideal (800 * 0.20)	160	---
5	ahorro >= ahorro_ideal	Falso	---
	raiz (raíz de |-100|)	10	**Ingrese el ingreso mensual:** 800 **Ingrese el gasto mensual:** 900 **Ahorro**: -100.00 **Ahorro ideal (20% del ingreso):** 160.00 **Cumple con el ahorro ideal**: NO **Raiz cuadrada del valor absoluto del ahorro:** 10.00

Codificación en lenguaje C

```c
#include <stdio.h>
#include <math.h>
#include <stdbool.h>
// Constante simbolica declarada al inicio
int main(void) {
    const float porcentaje_ideal = 0.20;
    double ingreso, gasto, ahorro, ahorro_ideal, raiz;
    bool cumple_ideal;
    printf("Ingrese el ingreso mensual: ");
    scanf("%lf", &ingreso);
    printf("Ingrese el gasto mensual: ");
    scanf("%lf", &gasto);
    ahorro = ingreso - gasto;
    ahorro_ideal = ingreso * porcentaje_ideal;
    cumple_ideal = (ahorro >= ahorro_ideal);
    raiz = sqrt(fabs(ahorro));
    printf("Ahorro: %.2f\n", ahorro);
    printf("Ahorro ideal (20%% del ingreso): %.2f\n", ahorro_ideal);
    printf("Cumple con el ahorro ideal: %s\n", cumple_ideal ? "SI" : "NO");
    printf("Raiz cuadrada del valor absoluto del ahorro: %.2f\n", raiz);

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ17_110.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer17.ex
Ingrese el ingreso mensual: 400
Ingrese el gasto mensual: 20
Ahorro: 380.00
Ahorro ideal (20% del ingreso): 80.00
Cumple con el ahorro ideal: SI
Raiz cuadrada del valor absoluto del ahorro: 19.49
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer17.exe
Ingrese el ingreso mensual: 800
Ingrese el gasto mensual: 900
Ahorro: -100.00
Ahorro ideal (20% del ingreso): 160.00
Cumple con el ahorro ideal: NO
Raiz cuadrada del valor absoluto del ahorro: 10.00
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- El valor absoluto evita un error en la raíz. No se puede sacar la raíz cuadrada de un

número negativo. Al aplicar primero el valor absoluto, el cálculo funciona siempre.

- Cuando el gasto supera al ingreso, el ahorro es negativo, lo que en la significa que se está gastando más de lo que se gana.

Enunciado del algoritmo

EJ-139 (Ejercicio 18)

En una carnicería, construya un algoritmo que lea un ángulo en grados sexagesimales (real). Use la constante simbólica PI = 3.14159. Calcule la equivalencia en radianes con la expresión aritmética (grados * PI / 180) y asígnela a radianes. Evalúe la expresión lógica (grados >= 0 AND grados <= 360). Muestre por pantalla 'X grados equivalen a Y radianes'. Adicionalmente, calcule el cociente del resultado entre un factor de 1.5. Agregue una validación para asegurar que los datos sean positivos.

Análisis del problema

Datos de entrada:

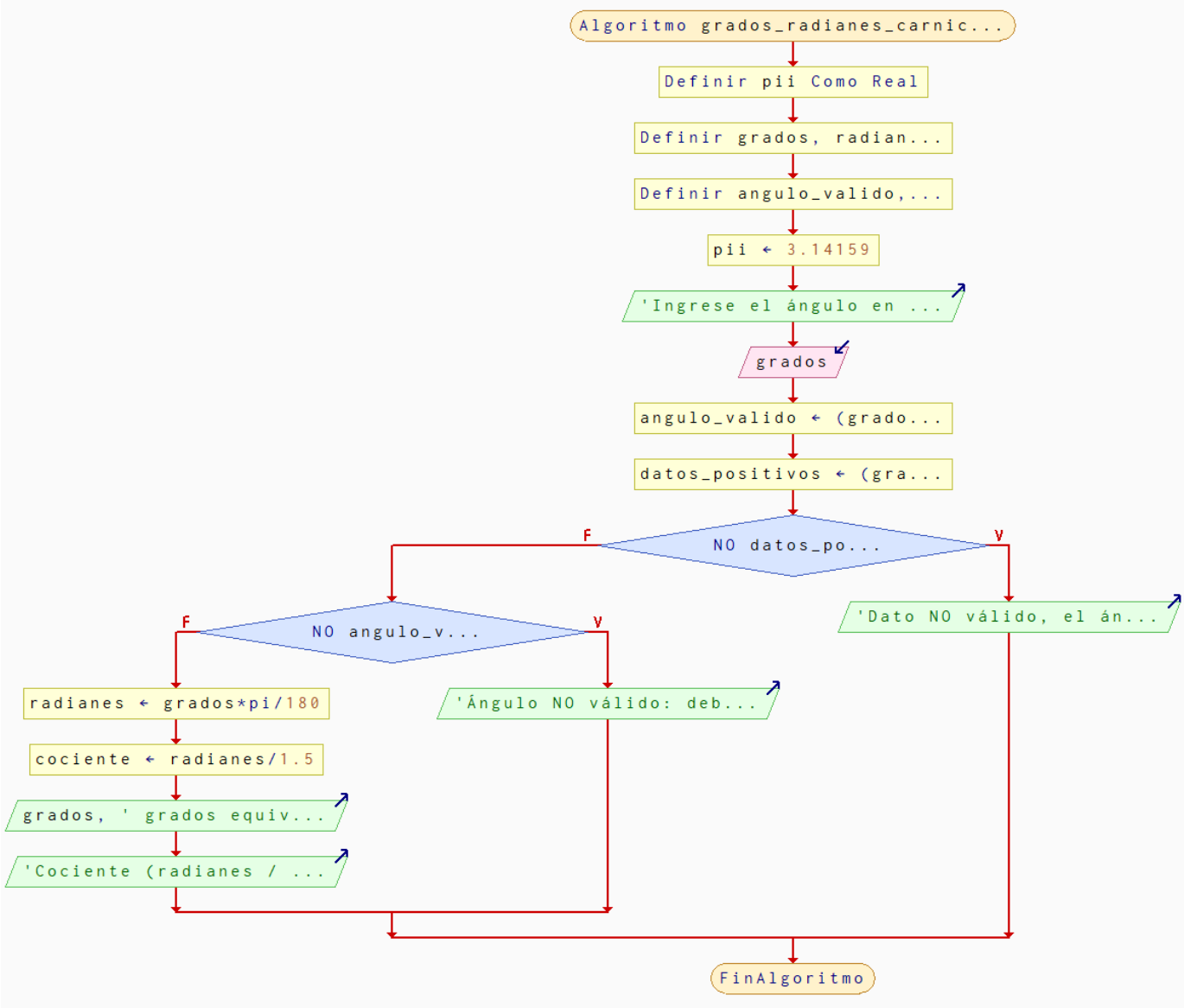
- **Variables:** grados
- **Constante:** pi = 3.14159

Proceso

- angulo_valido = (grados >= 0 AND grados <= 360).
- datos_positivos = (grados > 0).
- radianes = grados * pi / 180
- cociente = radianes / 1.5

Datos de salida: angulo_valido, datos_positivos, radianes, cociente

Diagrama de flujo



Pseudocódigo

Inicio
//constante y variables
pi = 3.14159
Definir grados, radianes, cociente Como Real
Definir angulo_valido, datos_positivos Como Logico
//Datos de entrada
Imprimir ("Ingrese el angulo en grados: ")
Leer grados
//calculos
angulo_valido = (grados >= 0) Y (grados <= 360)
datos_positivos = (grados > 0)
Si (!datos_positivos) Entonces
Imprimir ("Dato NO valido, el ángulo debe ser positivo")
SiNo
Si (!angulo_valido) Entonces
Imprimir ("Angulo NO valido: debe estar entre 0 y 360 grados")

```
        SiNo
        radianes = grados * pi / 180
        cociente = radianes / 1.5
        Imprimir (grados + " grados equivalen a " + radianes + " radianes")
        Imprimir ("Cociente (radianes / 1.5): " + cociente)
        FinSi
FinSi
Fin
```

Prueba de escritorio

Prueba 1

Paso	Variable / expresión	Valor	Salida
1	grados	40	---
2	datos_positivos (40 > 0)	Verdadero	---
3	angulo_valido (40 >= 0 y 40 <= 360)	Verdadero	---
4	radianes (40 * 3.14159 / 180)	0.69813	---
5	cociente (0.69813 / 1.5)	0.46542	**Ingrese el ángulo en grados:** 40 **40.00 grados equivalen a** 0.6981 radianes **Cociente (radianes / 1.5):** 0.4654

Prueba 2

Paso	Variable / expresión	Valor	Salida
1	grados	400	---
2	datos_positivos (400 > 0)	Verdadero	---
3	angulo_valido (400 >= 0 y 400 <= 360)	Falso	---
4	radianes	No se calcula	---
5	cociente	No se calcula	**Ingrese el ángulo en grados:** 400 **Ángulo NO valido: debe estar entre 0 y 360 grados**

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    // Constantes y variables
    const float pi = 3.14159;
    double grados, radianes, cociente;
    bool angulo_valido, datos_positivos;
    //pedir informacion
    printf("Ingrese el angulo en grados: ");
    scanf("%lf", &grados);
    //calculos
    angulo_valido = (grados >= 0) && (grados <= 360);
    datos_positivos = (grados > 0);
    if (!datos_positivos) {
        printf("Dato NO valido, el angulo debe ser positivo\n");
    } else if (!angulo_valido) {
        printf("Angulo NO valido, debe estar entre 0 y 360 grados\n");
    } else {
        radianes = grados * pi / 180;
        cociente = radianes / 1.5;
        printf("%.2f grados equivalen a %.4f radianes\n", grados, radianes);
        printf("Cociente (radianes / 1.5): %.4f\n", cociente);
    }

    return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ18_139.c

Salida del programa

Prueba 1

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\e
Ingrese el angulo en grados: 40
40.00 grados equivalen a 0.6981 radianes
Cociente (radianes / 1.5): 0.4654
```

Prueba 2

```
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC> .\ejer18.exe
Ingrese el angulo en grados: 400
Angulo NO valido, debe estar entre 0 y 360 grados
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Conclusiones del algoritmo

- La validación evita conversiones sin sentido, un ángulo negativo, igual a 0 o mayor que 360 no pasa.

- El cociente es un cálculo adicional simple. Dividir los radianes entre 1.5 no depende del tipo de ángulo

Enunciado del algoritmo

EJ-056 (Ejercicio 19) De los ejercicios anteriores

Construya un algoritmo que lea un monto en dólares (real) y lo convierta a euros y pesos. Use constantes simbólicas: USD_EUR = 0.92, USD_MXN = 17.50, USD_COP = 4100.0. Calcule cada conversión con expresiones aritméticas. Evalúe la expresión relacional (monto_usd > 0) y la lógica (monto_usd >= 100 AND monto_usd <= 1000). Asigne los resultados a variables reales. Muestre por pantalla todas las conversiones y los resultados booleanos. Justifique el uso de constantes frente a literales.

Análisis del problema

Datos de entrada:

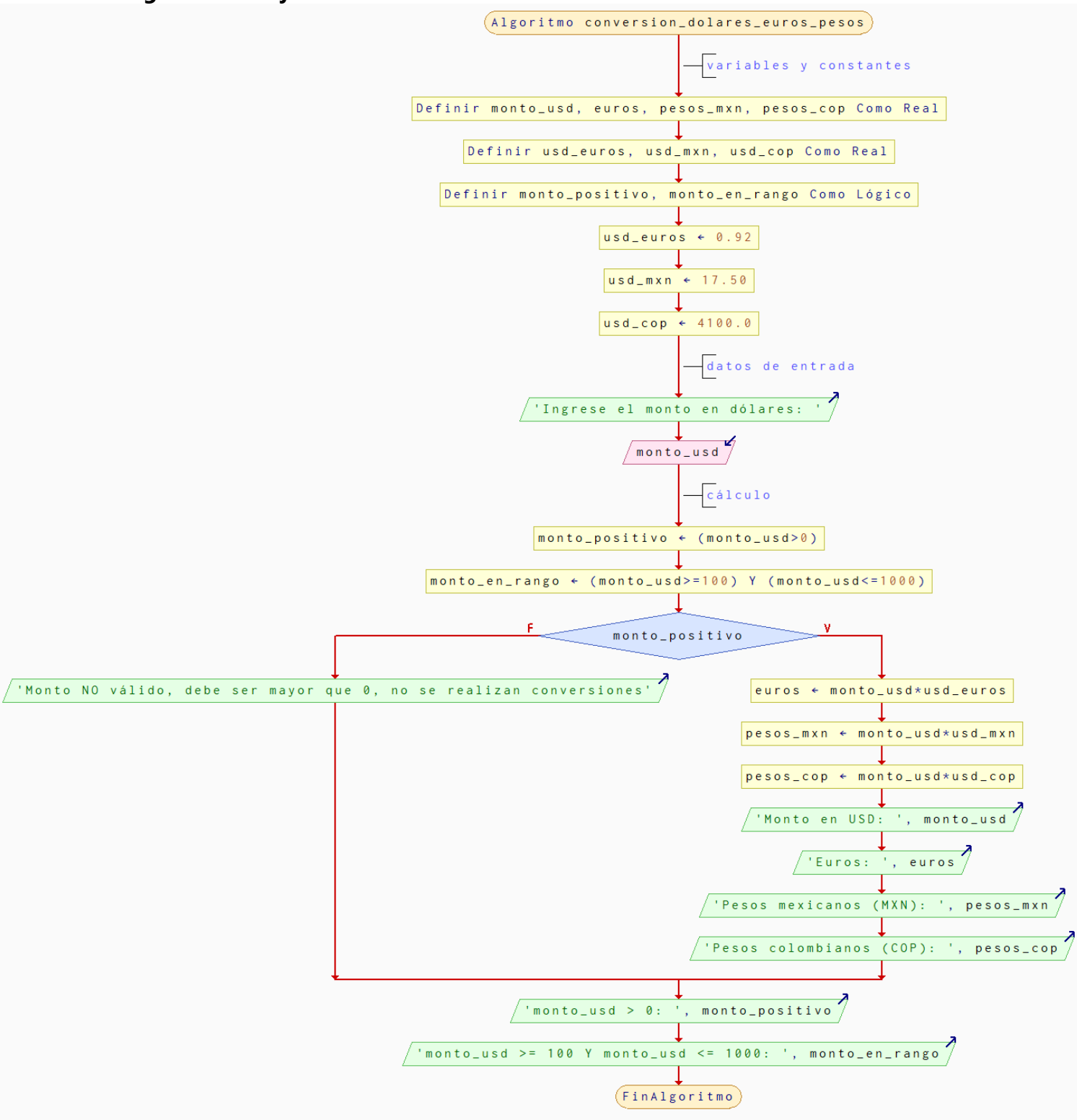
- **Constantes**: usd_euros = 0.92, usd_mxn= 17.50, usd_cop = 4100.0.
- **Variables**: monto_usd.

Proceso

- euros = monto_usd * usd_euros
- pesos_mxn = monto_usd * usd_mxn
- pesos_cop = monto_usd * usd_cop
- monto_positivo = (monto_usd > 0).
- monto_en_rango = (monto_usd >= 100 AND monto_usd <= 1000).

Datos de salida: euros, pesos_mxn, pesos_cop, monto_positivo, monto_en_rango.

Diagrama de flujo



Pseudocódigo

Inicio

//variables y constantes

Definir monto_usd, euros, pesos_mxn, pesos_cop como real

usd_euros = 0.92

usd_mxn= 17.50

usd_cop = 4100.0

Imprimir ("Ingrese el monto en dólares: ")

Leer monto_usd

//calculo

```
monto_positivo = (monto_usd > 0)
monto_en_rango = (monto_usd >= 100) Y (monto_usd <= 1000)
Si (monto_positivo) Entonces
        euros = monto_usd * usd_euros
        pesos_mxn = monto_usd * usd_mxn
        pesos_cop = monto_usd * usd_cop
        Imprimir ("Monto en USD: " + monto_usd )
        Imprimir ("Euros: " + euros )
        Imprimir ("Pesos mexicanos (MXN): " + pesos_mxn)
        Imprimir ("Pesos colombianos (COP): " + pesos_cop)
SiNo
        Imprimir ("Monto NO valido, debe ser mayor que 0, no se realizan conversiones")
FinSi
Imprimir ("monto_usd > 0: " + monto_positivo )
Imprimir ("monto_usd >= 100 Y monto_usd <= 1000: " + monto_en_rango)
Fin
```

Prueba de escritorio

Prueba 1

monto_usd	**usd_euros**	**usd_mxn**	**usd_cop**
250	0.92	17.50	4100.0
250	**euros**	**pesos_mxn**	**pesos_cop**
250	230.00 EUR	4375.00 MXN	1025000.00 COP

Evaluacion de condiciones	**Resultado_relacional (**¿El monto es mayor a 0?**)**	**resultado_logico (**¿El monto esta ente 100 y 1000 USD?**)**
250	Verdadero	Verdadero

Prueba 2

monto_usd	**usd_euros**	**usd_mxn**	**usd_cop**
67	0.92	17.50	4100.0
67	**euros**	**pesos_mxn**	**pesos_cop**
67	61.64 EUR	1172.50 MXN	274700.00 COP

Evaluacion de condiciones (monto_usd)	**Resultado_relacional (**¿El monto es mayor a 0?**)**	**Resultado_logico (**¿El monto esta ente 100 y 1000 USD?**)**
250	Verdadero	Falso

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>
// Constantes
#define USD_EUR 0.92
#define USD_MXN 17.50
#define USD_COP 4100.0
int main(void) {
    //variables
    double monto_usd, euros, pesos_mxn, pesos_cop;
    bool monto_positivo, monto_en_rango;
    //pedir informacion
    printf("Ingrese el monto en dolares: ");
    scanf("%lf", &monto_usd);
    //calculos
    monto_positivo = (monto_usd > 0);
    monto_en_rango = (monto_usd >= 100) && (monto_usd <= 1000);
    if (monto_positivo) {
        euros = monto_usd * USD_EUR;
        pesos_mxn = monto_usd * USD_MXN;
        pesos_cop = monto_usd * USD_COP;
        printf("Monto en USD: %.2f\n", monto_usd);
        printf("Euros: %.2f\n", euros);
        printf("Pesos mexicanos (MXN): %.2f\n", pesos_mxn);
        printf("Pesos colombianos (COP): %.2f\n", pesos_cop);
    } else {
        printf("Monto NO valido, debe ser mayor que 0, no se realizan conversiones\n");
    }
    printf("monto_usd > 0: %s\n", monto_positivo ? "verdadero" : "falso");
    printf("monto_usd >= 100 Y monto_usd <= 1000: %s\n", monto_en_rango ? "verdadero" : "falso");
    return 0;
}
```

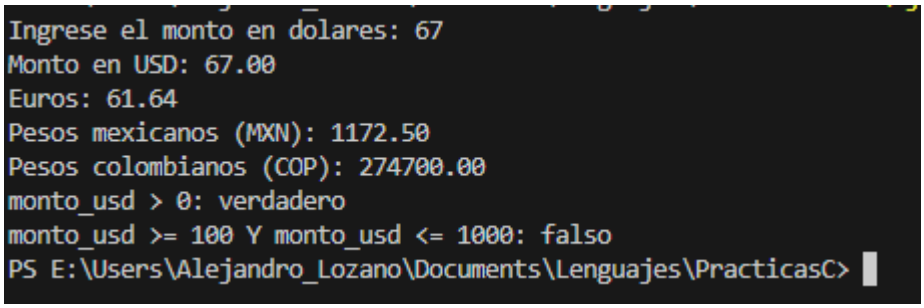
Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ19_56.c

Salida del programa

Prueba 1:

```
Ingrese el monto en dolares: 250
Monto en USD: 250.00
Euros: 230.00
Pesos mexicanos (MXN): 4375.00
Pesos colombianos (COP): 1025000.00
monto_usd > 0: verdadero
monto_usd >= 100 Y monto_usd <= 1000: verdadero
PS E:\Users\Alejandro_Lozano\Documents\Lenguajes\PracticasC>
```

Prueba 2



Conclusiones del algoritmo

- Los resultados booleanos se muestran siempre. Así se puede comprobar cómo se evaluó cada expresión, incluso cuando el monto no es válido.
- El algoritmo cumple lo que pide el enunciado. Lee un monto en dólares, lo convierte a euros, pesos mexicanos y pesos colombianos con constantes, evalúa las dos expresiones y muestra las conversiones y los resultados booleanos.

Enunciado del algoritmo

EJ-041 (Ejercicio 20) De los ejercicios anteriores
En un cine, construya un algoritmo que lea un volumen en galones (real). Use una constante simbólica GALON_LITRO = 3.78541. Calcule la equivalencia en litros con la expresión aritmética (galones * GALON_LITRO) y asígnela a litros. Evalúe la expresión relacional (litros > 0). Muestre por pantalla 'X galones equivalen a Y litros'. Además del cálculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operación realizada.

Análisis del problema
Datos de entrada:

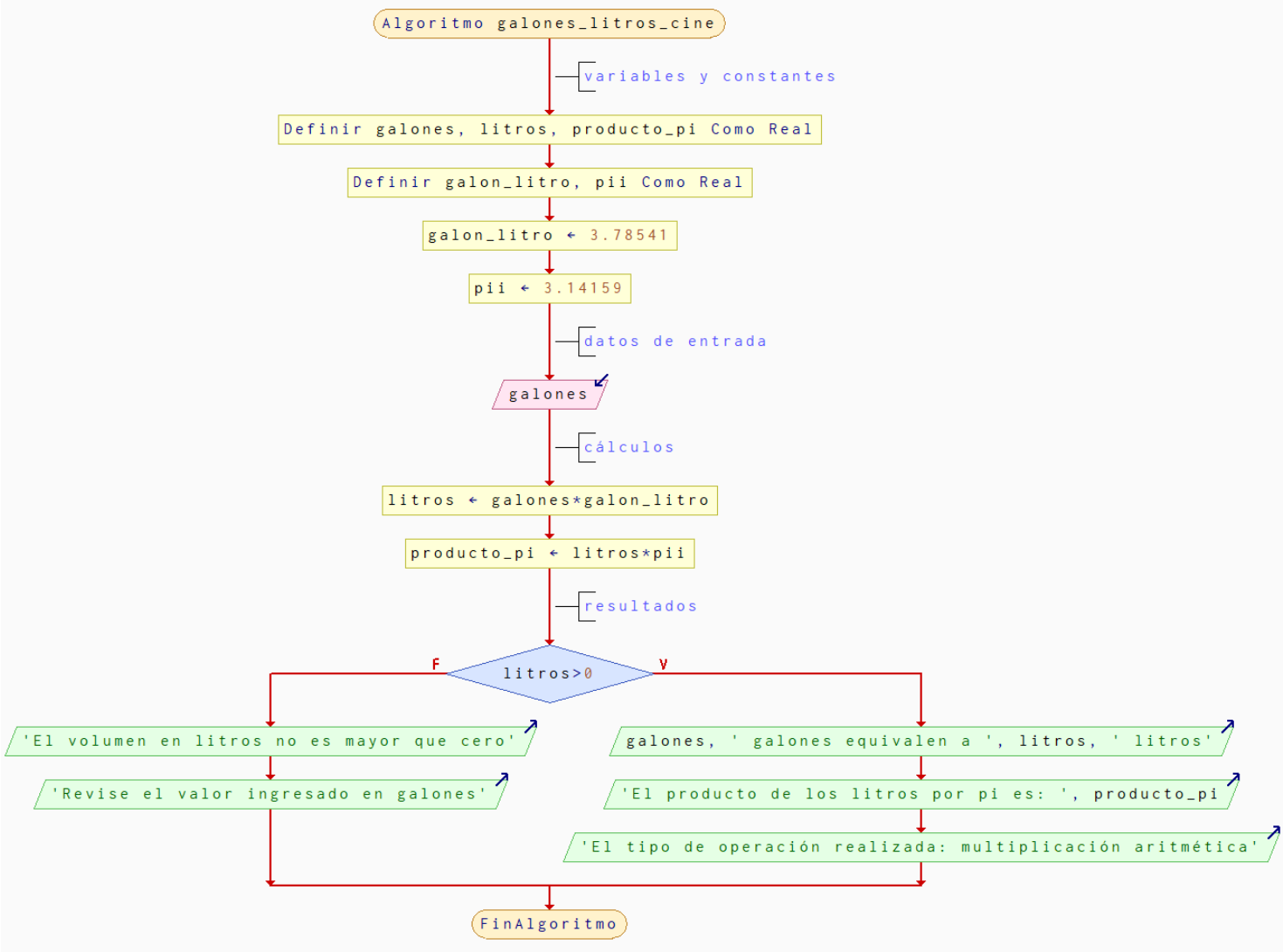
- **Constante:** galon_litro = 3.78541, pi = 3.14159
- **Variable:** galones

Proceso:

- litros = galones * galon_litro
- producto_pi = litros * PI

Datos de salida: litros, producto_pi

Diagrama de flujo



Pseudocódigo

Inicio
Definir galones, litros, producto_pi como real
galon_litro = 3.78541
pi = 3.14159
Leer galones
litros = galones * galon_litro
producto_pi = litros * pi
Si (litros > 0) entonces
Imprimir (galones + “galones equivales a” + litros + “litros”)
Imprimir (“El producto de los litros por pi es:” + producto_pi)
Imprimir (“El tipo de operación realizada: multiplicación aritmetica”)
Sino

Imprimir (“El volumen en litros no es mayor que cero”)
Imprimir (“Revise el valor ingresado en galones”)

Fin si

Fin

Prueba de escritorio

Prueba 1

Pasos	**galones**	**Galon_litro**	**Litros (**litros = galones * galon_litro**)**	**pi**	**producto_pi (**producto_pi = litros * pi**)**
1	0	3.78541	0	3.14159	0
2	5	3.78541	18.92705	3.14159	59.46103

Paso	**Si (litros > 0)**	**else**
3	• 5 galones equivalen a 18.92705 litros • El producto de los litros por pi es 59.46103 El tipo de operación realizada: multiplicación aritmética	Falso

Prueba 2

Pasos	**galones**	**Galon_litro**	**Litros (**litros = galones * galon_litro**)**	**pi**	**producto_pi (**producto_pi = litros * pi**)**
1	0	3.78541	0	3.14159	0
2	0	3.78541	0	3.14159	0

Paso	**Si (litros > 0)**	**else**
3	Falso	El volumen en litros no es mayor que cero • Revise el valor ingresado en galones

Codificación en lenguaje C

```c
#include <stdio.h>
int main (){
    //definir variable
    const double galon_litro = 3.78541;
    const double pi = 3.14159;
    float galones, litros, producto_pi ;
    //pedir datos
    printf("Ingrese el volumen en galones: ");
    scanf("%f", &galones);
    //Operaciones
    litros = galones * galon_litro;
    producto_pi = litros * pi;
    //Funcion lógica
    if(litros > 0){
        printf("%.2f galones equivale a %.5f litros.\n", galones, litros);
        printf("El producto de los litros por PI es: %.5f\n", producto_pi);
        printf("Tipo de operacion realizada: multiplicacion aritmetica.\n");
    }else{
        printf("El volumen en litros no es mayor que cero.\n");
        printf("Revise el valor ingresado en galones.\n");
    }
return 0;
}
```

Código en github: https://github.com/daniela-aleja/TeoriaProgramacion/blob/main/Portafolio_programacion/primera_unidad/trabajo_final/EJ20_041.c

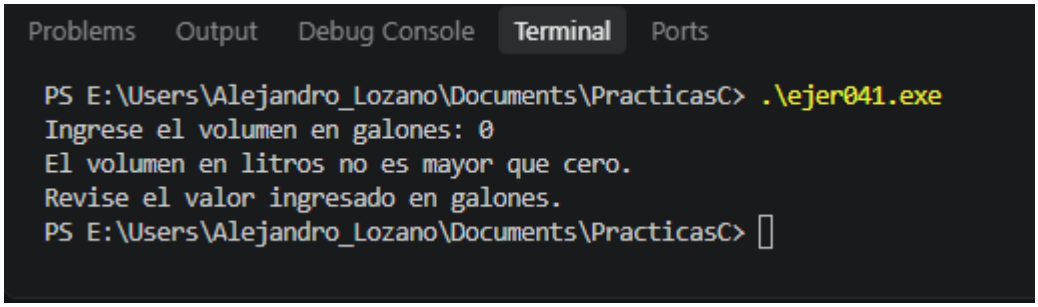
Salida del programa

Prueba 1:

```
Problems  Output  Debug Console  Terminal  Ports

Ingrese el volumen en galones: 5
5.00 galones equivale a 18.92705 litros.
El producto de los litros por PI es: 59.46103
Tipo de operacion realizada: multiplicacion aritmetica.
PS E:\Users\Alejandro_Lozano\Documents\PracticasC>
```

Prueba 2:



Conclusiones del algoritmo

- Los tipos de dato son reales. Los galones, los litros y el producto pueden tener decimales, y la variable litros_positivo es lógica porque solo guarda verdadero o falso.
- El mensaje final deja claro qué se hizo. Indica que el algoritmo realizó una conversión de unidades y un producto por PI, lo que ayuda a entender la salida sin leer el código.

Portafolio: Teoría de la programación, primera unidad: https://github.com/daniela-aleja/TeoriaProgramacion/tree/main/Portafolio_programacion/primera_unidad

1. Rúbrica de evaluación (sobre 10 puntos)

Se aplica a cada algoritmo desarrollado.

Criterio	Excelente	Bueno	Regular	Insuficiente	Punt.
Análisis del problema (0,5 pts)	**(0,5)** Identifica correctamente entradas, proceso, salidas y variables con su tipo de dato.	**(0,4)** Identifica entradas, proceso y salidas con errores mínimos.	**(0,2)** Identifica parcialmente los elementos del análisis.	**(0)** No presenta el análisis o este es incorrecto.	
Diagrama de flujo (2,0 pts)	**(2,0)** Simbología correcta y flujo lógico completo; el algoritmo resuelve el problema.	**(1,5)** Simbología mayormente correcta; lógica con pequeños errores que no afectan el resultado.	**(1,0)** Errores de simbología y/o lógica que afectan parcialmente la solución.	**(0–0,5)** Diagrama incompleto, con lógica incorrecta o no presenta.	
PUNTAJE TOTAL POR ALGORITMO					**/ 10**

Criterio	Excelente	Bueno	Regular	Insuficiente	Punt.
Pseudocódigo (1,5 pts)	**(1,5)** Estructura completa y sintaxis adecuada; corresponde al diagrama de flujo.	**(1,2)** Estructura completa con errores menores de sintaxis.	**(0,8)** Estructura incompleta o con errores de sintaxis frecuentes.	**(0–0,4)** Pseudocódigo incorrecto o no presenta.	
Prueba de escritorio (2,0 pts)	**(2,0)** Tabla completa, ordenada y con resultados correctos; usa mínimo dos casos de prueba.	**(1,5)** Tabla completa con algún error de cálculo menor.	**(1,0)** Tabla incompleta o con varios errores que no validan el algoritmo.	**(0–0,5)** No presenta la prueba de escritorio o está totalmente errada.	
Codificación en C (2,0 pts)	**(2,0)** Código funcional, compilado, comentado, con buenas prácticas e indentación.	**(1,5)** Código funcional con errores menores de estilo o comentarios.	**(1,0)** Código con errores que impiden compilar o lógica incorrecta.	**(0–0,5)** Código no funcional o no presenta.	
Salida del programa (1,0 pto)	**(1,0)** Incluye captura de la ejecución; los resultados coinciden con la prueba de escritorio.	**(0,8)** Incluye captura con diferencias mínimas respecto a la prueba de escritorio.	**(0,5)** Incluye captura pero no coincide con la prueba de escritorio.	**(0–0,3)** No incluye evidencia de la ejecución.	
Conclusiones (0,5 pts)	**(0,5)** Conclusiones bien redactadas, claras y coherentes; de elaboración propia y reflejan aprendizaje y reflexión sobre el proceso.	**(0,4)** Conclusiones adecuadas con errores menores de redacción u ortografía.	**(0,2)** Conclusiones superficiales, muy breves o con deficiencias de redacción.	**(0)** No presenta conclusiones, son copiadas o generadas con IA.	
Presentación y redacción (0,5 pts)	**(0,5)** Documento ordenado, con portada, formato claro y redacción adecuada.	**(0,4)** Documento ordenado con errores menores de formato.	**(0,2)** Documento desordenado o con deficiencias de formato.	**(0)** Documento sin estructura ni presentación adecuada.	
PUNTAJE TOTAL POR ALGORITMO					**/ 10**

Escala de calificación

9,0 – 10	Domina el desarrollo de algoritmos

8,0 – 8,9	Alcanza los aprendizajes esperados
7,0 – 7,9	Está próximo a alcanzar los aprendizajes
Menos de 7,0	No alcanza los aprendizajes esperados

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**